Lean-Backed NS Interface Note

May 21, 2026

Abstract

This note records only the Navier–Stokes material currently backed by Lean in the MeTTapedia repository. It intentionally omits manuscript-specific claims about Ben Goertzel's December 11, 2025 SG-Cole-Hopf draft unless they have been encoded and checked in Lean.

The present Lean status is interface-level rather than proof-complete. The formalized material isolates reusable shells for cutoff continuity, topological Cole–Hopf transport, approximation interfaces, finite identity-energy algebra, abstract vorticity bounds, a kernel-semigroup wrapper, and two small concrete models showing both positive approximation instantiations and wrong topology obstructions, including one exact same-state-space split where a summable geometric observable is continuous and a stronger tail detector is not. It also now contains an abstract good observable class, an abstract tail-spike no-go theorem, and a first concrete soft-vs-hard chart fork on that same shared mode space, together with more manuscript-shaped Sobolev-style and energy-style soft-vs-hard forks on that same space, plus a log-energy fork showing that a smoother radial profile still lands on the same good/bad split, and finally a generic energy-profile theorem packaging that split for any continuous radial profile dominated by the identity on nonnegative inputs. It also now contains a theorem-statement-back interface from a local Fefferman-style global-regularity target to the exact uniform vorticity-control tendril currently produced by the grassroots packages, together with a constructive top-down bridge interface decomposing the missing lift into candidate fields, regularity, dynamics, energy, and vorticity-compatibility layers. A new concrete equation-target file now fixes time $\mathbb{R}$, space $\mathbb{R}^3$, and the literal NS equation surface $\partial_t u + (u \cdot \nabla)u + \nabla p = \nu \Delta u$, $\operatorname{div} u = 0$, $u(0, \cdot) = u_0$, and records the honest one-way map from a full vector-valued NS witness to the scalar component shadow currently seen by the Fefferman-style bridge surface. It also now makes one PDE-side slot genuinely concrete: the divergence operator on $\mathbb{R}^3$ is defined as the trace of the spatial Fréchet derivative on the standard basis, and divergence-free initial data are defined by the corresponding vanishing condition. It now also makes the pressure-gradient slot concrete in the same way: the spatial pressure gradient is the actual gradient on $\mathbb{R}^3$, and the momentum equation in the more concrete constructor literally uses that operator. The newest pass also makes the time-derivative slot concrete: $\partial_t u$ is now defined by the actual time Fréchet derivative in the unit time direction, so the current most concrete constructor literally packages $\partial_t u$, ∇p, and $\operatorname{div} u$ on $\mathbb{R} \times \mathbb{R}^3$. It now also makes the convection slot concrete: $(u \cdot \nabla)u$ is defined as the spatial Fréchet derivative applied to the current velocity vector, so the current most concrete constructor now packages the full left-hand side of the NS momentum equation literally, leaving only the Laplacian and analytic regularity slots abstract. It now also makes the Laplacian slot concrete using mathlib's actual Laplace operator on finite-dimensional inner-product spaces, so the current most concrete constructor packages the full differential operator content of the NS equation on $\mathbb{R} \times \mathbb{R}^3$, leaving only the analytic regularity and energy predicates abstract. The next pass also fixes the smoothness predicates themselves: velocity and pressure are now required to be actual C^∞ maps on $\mathbb{R} \times \mathbb{R}^3$, and initial velocity data are actual C^∞ maps on $\mathbb{R}^3$. The next strengthening then also fixes bounded energy to a literal uniform bound on the spatial kinetic-energy integrals $\int_{\mathbb{R}^3} |u(t,x)|^2 \, dx$, so what remains abstract is no longer the theorem surface but the actual existence/regularity bridge. The next analytic pass then adds a first concrete continuation surface on top of that explicit theorem

statement: finite-time smooth witnesses on slabs $0 \le t \le T$, a literal curl/vorticity operator on $\mathbb{R}^3$, and a uniform-vorticity continuation target saying that such a slab witness should extend to a global one. The next sharpening then adds a closer-to-BKM continuation target using an explicit nonnegative time-envelope $\Omega(t)$ for $\|\omega(t,\cdot)\|$ which is integrable on $[0,T]$. The newest pass also specializes that compatibility frontier back down to the concrete windowed Cole–Hopf heat route: a genuine smooth window still survives the present approximation and heat-envelope shells, and the resulting scaled and affine seed-blend descendants already satisfy bridge-level rigidity theorems for original same-route closure. It does *not* yet formalize the manuscript's specific chart, local hypotheses $(H1)$, $(H2)$, or the BKM-closing analytic bridge.

1 Scope

The Navier–Stokes Lean work currently lives in:

1. `FluidDynamics/NavierStokes/CutoffContinuity.lean`,
2. `FluidDynamics/NavierStokes/WeakTopologyCountermodel.lean`,
3. `FluidDynamics/NavierStokes/ProductTopologyL1Countermodel.lean`,
4. `FluidDynamics/NavierStokes/ColeHopfTopology.lean`,
5. `FluidDynamics/NavierStokes/ApproximationInterface.lean`,
6. `FluidDynamics/NavierStokes/ChartApproximationModel.lean`,
7. `FluidDynamics/NavierStokes/GeometricModeApproximation.lean`,
8. `FluidDynamics/NavierStokes/ModeStateProductTopologyObstruction.lean`,
9. `FluidDynamics/NavierStokes/ModeStateProductTopologyContinuity.lean`,
10. `FluidDynamics/NavierStokes/ModeStateObservableClass.lean`,
11. `FluidDynamics/NavierStokes/ModeStateTailSensitivityObstruction.lean`,
12. `FluidDynamics/NavierStokes/ModeStateChartFork.lean`,
13. `FluidDynamics/NavierStokes/ModeStateSobolevChartFork.lean`,
14. `FluidDynamics/NavierStokes/ModeStateEnergyChartFork.lean`,
15. `FluidDynamics/NavierStokes/ModeStateLogEnergyChartFork.lean`,
16. `FluidDynamics/NavierStokes/ModeStateEnergyProfileFork.lean`,
17. `FluidDynamics/NavierStokes/FeffermanTargetBack.lean`,
18. `FluidDynamics/NavierStokes/FeffermanTopDownBridge.lean`,
19. `FluidDynamics/NavierStokes/NavierStokesEquationTarget.lean`,
20. `FluidDynamics/NavierStokes/NavierStokesComponentShadowIncoherence.lean`,
21. `FluidDynamics/NavierStokes/NavierStokesSharedWitnessComponent.lean`,

22. FluidDynamics/NavierStokes/NavierStokesSharedWitnessBridgeObstruction.lean,
23. FluidDynamics/NavierStokes/FeffermanCompatibilityFrontier.lean,
24. FluidDynamics/NavierStokes/FeffermanWeakSelfModel.lean,
25. FluidDynamics/NavierStokes/FeffermanSeededSelfModel.lean,
26. FluidDynamics/NavierStokes/FeffermanBoundedSeededSelfModel.lean,
27. FluidDynamics/NavierStokes/FeffermanPressureSeededSelfModel.lean,
28. FluidDynamics/NavierStokes/FeffermanScaledPressureSeededFrontier.lean,
29. FluidDynamics/NavierStokes/FeffermanSeedBlendPressureFrontier.lean,
30. FluidDynamics/NavierStokes/FeffermanSaturatedPressureFrontier.lean,
31. FluidDynamics/NavierStokes/FeffermanFrozenGatePressureFrontier.lean,
32. FluidDynamics/NavierStokes/FeffermanSeedLiveProfileFrontier.lean,
33. FluidDynamics/NavierStokes/FeffermanSeedLiveOperatorFrontier.lean,
34. FluidDynamics/NavierStokes/FeffermanSeedLiveSampleKernelFrontier.lean,
35. FluidDynamics/NavierStokes/FeffermanSeedLiveShiftKernelFrontier.lean,
36. FluidDynamics/NavierStokes/WindowedCutoffApproximation.lean,
37. FluidDynamics/NavierStokes/WindowedColeHopfHeatPackage.lean,
38. FluidDynamics/NavierStokes/WindowedColeHopfHeatFoundationFrontier.lean,
39. FluidDynamics/NavierStokes/WindowedColeHopfHeatSeededModel.lean,
40. FluidDynamics/NavierStokes/WindowedColeHopfHeatScaledSeededFrontier.lean,
41. FluidDynamics/NavierStokes/WindowedColeHopfHeatSelfCompatibilityObstruction.lean,
42. FluidDynamics/NavierStokes/WindowedColeHopfHeatSeedBlendFrontier.lean,
43. FluidDynamics/NavierStokes/WindowedColeHopfHeatFrozenGateFrontier.lean,
44. FluidDynamics/NavierStokes/WindowedColeHopfHeatFrozenGateObstruction.lean,
45. FluidDynamics/NavierStokes/WindowedColeHopfHeatSeedLiveProfileFrontier.lean,
46. FluidDynamics/NavierStokes/WindowedColeHopfHeatSeedLiveProfileObstruction.lean,
47. FluidDynamics/NavierStokes/WindowedColeHopfHeatSeedLiveOperatorFrontier.lean,
48. FluidDynamics/NavierStokes/WindowedColeHopfHeatSeedLiveOperatorObstruction.lean,
49. FluidDynamics/NavierStokes/WindowedColeHopfHeatSeedOnlyOperatorObstruction.lean,
50. FluidDynamics/NavierStokes/WindowedColeHopfHeatLiveOnlySampleKernelObstruction.lean,

51. FluidDynamics/NavierStokes/WindowedColeHopfHeatSampleKernelFrontier.lean,
52. FluidDynamics/NavierStokes/WindowedColeHopfHeatShiftKernelFrontier.lean,
53. FluidDynamics/NavierStokes/WindowedColeHopfHeatSeedBlendObstruction.lean,
54. FluidDynamics/NavierStokes/WindowedColeHopfHeatSampleKernelObstruction.lean,
55. FluidDynamics/NavierStokes/WindowedColeHopfHeatShiftKernelObstruction.lean,
56. FluidDynamics/NavierStokes/WindowedColeHopfHeatMaskedShiftKernelObstruction.lean,
57. FluidDynamics/NavierStokes/WindowedColeHopfHeatGeneralShiftKernelObstruction.lean,
58. FluidDynamics/NavierStokes/WindowedColeHopfHeatGeneralShiftKernelMaskedSpecialization.lean,
59. FluidDynamics/NavierStokes/WindowedColeHopfHeatGeneralShiftKernelIdentitySampleSpecializatio
60. FluidDynamics/NavierStokes/WindowedColeHopfHeatIdentitySampleKernelObstruction.lean,
61. FluidDynamics/NavierStokes/WindowedColeHopfHeatSaturatedFrontier.lean,
62. FluidDynamics/NavierStokes/WindowedColeHopfHeatSaturatedObstruction.lean,
63. FluidDynamics/NavierStokes/WindowedColeHopfHeatSaturatedSampleKernelObstruction.lean,
64. FluidDynamics/NavierStokes/WindowedColeHopfHeatSaturatedMaskedShiftKernelObstruction.lean,
65. FluidDynamics/NavierStokes/WindowedColeHopfHeatSaturatedShiftKernelObstruction.lean,
66. FluidDynamics/NavierStokes/WindowedColeHopfHeatSaturatedShiftKernelConstantMagnitudeFrontier
67. FluidDynamics/NavierStokes/WindowedColeHopfHeatSaturatedShiftKernelInvariantFrontier.lean,
68. FluidDynamics/NavierStokes/WindowedColeHopfHeatSaturatedShiftKernelInvariantObstruction.lean
69. FluidDynamics/NavierStokes/WindowedColeHopfHeatSaturatedAnchoredShiftInvariantFrontier.lean,
70. FluidDynamics/NavierStokes/WindowedColeHopfHeatSaturatedAnchoredShiftInvariantObstruction.le
71. FluidDynamics/NavierStokes/WindowedColeHopfHeatSaturatedShiftKernelInvariantAnchoredSpeciali
72. FluidDynamics/NavierStokes/WindowedColeHopfHeatSaturatedShiftKernelMaskedSpecialization.lean
73. FluidDynamics/NavierStokes/WindowedColeHopfHeatSaturatedShiftKernelIdentitySampleSpecializat
74. FluidDynamics/NavierStokes/WindowedColeHopfHeatSaturatedIdentitySampleKernelObstruction.lean
75. FluidDynamics/NavierStokes/IdentityEnergy.lean,
76. FluidDynamics/NavierStokes/ColeHopfPositiveLowerBound.lean,
77. FluidDynamics/NavierStokes/ColeHopfInterface.lean,
78. FluidDynamics/NavierStokes/KernelSemigroupInterface.lean.

These modules form the live NS interface shell. The currently verified slice is recorded explicitly in Section 6. Together they provide a formal shell for what a future SG-Cole-Hopf proof attempt would need to instantiate.

2 Lean-Backed Results

2.1 Cutoff Continuity Shell

In `CutoffContinuity.lean`, the generic cutoff-shaped potential

$$\text{cutoffPotential}(x) = \text{cutoff}(\text{radius}(x)) \cdot \text{statistic}(x)$$

is formalized together with:

1. continuity of the cutoff potential when the scalar pieces are continuous;
2. a sequential obstruction theorem: if the cutoff factor and statistic converge to the wrong product, then the cutoff potential cannot converge to its claimed value;
3. the corresponding pointwise non-continuity criterion.

So Lean already captures the generic shape of a chart-cutoff continuity argument and its failure mode.

2.2 Concrete Weak-Topology Countermodel

In `WeakTopologyCountermodel.lean`, the abstract cutoff/Cole–Hopf shell is instantiated on the concrete state space $\mathbb{R} \times \mathbb{R}$ equipped with the topology induced by the first coordinate. Lean proves:

1. a sequence can converge in the ambient topology while changing a hidden coordinate;
2. a cutoff potential depending on that hidden coordinate is not continuous;
3. the associated Cole–Hopf transform is likewise not continuous.

This is a formal countermodel for the generic wrong-topology failure mode: ambient convergence can be too weak for the cutoff datum it is supposed to control.

2.3 Infinite-Mode Product-Topology Countermodel

In `ProductTopologyL1Countermodel.lean`, Lean moves the same mismatch to the raw infinite-mode sequence space $\mathbb{N} \to \mathbb{R}$ with the product topology. It proves:

1. finite-mode truncations converge coordinatewise in the product topology;
2. a tail spike moving to higher and higher coordinates converges to the zero sequence in that topology;
3. the stronger tail-sensitive observable $\sum_n |x_n|$ stays equal to 1 on that spike sequence and is therefore not continuous at zero;
4. the associated Cole–Hopf datum is likewise not continuous.

This is the first Lean-backed infinite-mode obstruction in the NS lane that directly separates weak coordinatewise convergence from a stronger tail norm.

2.4 Topological Cole–Hopf Shell

In `ColeHopfTopology.lean`, Lean formalizes:

1. continuity of the scalar map $y \mapsto \exp(-y/(2\nu))$;
2. continuity of the induced field-level Cole–Hopf transform;
3. injectivity when $\nu \neq 0$;
4. transfer of potential-limit mismatch into Cole–Hopf-limit mismatch;
5. the same obstruction specialized to cutoff-shaped potentials.

This means the topological $W \mapsto \phi$ step is already packaged in Lean, provided one supplies continuity or convergence of the underlying potential.

2.5 Approximation Interface

In `ApproximationInterface.lean`, Lean packages an abstract positive repair obligation:

> if an approximation scheme makes the chart radius observable and the chart statistic converge, then the cutoff potential and the Cole–Hopf datum also converge.

This is useful because it isolates exactly what an approximation/truncation theorem would have to prove in order for the cutoff/Cole–Hopf layer to behave well.

2.6 Concrete Chart/Truncation Model

In `ChartApproximationModel.lean`, the abstract approximation interface is instantiated on the concrete chart state space $\mathbb{R} \times \mathbb{R}$ with:

1. a hidden-coordinate radius observable;
2. a simple mixed chart statistic;
3. a truncation map that drops the hidden coordinate only at stage 0 and is exact thereafter.

Lean then proves:

1. convergence of the radius observable under truncation;
2. convergence of the chart statistic under truncation;
3. the resulting convergence of the cutoff potential;
4. the resulting convergence of the induced Cole–Hopf datum.

So the approximation interface is now backed by a concrete positive model, not just an abstract shell.

2.7 Geometric Finite-Mode Approximation Model

In `GeometricModeApproximation.lean`, the same approximation interface is instantiated on bounded infinite coefficient sequences. Lean formalizes:

1. a geometrically weighted mode radius;
2. a chart statistic built from the zeroth visible mode plus that radius;
3. finite-mode truncation keeping only the first N coefficients;
4. summability of the weighted mode series under the uniform coefficient bound;
5. convergence of the weighted radius under finite-mode truncation;
6. convergence of the derived chart statistic, the cutoff potential, and the induced Cole–Hopf datum.

This is the first genuinely infinite-mode positive instantiation in the NS Lean lane: the approximation is not just "wrong at stage 0, exact thereafter," but a real truncation-to-limit model on an infinite sequence state space.

2.8 Same-State-Space Product-Topology Obstruction

In `ModeStateProductTopologyObstruction.lean`, Lean returns to the exact `ModeState` and `truncateModes` objects from `GeometricModeApproximation.lean`, but equips that same state space with the coordinatewise product topology induced by the coefficient map. It proves:

1. the existing finite-mode truncations `truncateModes` do converge to the target state in this product topology;
2. a spike sequence still converges to the zero state in that same topology;
3. a stronger tail-presence observable, which records whether any nonzero coefficient exists, is not continuous there;
4. the associated Cole–Hopf datum is likewise not continuous.

So Lean now contains a positive and a negative result on one common infinite-mode object: the geometric weighted radius behaves well under truncation, while a stronger tail-sensitive observable on the same state space does not.

2.9 Same-State-Space Product-Topology Continuity

In `ModeStateProductTopologyContinuity.lean`, Lean proves the matching positive theorem on the exact same `ModeState` equipped with the exact same coordinatewise product topology:

1. each coordinate observable $x \mapsto x_n$ is continuous;
2. each weighted mode term $x \mapsto |x_n|2^{-n}$ is continuous;
3. the summable geometric radius $\sum_n |x_n|2^{-n}$ is continuous;
4. the derived chart statistic, cutoff potential, and Cole–Hopf datum are therefore continuous as well.

So the current NS Lean lane now cleanly separates two observables on one common infinite-mode state space: a summably dominated geometric radius that behaves well in the product topology, and a stronger tail detector that does not.

2.10 Abstract Good Observable Class

In `ModeStateObservableClass.lean`, Lean packages the good side on the shared `ModeState`: nonnegative summable mode weights. For that whole class, Lean proves:

1. continuity of the weighted radius observable;
2. continuity of the derived statistic, cutoff potential, and Cole–Hopf datum;
3. convergence of all of these along the existing truncations `truncateModes`;
4. recovery of the earlier geometric radius model as a special case.

So the good side is no longer just one geometric example. It is now an abstract summable observable class on the shared mode space.

2.11 Abstract Tail-Spike No-Go Theorem

In `ModeStateTailSensitivityObstruction.lean`, Lean packages the bad side abstractly too. If an observable is constant on every remote single-spike state `tailSpikeMode`(N) but takes a different value at `modeZero`, Lean proves:

1. the raw observable is not continuous at `modeZero`;
2. the same obstruction lifts to cutoff potentials;
3. the same obstruction lifts further to Cole–Hopf data.

So the bad side is no longer tied only to the one hand-built `tailPresenceRadius`. It is now a reusable theorem pattern.

2.12 First Concrete Chart Fork

In `ModeStateChartFork.lean`, Lean forces the first explicit chart-style fork on the shared mode space:

1. the soft support observable $t \mapsto |t|/(1+|t|)$, combined with summable mode weights, lies on the good side and yields a continuous truncation-friendly chart radius, statistic, cutoff potential, and Cole–Hopf datum;
2. the hard support observable, which only asks whether any coefficient is nonzero, lies on the bad side and fails continuity at `modeZero`, together with its cutoff and Cole–Hopf lifts.

This is the first genuine theorem-level NS fork in Lean:

> soft support belongs to the good summable side, hard support belongs to the bad tail-sensitive side.

2.13 Sobolev-Style Chart Fork

In `ModeStateSobolevChartFork.lean`, Lean pushes that fork one step closer to manuscript shape by amplifying each mode by the polynomial factor $((n+1)^m)$ before applying the chart cutoff:

1. the soft Sobolev observable, built from the bounded scalar $t \mapsto |t|/(1+|t|)$ together with a concrete inverse-square envelope, is continuous and truncation-friendly;

2. the corresponding hard Sobolev observable, which only asks whether any amplified mode is nonzero, is proved equal to the earlier hard support detector and is therefore not continuous at `modeZero`, together with its cutoff and Cole–Hopf lifts.

   So the current Lean fork is now sharper:

   > even after Sobolev-style high-frequency amplification, the soft bounded version lands on the good summable side, while the hard support version still lands on the bad side.

2.14 Energy-Style Chart Fork

In `ModeStateEnergyChartFork.lean`, Lean pushes the same shared-state fork one step further toward an energy-style observable:

1. the soft energy observable squares the Sobolev-amplified modes and then applies a compensating inverse-power envelope; Lean proves that this radius is continuous and truncation-friendly, and so are its cutoff and Cole–Hopf lifts;

2. the hard energy observable asks whether any amplified squared mode crosses the threshold 1; Lean proves that this detector is stable on the remote spike family, disagrees at `modeZero`, and is therefore not continuous, again together with its cutoff and Cole–Hopf lifts.

   So the current shared-mode-space fork is now sharper:

   > if the chart observable keeps enough inverse-power compensation to remain summable after Sobolev amplification and squaring, Lean puts it on the good approximation-friendly side; if it turns into a hard energy threshold on those amplified modes, Lean puts it on the bad tail-sensitive side.

2.15 Log-Energy Chart Fork

In `ModeStateLogEnergyChartFork.lean`, Lean tests a smoother radial profile on the same shared mode space by replacing raw squared Sobolev energy by $\log(1+\text{energy})$:

1. the soft log-energy observable is continuous and truncation-friendly; the key formal domination is $\log(1+u) \le u$, which lets Lean compare the log-energy terms to the already-controlled soft energy terms;

2. the hard-side analogue asking whether $\log(1+\text{energy}) > 0$ still lands on the bad side: it is stable on the remote spike family, disagrees at `modeZero`, and therefore its cutoff and Cole–Hopf lifts are also not continuous.

   So the shared-mode-space fork is now sharper again:

   > even replacing raw energy by the smoother profile $\log(1+\text{energy})$ does not escape the same dichotomy: the summably compensated soft version stays on the good side, while the hard detection version stays on the bad side.

2.16 Generic Energy-Profile Fork

In `ModeStateEnergyProfileFork.lean`, Lean abstracts the preceding examples into one theorem pattern. An `EnergyProfile` is a globally continuous radial profile $\rho : \mathbb{R} \to \mathbb{R}$ satisfying, on nonnegative inputs:

1. $0 \le \rho(u)$,
2. $\rho(u) \le u$,
3. $\rho(0) = 0$,
4. $u > 0 \Rightarrow \rho(u) > 0$.

For every such profile, Lean proves:

1. the summably compensated soft radius is continuous and truncation-friendly;
2. the induced hard detector agrees with the raw hard Sobolev support detector;
3. that hard detector is tail-spike stable and therefore not continuous at `modeZero`, together with its cutoff and Cole–Hopf lifts.

The file also includes canonical instances:

1. the identity profile, recovering the raw soft energy radius;
2. the clipped log profile $u \mapsto \log(1 + \max(u, 0))$, recovering the soft and hard log-energy constructions on the nonnegative energy inputs that actually occur;
3. a bounded rational saturation profile $u \mapsto \max(u, 0)/(1 + \max(u, 0))$, which is closer to a chart-cutoff saturation shape and whose hard detector Lean proves collapses back to the hard Sobolev support detector;
4. a smooth exponential saturation profile $u \mapsto 1 - \exp(-\max(u, 0))$, again with the same hard-detector collapse;
5. a smooth-transition profile $u \mapsto u \cdot \mathrm{smoothTransition}(u)$, using mathlib's `Real.smoothTransition`, again with the same hard-detector collapse;
6. more strongly, a positive-parameter family $u \mapsto u \cdot \mathrm{smoothTransition}(cu)$ for $c > 0$, all with the same hard-detector collapse;
7. stronger still, an affine two-parameter family $u \mapsto u \cdot \mathrm{smoothTransition}(cu + d)$ for $c > 0$, $d \ge 0$, all with the same hard-detector collapse.

The new boundary result is that a genuinely windowed difference-of-transitions profile already leaves the shell. Lean defines

$$u \mapsto u \cdot \big(\mathrm{smoothTransition}(cu + 1) - \mathrm{smoothTransition}(cu)\big)$$

and proves that for every $c > 0$ this profile cannot be the `profile`-field of any `EnergyProfile`: it vanishes at the positive input $u = c^{-1}$, so it cannot satisfy the required strict positivity on all positive energies.

So the current shared-mode-space NS fork is now theorem-level rather than example-level:

> every continuous radial profile that stays nonnegative, vanishes only at zero, and is dominated by the identity on nonnegative inputs lands on the same good/bad fork.

2.17 Finite Identity-Energy Algebra

In `IdentityEnergy.lean`, Lean formalizes a finite carré-du-champ-style energy

$$\gamma(D) = \sum_i D_i^2$$

and proves:

1. basic positivity and scaling laws for γ;
2. the log-gradient comparison used in the identity-only Cole–Hopf calculation;
3. the finite Cauchy–Schwarz frame-evaluation bound;
4. the resulting pointwise vorticity-style estimate after push-down through a curl frame.

So the finite algebraic spine of the identity-only bound is already Lean-backed.

2.18 Cole–Hopf Positivity Gate

The manuscript's Appendix B log-chain and HJB conversion steps are not just formal heat-equation algebra: they require a strictly positive heat field at the log transform. The new module `ColeHopfPositiveLowerBound.lean` records this as an explicit Lean gate. It proves that every `ColeHopfIdentityData` package has $\Phi(t) > 0$ at every time and, conversely, that if a proposed heat field has $\Phi(t_0) \le 0$, then no `ColeHopfIdentityData` package with that field exists. The zero-valued corollary is `not_exists_ColeHopfIdentityData_of_zero_phi`. The same module now closes the next natural repair attempt as well: `HasArbitrarilySmallColeHopfValues` says that for every $m > 0$ some value of Φ lies below m, and `not_exists_ColeHopfIdentityData_of_arbitrarilySmall_phi` proves that no identity-level Cole–Hopf package can use such a field. The regression suite includes a concrete strictly positive example, $\Phi(x) = x$ on the positive real ray, whose values are nevertheless arbitrarily close to zero.

The newest quotient-level result targets a stronger creative repair. Suppose one keeps pointwise positivity but lets Φ approach zero, while a scalar derivative $d\Phi$ stays bounded below in absolute value by some $a > 0$. Lean proves that $|d\Phi/\Phi|$ then exceeds every real threshold somewhere, and that the corresponding one-direction finite γ-energy has no uniform real bound. Thus a repair that drops the uniform positive lower bound cannot still reuse the manuscript's identity-energy estimate merely by asserting pointwise positivity; it would also need a real theorem forcing the relevant derivatives to vanish at least proportionally to Φ near every near-zero point.

That last sentence is now Lean-backed as a necessary condition, not just a diagnosis. The theorem `abs_derivative_arbitrarilySmall_of_uniform_abs_logDerivative_bound` proves that if $\Phi > 0$, Φ has arbitrarily small values, and $|d\Phi/\Phi|$ is uniformly bounded, then $d\Phi$ has arbitrarily small absolute values. Consequently `not_exists_absDerivativeLowerBound_of_uniform_abs_logDerivative_bound` rules out any uniform positive lower bound on $|d\Phi|$. The positive-ray regression $d\Phi = \Phi = x$ shows that this proportional-vanishing condition is logically coherent at the quotient layer, so it is the next real analytic obligation rather than a free repair.

This is a modest but hard-edged repair blocker. Replacing strict positivity by nonnegativity does not preserve the checked route, and neither does replacing uniform positivity by pointwise positivity. A single zero value, or strictly positive values with no uniform lower bound, already prevents the field from entering the identity-level Cole–Hopf interface; with a nonvanishing derivative lower bound, the denominator quotient itself becomes unbounded.

2.19 Abstract Cole–Hopf/Vorticity Interface

In `ColeHopfInterface.lean`, the finite identity-energy algebra is packaged into an abstract structure carrying:

1. uniform positivity of $\Phi(t)$;
2. a uniform identity-energy bound on its directional derivatives;
3. a uniform frame-curl bound.

From those hypotheses, Lean derives:

1. a bound on the log-potential coefficients $W(t) = -2\nu \log \Phi(t)$;
2. a pointwise abstract vorticity bound;
3. the uniform-in-time version of the same estimate.

2.20 Kernel-Semigroup Wrapper

In `KernelSemigroupInterface.lean`, the same identity-only Cole–Hopf/vorticity algebra is wrapped around a local Markov-kernel semigroup. This does not add new analytic content. It says that if a future NS evolution is encoded as a local kernel semigroup, then the existing identity-energy bridge can be reused unchanged.

2.21 Fefferman-Style Target Backward Interface

In `FeffermanTargetBack.lean`, Lean now works backward from a local mirror of the Clay/Fefferman output clause. The file first defines a `FeffermanPredicateKit`, so the target is parameterized by abstract output-side predicates but still requires actual proofs of them. It then does three things:

1. it defines `UniformVorticityTendril`, the exact smallest theorem-statement-back fragment currently reached by the grassroots lane: a globally defined vorticity field with one uniform envelope;
2. it proves that the existing local shells `ColeHopfIdentityData`, `FiniteModeColeHopfData`, `ColeHopfKernelSemigroupData`, `FiniteModeColeHopfKernelData`, `ManuscriptTruncationPackage`, and `SharedApproximationPackage` all land in that same tendril;
3. it defines `FeffermanLiftObligations`, the exact additional lift still missing if one wants to climb from that tendril to a local Fefferman-style global-regularity output witness: velocity, pressure, proofs of the chosen smoothness / dynamics / energy predicates, and a proof of the chosen vorticity-to-velocity compatibility predicate.

So the target-back picture is now explicit in Lean:

> current grassroots NS packages already produce a uniform vorticity-control tendril, but a separate lift is still required before one has even a local mirror of the Fefferman-style global-regularity output clause.

2.22 Top-Down Fefferman Bridge

In `FeffermanTopDownBridge.lean`, Lean turns that missing lift into a positive program rather than a vague gap. Relative to a chosen predicate kit and a chosen vorticity-compatibility predicate, the file defines:

1. `VelocityPressureCandidate`, the candidate output fields;
2. `RegularityCertificate`, `DynamicsCertificate`, and `EnergyCertificate`, separating smoothness, PDE/incompressibility / initial-condition, and bounded-energy obligations;
3. `VorticityCompatibility`, isolating the extra theorem that ties the already-available vorticity tendril back to the chosen velocity field;
4. `TopDownFeffermanBridge`, bundling exactly those layers.

Lean then proves that any such bridge assembles:

1. the explicit `FeffermanLiftObligations` from the target-back file;
2. the local Fefferman-style global-regularity clause;
3. preservation of the original uniform vorticity tendril.

Finally, the file proves package-level realization theorems for the current NS lane: if a top-down bridge is supplied on the tendril induced by `ColeHopfIdentityData`, `FiniteModeColeHopfData`, `ColeHopfKernelSemigroupData`, `FiniteModeColeHopfKernelData`, `ManuscriptTruncationPackage`, or `SharedApproximationPackage`, then the local Fefferman-style clause follows immediately.

So the constructive NS picture is now also explicit in Lean:

> the current grassroots route already reaches a uniform vorticity tendril, and the remaining top-down work is decomposed into theorem-sized lift layers rather than one monolithic miracle step.

2.23 Concrete Navier–Stokes Equation Target

In `NavierStokesEquationTarget.lean`, Lean now states the first literal NS theorem surface in the current lane. The file fixes time to $\mathbb{R}$, space to $\mathbb{R}^3$, velocity to a vector field $u : \mathbb{R} \to \mathbb{R}^3 \to \mathbb{R}^3$, and pressure to a scalar field $p : \mathbb{R} \to \mathbb{R}^3 \to \mathbb{R}$. Relative to a chosen differential-operator kit, it defines:

1. the pointwise momentum equation
$$\partial_t u + (u \cdot \nabla)u + \nabla p = \nu \Delta u;$$
2. the incompressibility condition $\operatorname{div} u = 0$;
3. the initial condition $u(0, \cdot) = u_0$;
4. a concrete global-regularity witness and the corresponding existence clause for smooth divergence-free initial data with positive viscosity.

The same file also records the honest interface to the current scalar Fefferman-style bridge surface: every full vector-valued NS witness induces, for each coordinate $i \in \{0, 1, 2\}$, a scalar component witness for an associated `FeffermanPredicateKit`. So the current route is now pinned against an actual PDE-side target, but only through its scalar component shadow.

The new file `NavierStokesComponentShadowIncoherence.lean` makes the main weakness of that shadow explicit. The theorem `componentShadowZeroOutput` proves that if one component of the initial datum vanishes identically, then the current component predicate kit is already inhabited by the zero scalar field: smoothness, PDE, incompressibility, and bounded energy are witnessed by the zero vector field, while the initial condition is witnessed separately by the time-independent seed of the full initial datum. So the present scalar shadow does not encode one coherent vector-valued NS witness. The same file then specializes this to nonzero constant initial data with one vanishing coordinate and proves `componentFeffermanClause_without_concreteClause_constantInitialVelocity`: the scalar component clause is true while the full concrete vector-valued clause is false by the whole-space kinetic-energy obstruction. Lean now also packages the datum-uniform failure as `not_componentFeffermanClause_implies_concreteNavierStokesGlobalRegularityClause`: there is no theorem from one current scalar component shadow back to the full concrete vector-valued clause. This is a real route-level blocker, not just a typing annoyance: any repair of the current scalar Fefferman bridge has to supply a shared vector witness across all slots, not merely separate existential lifts for each scalar predicate.

The follow-up file `NavierStokesSharedWitnessComponent.lean` packages that missing burden explicitly. It defines a shared-witness component clause whose witness is one concrete vector-valued NS solution together with one chosen scalar coordinate, and Lean proves `sharedWitnessComponentRegularityClaus` this shared-witness clause is exactly equivalent to the full vector-valued concrete NS clause. The same file then combines that equivalence with the previous constant-data counterexample and proves `componentFeffermanClause_without_sharedWitnessComponentClause_constantInitialVelocity`: the current scalar component shadow still does not imply even the weakest coherent "one shared vector witness" repair. So any same-route rescue now has to add genuinely new coupling structure beyond the present `toComponentPredicateKit`; reinterpreting the existing scalar shadow as a coherent vector witness is formally blocked.

The next file `NavierStokesSharedWitnessBridgeObstruction.lean` packages the stronger theorem-surface consequence. For each fixed coordinate $i \in \{0, 1, 2\}$, Lean proves that there is no datum-uniform bridge theorem from the current scalar shadow clause `FeffermanGlobalRegularityClause (toComponentPredicateKit P i)` to the coherent shared-witness clause. The obstruction theorems `not_uniform_componentShadowBridge_to_sharedWitnessComponent_nsCoord0`, `...nsCoord1`, and `...nsCoord2` use constant initial data pointing in a different coordinate; the summary theorem `not_uniform_componentShadowBridge_to_sharedWitnessComponent` then shows there is no coordinate-uniform upgrade theorem either. This sharpens the route blocker again: Ben does not just need a better interpretation on a bad datum, he needs genuinely new structure because the current component-shadow API cannot support any theorem of the required form even when the coordinate is fixed in advance.

The next small but real strengthening is that one operator is no longer merely named. In the partially concrete constructor `mkWithConcreteDivergence`, the divergence slot is fixed to the trace of the spatial Fréchet derivative on the standard basis of $\mathbb{R}^3$, and Lean proves that incompressibility and divergence-free initial data are exactly the vanishing of those explicit divergence expressions.

The next strengthening fixes a second PDE-side slot. In the more concrete constructor `mkWithConcreteGradientAndDivergence`, the pressure-gradient slot is the actual spatial gradient on $\mathbb{R}^3$, so the momentum equation is no longer merely named there: Lean proves that it is literally the equation with ∇p given by that explicit gradient operator.

The next strengthening fixes a third PDE-side slot. In `mkWithConcreteTimeGradientAndDivergence`, the time-derivative slot is the actual time Fréchet derivative in the unit time direction, so the momentum equation is now literally the equation with explicit $\partial_t u$, ∇p, and $\operatorname{div} u$ on $\mathbb{R} \times \mathbb{R}^3$, while divergence-free initial data remain the explicit vanishing condition from the concrete divergence constructor.

The next strengthening fixes a fourth PDE-side slot. In `mkWithConcreteTimeConvectionGradientAndDivergen` the convection term $(u \cdot \nabla)u$ is the spatial Fréchet derivative applied to the current velocity vector, so the momentum equation is now literally the equation with explicit $\partial_t u$, $(u \cdot \nabla)u$, ∇p, and $\operatorname{div} u$ on $\mathbb{R} \times \mathbb{R}^3$; only the Laplacian and the analytic regularity/energy predicates remain abstract at this surface.

The next strengthening fixes the fifth PDE-side slot. In `mkWithConcreteDifferentialOperators`, the Laplacian is the actual mathlib Laplace operator on the fixed-time velocity slice, so the momentum equation is now literally the full differential equation with explicit $\partial_t u$, $(u \cdot \nabla)u$, ∇p, Δu, and $\operatorname{div} u$ on $\mathbb{R} \times \mathbb{R}^3$. At that surface, what remains abstract is no longer the differential operator content of the PDE, but the analytic regularity, energy, and existence machinery.

The next strengthening fixes the smoothness predicates themselves. In `mkWithConcreteDifferentialOperators` velocity and pressure are required to be literal C^∞ maps on $\mathbb{R} \times \mathbb{R}^3$, while initial velocity data are literal C^∞ maps on $\mathbb{R}^3$. So at this surface the remaining abstract slot is no longer "smoothness" in name only: it is essentially the bounded-energy predicate and the actual existence/regularity theorem. An important local repair was needed here after mathlib's current `ContDiff` indexing semantics became sharper: `ContDiff` $\mathbb{R}$ $\top$ is an ω-level analytic requirement, not the intended C^∞ surface. The local NS theorem surface now explicitly uses `ContDiff` $\mathbb{R}$ ∞ for velocity, pressure, and initial data, so the formalized smoothness slot once again matches the manuscript-level "smooth" / Schwartz reading rather than an unintended analyticity demand.

The next strengthening fixes that remaining theorem-surface predicate too. In `mkFullyConcreteNavierStokesSu` the bounded-energy slot is literally the statement that there exists a uniform constant $C \geq 0$ such that for every time t, the kinetic-energy density $|u(t,x)|^2$ is integrable on $\mathbb{R}^3$ and

$$\int_{\mathbb{R}^3} |u(t,x)|^2 \, dx \leq C.$$

So the local target is now fully concrete at the PDE-surface level: differential operators, smoothness, incompressibility, initial condition, and bounded energy are all literal Lean definitions on $\mathbb{R} \times \mathbb{R}^3$. What remains abstract is the actual theorem asserting global existence and regularity for that fully concrete surface.

The next tightening removes one more layer of packaging. The same file now defines an explicit proposition saying directly that for every viscosity $\nu > 0$ and every smooth divergence-free initial velocity u_0, there exist u and p such that u and p are smooth,

$$\partial_t u + (u \cdot \nabla)u + \nabla p = \nu \Delta u, \qquad \operatorname{div} u = 0, \qquad u(0,\cdot) = u_0,$$

and the kinetic-energy integrals are uniformly bounded in time. Lean proves that this fully explicit statement is equivalent to the current structured `ConcreteNavierStokesMillenniumTarget`. So the target is no longer only concrete "up to record fields"; it is now pinned to a literal theorem statement.

The first bottom-up follow-up file, `VectorCalculusR3.lean`, extracts a reusable concrete vector-calculus layer from that theorem surface. It defines coordinate derivatives and the literal curl/vorticity operator on $\mathbb{R}^3$, then proves real linearity and zero lemmas for the concrete operators ∂_t, div, ∇p, and Δ, plus the corresponding coordinate-derivative facts and concrete vorticity identities (zero

field and scalar-multiplication commutation). This is meant as curriculum-style material with independent value, not more interface packaging. The same concrete file now also proves exact pointwise vorticity-norm formulas for steady linear shear, damped heat shear, transported heat shear, and the transported full-drift heat-shear family. In particular, Lean has the reusable phase-independent envelope bounds `norm_spatialVorticity_heatShearVelocityField_le_exp_abs`, `norm_spatialVorticity_heatShearTransportVelocityField_le_exp_abs`, and `norm_spatialVorticity_heat` These keep the damping factor $\exp(-(\nu k^2)t)$ rather than immediately collapsing the estimate to a coarser slab constant, so later BKM-envelope work can cite a concrete $\mathbb{R}^3$ curl calculation instead of reproving the trigonometric bound inside each continuation shell. That citation path is now present in the BKM layer itself: the definition `heatShearExpVorticityEnvelope` records the damped scalar envelope, and `heatShearVelocityField_has_expBKMEnvelope`, `heatShearTransportVelocityField_has_expBKME` and `heatShearTransportFullDriftVelocityField_has_expBKMEnvelope` package it as integrable BKM data on every nonnegative slab when $\nu \geq 0$. The integral bound is still the coarse $T|ak|$, so this is not a new continuation theorem; it is a sharper, reusable route from explicit curl computation to the existing BKM-envelope interface. That coarse-budget limitation is now removed for the nondegenerate damped branch. Lean proves the exact interval integral

$$\int_0^T |a| \, \exp(-(\nu k^2)t) \, |k| \, dt = |a| \, |k| \, \frac{1 - \exp(-(\nu k^2)T)}{\nu k^2}$$

when $T \geq 0$, $\nu > 0$, and $k \neq 0$, and packages the pure, transported, and transported full-drift heat-shear candidates with this exact BKM budget through the corresponding `_has_exactExpBKMEnvelope` theorems. This is still not analytic continuation progress; it says the heat-shear obstruction below does not depend on the earlier coarse integral estimate either. The energy/BKM bridge now consumes this sharper package in its obstruction statements as well. The theorems `heatShearVelocityField_has_expBKMEnv` and `heatShearTransportFullDriftVelocityField_has_expBKMEnvelope_but_not_energyBKMBridge_kinetic` show that the pure and transported full-drift heat-shear candidates can carry the damped BKM envelope itself while still failing the corrected energy/BKM bridge's kinetic-integrability input. The obstruction is not a weak choice of constant envelope; it is the time-zero finite-energy mismatch of the matched whole-space initial datum.

The next file, `NavierStokesUniformVorticityContinuationTarget.lean`, adds the first concrete analytic bridge surface on top of that literal theorem. It defines explicit finite-time smooth witnesses on a slab $0 \leq t \leq T$, the concrete vorticity operator $\nabla \times u$ on $\mathbb{R}^3$, a uniform vorticity bound on the slab, and the corresponding continuation target saying that such a witness should extend to a global smooth incompressible solution with bounded kinetic energy. Lean also proves the sanity theorem that the fully explicit global theorem target subsumes this continuation target, and hence so does the structured fully concrete target via the explicit-equivalence theorem above. At present this is only a one-way subsumption result: the Lean proof simply reuses the full global theorem target and does not yet use the vorticity-bound hypothesis in any analytic argument. But this file now also contains an operational zero-data instantiation: it constructs a canonical finite-time zero witness with uniform zero vorticity bound and proves that, if the uniform continuation target is assumed, one gets an explicit global output for zero initial data at any positive viscosity. In addition, this same file now proves a target-free baseline theorem: `ExplicitConcreteNavierStokesGlobalOutput_zero`, together with the matching clause-level inhabitant `ExplicitConcreteNavierStokesRegularityClause_zero`. Those two lemmas do not assume any continuation target; they are direct concrete PDE-side sanity anchors on the literal $\mathbb{R} \times \mathbb{R}^3$ statement surface. The same concrete layer now also exposes a real pressure-gauge symmetry: `smoothSpaceTimePressure_contDiff_spatialSlice` and `smoothSpaceTimePressure_differentiableAt_spatialSlice` give the fixed-time smoothness

bridge for pressure slices, `smoothSpaceTimeVelocity_add`, `smoothSpaceTimeVelocity_sub`, and `smoothSpaceTimeVelocity_linearCombination` now make the corresponding smooth velocity algebra explicit on $\mathbb{R}\times\mathbb{R}^3$, `smoothSpaceTimePressure_const`, `smoothSpaceTimePressure_const_smul`, `smoothSpaceTimePressure_sub`, and `smoothSpaceTimePressure_linearCombination` do the same for smooth pressure fields, `spatialPressureGradient_add`, `spatialPressureGradient_const_smul`, `spatialPressureGradient_sub`, and `spatialPressureGradient_linearCombination` give the matching affine algebra for concrete pressure gradients on each fixed spatial slice, `ExplicitFiniteTimeRegularityWi` and `ExplicitConcreteNavierStokesGlobalOutput.addPressureOfZeroSpatialGradient` now package the gauge transport principle itself at the explicit witness and global-output layers, while `ExplicitFiniteTimeRegularityWitness.uniformVorticityBound_addPressureOfZeroSpatialGradient` and `ExplicitFiniteTimeRegularityWitness.BKMEnvelope_addPressureOfZeroSpatialGradient` push the same invariance into the two concrete continuation-data packages, `smoothSpaceTimePressure_timeOnly` records that any smooth scalar function of time alone gives a smooth space-time pressure field, and `spatialPressureGradient_timeOnly` records that such a time-only pressure has zero spatial gradient. The resulting witness/global-output transport theorems `ExplicitFiniteTimeRegularityWitness.addTimeOn` and `ExplicitConcreteNavierStokesGlobalOutput.addTimeOnlyPressure` show that adding any smooth pressure gauge of the form $p(t)$ leaves the concrete Navier–Stokes data valid. The older constant-pressure transport theorems are now honest special cases of this larger gauge symmetry. Consequently Lean now also proves the explicit time-gauge baseline `ExplicitConcreteNavierStokesGlobalOutput_zer` zero velocity with arbitrary smooth time-only pressure is still a genuine global solution on the literal PDE surface. The same baseline is now packaged at the regularity-clause level too by `ExplicitConcreteNavierStokesRegularityClause_addPressureOfZeroSpatialGradient`, `ExplicitFiniteEne` `ExplicitConcreteNavierStokesGlobalOutput_zero_addPressureOfZeroSpatialGradient`, `ExplicitConcrete` `ExplicitFiniteEnergyAdmissibleNavierStokesRegularityClause_zero_addPressureOfZeroSpatialGradien` which package the generic slice-wise zero-gradient gauge symmetry first on the arbitrary-data explicit clause layer and then on the zero baseline, and the older constant-pressure and time-only baseline lemmas are now just wrappers around those generic transports: `ExplicitConcreteNavierStokesRegularityClause_z` `ExplicitFiniteEnergyAdmissibleNavierStokesRegularityClause_zero_constantPressure`, `ExplicitConcre` and `ExplicitFiniteEnergyAdmissibleNavierStokesRegularityClause_zero_timeOnlyPressure`. The same generic transport now also reaches the structured fully concrete whole-space clause layer via `concreteNavierStokesGlobalRegularityClause_addPressureOfZeroSpatialGradient` and `finiteEnergyConcreteNavierStokesGlobalRegularityClause_addPressureOfZeroSpatialGradient`, with the zero-data instances exposed via `concreteNavierStokesGlobalRegularityClause_zero_addPressureOfZ` and `finiteEnergyConcreteNavierStokesGlobalRegularityClause_zero_addPressureOfZeroSpatialGradien` so the concrete clause layer itself inherits this pressure-gauge symmetry. This same pressure-gauge invariance now also reaches the uniform continuation side: `ExplicitUniformVorticityContinuationClause_apply_` shows that a same-horizon uniform continuation clause can be applied after a smooth time-only pressure-gauge change, while `ExplicitFiniteTimeRegularityWitness.uniformVorticityBound_addTimeOnlyPr` shows that the same uniform bound survives gauge change plus restriction to a shorter slab, and `ExplicitUniformVorticityContinuationClause_apply_of_uniformVorticityBound_addTimeOnlyPressure_` turns that directly into a smaller-horizon operational clause theorem. Lean now also instantiates that operational theorem concretely at zero data first for arbitrary slice-wise zero-gradient gauges: `zeroFiniteTimeRegularityWitness_addPressureOfZeroSpatialGradient_restrict_has_uniformVorticityB` shows that the gauged-and-restricted zero witness itself already carries the explicit shorter-slab zero vorticity bound, and `ExplicitUniformVorticityContinuationClause_implies_zeroGlobalOutput_addPressur` shows that a smaller-slab uniform continuation clause can therefore be applied directly to that concrete gauged zero witness. The older `..._timeOnlyPressure` and constant-pressure lemmas are now just wrappers around that general zero-spatial-gradient transport. The same zero-data operational

packaging now exists on the BKM side too via `zeroFiniteTimeRegularityWitness_addPressureOfZeroSpatialGr` `ExplicitBKMContinuationClause_implies_zeroGlobalOutput_addPressureOfZeroSpatialGradient`, and their repaired finite-energy analogues. The same layer now also includes the direct clause-level formulation `ExplicitUniformVorticityContinuationClause_implies_zeroGlobalOutput`, which applies one concrete uniform continuation clause instance to the canonical zero slab witness. The premise is now also packaged explicitly by `zeroFiniteTimeRegularityWitness_exhibits_uniformContinuationP` which says the quantified witness-and-bound hypothesis is genuinely inhabited at zero data and therefore not merely vacuous there. The zero-data route is now factored through a reusable uniform bound lemma `uniformVorticityBoundUpTo_zero`. This layer now also contains a reusable rescaling transport lemma `uniformVorticityBoundUpTo_const_smul`: if $\|\omega_u\| \le B$ on a slab, then $\|\omega_{a\cdot u}\| \le |a|\, B$ on the same slab, plus monotonicity and sign-flip transport lemmas `uniformVorticityBoundUpTo_mono` and `uniformVorticityBoundUpTo_neg`. The concrete vector-calculus layer underneath now also exposes `smoothSpaceTimeVelocity_contDiff_spatialSlice` and `smoothSpaceTimeVelocity_differentiableAt_` which turn full space-time smoothness into fixed-time spatial differentiability. Using that bridge, the same file now proves the additive closure theorem `uniformVorticityBoundUpTo_add`: smooth slab bounds for u and v combine into a slab bound for $u + v$ with parameter $B + B'$. So the uniform layer now supports not only rescaling and restriction, but also explicit composition of concrete vorticity bounds under field addition. It also has time-restriction transport lemmas `uniformVorticityBoundUpTo_restrict` and `boundedKineticEnergyUpTo_restrict`. It also now adds a witness-level restriction constructor `ExplicitFiniteTimeRegularityWitness.restrict` and the induced transport lemma `ExplicitFiniteTimeRegularityWitness.uniformVorticityBound_restrict`, as well as the clause-level horizon-monotonicity theorem `ExplicitUniformVorticityContinuationClause_mono_h` The same bottom-up lane now also contains an explicit nonzero-looking sanity family that exposes a genuine obstruction instead of more interface layering. The vector-calculus file introduces `constantVelocityField` and `constantInitialVelocity` and proves that such fields have zero time derivative, zero convection, zero Laplacian, zero divergence, and zero vorticity, so they are honest differential-side constant-field solutions on the literal $\mathbb{R} \times \mathbb{R}^3$ PDE surface. The uniform-vorticity file then proves `uniformVorticityBoundUpTo_constantVelocityField`, showing that every constant field carries the trivial vorticity envelope $B = 0$. But Lean now also proves the obstruction the manuscript actually needs: `not_integrable_kineticEnergyDensity_constantVelocityField`, `not_boundedKineticEnergy_constantVelocityField`, and `not_boundedKineticEnergyUpTo_constantVelocit` show that on whole-space $\mathbb{R}^3$, every nonzero constant velocity field fails the bounded-energy slot. So the tempting "nonzero constant witness" ansatz is ruled out by a real proved theorem rather than by informal handwaving. Lean now also proves the stronger initial-data obstruction the manuscript actually needs for the explicit theorem surfaces: if a field merely matches nonzero constant initial data at time $t = 0$, then its initial kinetic-energy density already agrees with the forbidden constant one. Concretely, `not_integrable_kineticEnergyDensity_zero_of_matchesInitialVelocity_constantInitialVeloci` `not_boundedKineticEnergyUpTo_of_matchesInitialVelocity_constantInitialVelocity`, and `not_boundedKineticEnergy_of_matchesInitialVelocity_constantInitialVelocity` push the obstruction from the special ansatz $u(t, x) \equiv c$ to any field with nonzero constant initial slice. The same whole-space energy gate is now also factored through the reusable concrete theorems `not_boundedKineticEnergyUpTo_of_not_integrable_kineticEnergyDensity_zero` and `not_boundedKineticE` plus the matched-data contrapositive `not_boundedKineticEnergyUpTo_of_matchesInitialVelocity_of_not_fi` So the constant, heat-shear, transported heat-shear, and linear-shear matched-data bounded-energy obstructions on this surface are now corollaries of one concrete time-zero integrability blocker rather than separately re-running the same slab-energy extraction argument. Consequently `not_nonempty_ExplicitFiniteTimeRegularityWitness_constantInitialVelocity`, `not_ExplicitConcreteNa` and `not_ExplicitConcreteNavierStokesRegularityClause_constantInitialVelocity` show that

the current explicit witness, global-output, and regularity-clause surfaces all honestly reject nonzero constant initial data on $\mathbb{R}^3$. The new module `NavierStokesConstantFieldWitnessObstruction.lean` now pushes that same point down to the exact finite-time witness surface itself. For any smooth time-only pressure gauge $p(t,x) = \pi(t)$, Lean proves that the pair (u,p) with $u(t,x) \equiv c \neq 0$ satisfies every witness field except bounded energy: smoothness, the slab momentum equation, incompressibility, and the initial condition are all available literally on $\mathbb{R}\times\mathbb{R}^3$. But `not_exists_constantVelocityField_finiteTime` and its time-only-pressure corollary show that no `ExplicitFiniteTimeRegularityWitness` can carry that exact velocity. So the constant-field route does not fail because of a missing pressure choice or a hidden PDE identity; it fails exactly at the whole-space `bounded_energy_on` slot. Lean now also packages the exact classification theorem on this concrete ansatz. On any nonnegative slab, `constantVelocityField_finiteTimeWitness_iff_zero_and_spatialPressureGradient_zero` shows that a witness with velocity $u(t,x) \equiv c$ and fixed smooth pressure p exists if and only if $c = 0$ and the spatial pressure gradient vanishes on the slab. So even on the most favorable steady exact field, the whole-space witness surface collapses all the way back to the zero-data pressure-gauge case. Lean now also packages the stronger type-level boundary: `nonempty_ExplicitFiniteTimeRegularityWitness_constantIni` shows that on any nonnegative slab the entire witness type for constant initial data $u_0(x) \equiv c$ is inhabited if and only if $c = 0$. So there is no creative escape through a nonconstant evolving witness with the same initial slice; the whole constant-initial-data route collapses exactly to the zero datum. Lean now also pushes this exact boundary down to the repaired whole-space energy gate itself. `finiteInitialKineticEnergy_constantInitialVelocity_iff` shows that constant initial data have finite initial kinetic energy if and only if $c = 0$. So the constant family now collapses exactly already at the whole-space finite-energy surface, not only at the witness and route-facing theorem layers. Lean now pushes that same exact boundary onto the route-facing theorem surfaces. `ExplicitConcreteNavierStokesGlobalOutput_constantInitialVelocity_iff`, `ExplicitConcreteNavierStokesRegularityClause_constantInitialVelocity_iff`, and `concreteNavierStok` show that the explicit global-output surface, the unrepaired explicit regularity clause, and the fully concrete structured clause all hold for constant initial data if and only if $c = 0$ (for the latter two, at positive viscosity where those clause surfaces are stated). So the constant-initial-data route is now exact not only on the candidate and witness layers, but also on the concrete theorem surfaces the proof attempt would actually need to inhabit. Lean now also contains the first genuinely nonconstant version of the same obstruction. The vector-calculus layer introduces `linearShearMap`, `linearShearInitialVelocity`, and the actual space-time field `linearShearVelocityField`, proving that the profile $u(t,x) = (ax_1, 0, 0)$ is smooth, divergence-free, has zero time derivative, zero convection, zero Laplacian, and constant vorticity $(0, 0, -a)$. So this is not merely a tempting initial datum: it is an honest exact Navier–Stokes differential-side candidate on $\mathbb{R}\times\mathbb{R}^3$ with zero pressure. The uniform-vorticity layer then proves `uniformVorticityBoundUpTo_linearShearVelocityField`, so this nonzero candidate carries the explicit bound $B = |a|$. Lean now packages the whole point directly as `linearShearVelocityField_exhibits_concreteCandidate_except_boundedEnergy`: already on the global explicit theorem surface, the steady shear field gives an honest smooth incompressible zero-pressure Navier–Stokes candidate whose only missing slot is bounded kinetic energy. Lean then packages the resulting global-surface mismatch as `linearShearInitialVelocity_exhibits_concreteCandidate_` the explicit candidate exists, but `ExplicitConcreteNavierStokesGlobalOutput` is still false. Lean now pushes this all the way to the theorem surface: `not_ExplicitConcreteNavierStokesMillenniumTarget` and `not_ConcreteNavierStokesMillenniumTarget` show that the current formal target is outright false, because it quantifies over smooth divergence-free data like nonzero linear shear without imposing a finite-energy admissibility condition on the input side. Lean now also records the repaired whole-space theorem surface itself: *This is a bug in our local over-broad whole-space surrogate, not in the upstream Clay-style theorem surface, and not by itself in Goertzel's current*

manuscript route. The reference Lean repo already builds the $\mathbb{R}^3$ statement with bounded-energy admissibility on the input side, and Goertzel's draft works with narrower input surfaces such as the torus and Schwartz-type whole-space data. Accordingly, the negative theorem here should be read as a regression guard on our theorem shaping: any local $\mathbb{R}^3$ target that forgets that input restriction is wrong. `finiteInitialKineticEnergy`, `FiniteEnergyAdmissibleNavierStokesProblemData`, `FiniteEnergyConcreteNavierStokesMillenniumTarget`, `ExplicitFiniteEnergyAdmissibleNavierStokesRegu` and `FiniteEnergyConcreteNavierStokesMillenniumTarget_iff_explicit`. So the correction is no longer only informal commentary: it is a separate Lean target. Lean now also records the direct theorem-surface implication layer: `ExplicitConcreteNavierStokesRegularityClause_implies_finiteEnergy`, `ExplicitFiniteEnergyAdmissibleNavierStokesRegularityClause_iff_of_finiteInitialKineticEnergy`, `ExplicitConcreteNavierStokesMillenniumTarget_implies_finiteEnergy`, and `ConcreteNavierStokesMille` so the repaired theorem surface is now explicitly packaged as a restriction of the older unrepaired one. The zero datum satisfies the repaired input hypothesis via `finiteInitialKineticEnergy_zero`, and `NavierStokesSchwartzData.lean` now adds the manuscript-compatible restricted whole-space input lane: `NSSchwartzInitialVelocity`, `smoothInitialVelocityData_of_schwartzInitialVelocity`, `finiteInitialKineticEnergy_of_schwartzInitialVelocity`, `ExplicitSchwartzAdmissibleNavierStokesReg` and `ExplicitFiniteEnergyAdmissibleNavierStokesMillenniumTarget_implies_schwartzTarget` package the fact that Schwartz initial data automatically satisfy the repaired $\mathbb{R}^3$ input hypotheses. This gives the local Lean lane a theorem surface that matches the manuscript's intended torus-to-Schwartz exhaustion story more closely than the old unrepaired whole-space surrogate. `NavierStokesEquationTarget.lean` was also repaired at this stage so the underlying smoothness predicates are genuinely C^∞ rather than the stronger ω-analytic `ContDiff` $\top$ tier; this matters for the Schwartz route because generic Schwartz data are smooth, but not automatically analytic. The next refinement is now present explicitly in `NavierStokesSchwartzDivergenceFreeData.lean`: Lean packages the manuscript's Appendix H.1 input class $S_\sigma(\mathbb{R}^3)$ itself as the subtype `NSSchwartzDivergenceFreeInitia` of Schwartz data whose initial divergence vanishes, and theorems such as `ExplicitSchwartzDivergenceFreeNavier` and `ExplicitFiniteEnergyAdmissibleNavierStokesMillenniumTarget_implies_schwartzDivergenceFreeTa` show that the repaired whole-space target now specializes directly to that manuscript-style input surface rather than only to the looser "Schwartz plus a separate divergence-free hypothesis" packaging. The same file now also packages the structured side of that restriction: a manuscript-style datum with $\nu > 0$ canonically produces repaired `FiniteEnergyAdmissibleNavierStokesProblemData`, and `StructuredSchwartzDivergenceFreeNavierStokesRegularityClause_iff` identifies the resulting record-based whole-space clause with the explicit $S_\sigma(\mathbb{R}^3)$ clause. So the manuscript-compatible subtype is now connected to both local theorem surfaces, not only the explicit one. `NavierStokesSchwartzPartialPeriodization.lean` now adds the next bottom-up Appendix H.2/H.3 precursor: a finite lattice-sum periodization layer. Instead of jumping straight to the full infinite torus-periodization series, Lean first packages the local algebra that the later exhaustion argument needs: `periodizationShift`, `translateInitialVelocity`, and `finitePartialPeriodizedInitialVelocity`, together with proofs that spatial translation preserves smoothness and initial divergence, finite sums of translated data preserve both properties, and finite partial periodizations therefore preserve the manuscript's $S_\sigma(\mathbb{R}^3)$ input class via `finitePartialPeriodizationSchwartzDivergenceFreeInitialVelocity`. The companion regression file checks the empty and singleton-zero partial periodizations explicitly, so the torus-to-whole-space route now has a verified finite prelude rather than only an informal next-step note. `NavierStokesSchwartzPeriodizationBoxes.lean` now adds the next H.4-side exhaustion layer on top of that prelude: the concrete centered cubic boxes $[-N, N]^3 \subset \mathbb{Z}^3$, the shell decomposition from radius N to radius $N+1$, and the boxed partial-periodization operators obtained by specializing the general finite-sum interface to those boxes. This makes the truncation family itself available as named Lean data via `centeredLatticeBox`, `centeredLatticeShell`,

and `boxedPartialPeriodizedInitialVelocity`, with direct zero-radius and successor-step theorems. So the manuscript's large-box exhaustion is no longer just an intended indexing convention: the finite approximation family and its one-step growth law now exist explicitly in the local formalization. `NavierStokesSchwartzPeriodizationConvergence.lean` adds the first honest local limit interface above those finite boxes. Instead of claiming the full Schwartz periodization limit already exists, Lean now packages cube-local convergence of the boxed partial periodizations on $Q_R = [-R, R]^3$ via `BoxedPartialPeriodizationConvergesOnCube`, proves monotonicity in the cube radius, and derives the key structural consequence that if the boxed sums converge on a fixed cube then the added shell contributions must converge to zero there. The newest strengthening also packages this shell-vanishing corollary directly for genuine Schwartz data and nonzero lattice period, so Appendix H.4's first local decay step is now available in Lean without first restating a separate convergence hypothesis. This is exactly the first nontrivial convergence fact needed before any Arzelà–Ascoli / compactness step in Appendix H.4 can be stated cleanly. `ExplicitFiniteEnergyAdmissibleNavierStokesRegularityClause_zero` shows the repaired explicit clause is inhabited there. On the other hand, Lean now also records that this repaired explicit clause is still vacuous outside the finite-energy domain: `ExplicitFiniteEnergyAdmissibleNavierStokesRegulari` packages the generic logical fact, while `ExplicitFiniteEnergyAdmissibleNavierStokesRegularityClause_cons` and `ExplicitFiniteEnergyAdmissibleNavierStokesRegularityClause_linearShearInitialVelocity_witho` show concretely that the repaired clause can still be true on the old bad families while the unrepaired concrete regularity clause is false there. `not_finiteInitialKineticEnergy_linearShearInitialVelocity` and `not_finiteInitialKineticEnergy_constantInitialVelocity` show that the counterexample families are now excluded at the input side rather than only killed later by the output bounded-energy slot, and Lean now also records that this exclusion is literal on the repaired structured theorem surface via the generic theorem `not_nonempty_FiniteEnergyAdmissibleNavierStokesProblemData_of_not_fin` with the concrete constant and shear instances exposed as `not_nonempty_FiniteEnergyAdmissibleNavierStokesP` and `not_nonempty_FiniteEnergyAdmissibleNavierStokesProblemData_linearShearInitialVelocity`. Lean now also packages the positive converse for fixed concrete data: `nonempty_FiniteEnergyAdmissibleNavierSt` says that once viscosity positivity, initial smoothness, and initial divergence-freeness are fixed, inhabiting the repaired structured input surface is equivalent to having finite initial kinetic energy. Lean now also pushes that input exclusion down to the corrected energy/continuation bridge itself. In `NavierStokesEnergyContinuationBridge.lean`, `finiteInitialKineticEnergy_of_matchesInitialVelocity_` shows that the bridge's raw slab kinetic-integrability input already forces finite initial energy for matched data on any nonnegative slab, and `not_energyContinuationBridge_kineticIntegrabilityOnSlab_of_n` packages the contrapositive. So the whole-space obstruction no longer appears only after the bridge has produced `boundedKineticEnergyUpTo`; for non-finite-energy data the bridge cannot even be instantiated. The concrete theorem `constantVelocityField_has_coordinateEnergyIdentity_but_not_energyCo` makes this exact on the constant-field ansatz: the corrected coordinate energy identity still holds, but the bridge's time-zero kinetic-integrability input does not. Lean now also makes that bridge input exact on stationary seeds. `kineticIntegrabilityOnSlab_timeIndependentVelocity_iff` identifies the slab kinetic-integrability hypothesis for `timeIndependentVelocity u_0` with `finiteInitialKineticEnergy u_0`, and `kineticIntegrabilityOnSlab_constantVelocityField_iff` collapses the constant-field case all the way down to `c = 0`. So on the seed family the route actually uses, there is no residual flexibility in the bridge input: whole-space finite energy is exactly the gate. Lean now makes the actual bounded-energy witness slot exact on that same stationary-seed family. `boundedKineticEnergyUpTo_timeIndependentVelocity_iff` identifies the finite-time bounded-energy predicate for `timeIndependentVelocity u_0` with `finiteInitialKineticEnergy u_0`, and `boundedKineticEnergyUpTo_constantVelocityField_iff` again collapses the constant-field case to `c = 0`. So there is no remaining gap between the bridge input and the witness slot on

stationary seeds: both exact surfaces are the same whole-space finite-energy gate. Lean now also folds that gate into the full stationary-seed witness theorem itself. On nonnegative slabs, `timeIndependentVelocity_finiteTimeWitness_iff_finiteInitialKineticEnergy_and_stationaryMomentu` shows that a witness with velocity `timeIndependentVelocity u_0` and fixed pressure p exists exactly when `finiteInitialKineticEnergy u_0` holds and the stationary momentum balance is satisfied on the slab. So the steady-seed route no longer has a hidden external energy side condition: the exact witness surface itself already exposes finite whole-space energy as part of the theorem statement. On the positive finite-mode side, the energy/continuation bridge now has a less specialized middle theorem for bounded two-profile Schwartz velocity fields. The theorem `boundedKineticEnergyUpTo_of_add_scalar_smul_schwartz_of_abs_le_of_spatialDivergence_zero_of_pre` works for an arbitrary pressure field p: it assumes the pointwise Navier–Stokes momentum equation, pressure-pairing integrability, zero pressure work, and slice-wise divergence-freeness, and then supplies the `boundedKineticEnergyUpTo` witness slot. The bounded Schwartz velocity class supplies kinetic-energy integrability, viscous-pairing integrability, and nonlinear energy cancellation internally. This is not a pressure theorem: pressure-pairing integrability and zero pressure work remain real obligations. But it removes an artificial route restriction to affine-plus-localized Schwartz pressures at the continuation-bridge layer. Lean now closes that pressure obligation for a broader concrete decay class. In `NavierStokesEnergyInequality.lean`, `pressureEnergyPairing_schwartzPressureSlice` expands $u \cdot \nabla p$ coordinatewise for any time-indexed Schwartz pressure slice $p(t,\cdot)$. The theorem `integrable_pressureEnergyPairing_of_schwartzSlice_schwartzPressureSlice` proves integrability against a Schwartz velocity slice, and `integral_pressureEnergyPairing_of_schwartzSlice_schwartzP` proves zero pressure work by integration by parts from the divergence-free condition. The continuation theorem `boundedKineticEnergyUpTo_of_add_scalar_smul_schwartz_of_abs_le_of_spatialDivergence_` then applies the energy bridge directly to bounded two-profile Schwartz velocities with arbitrary time-varying Schwartz pressure slices. This is a real strengthening over the affine-plus-fixed-profile pressure lane: the remaining pressure burden is no longer present for spatially Schwartz pressure slices, while genuinely nondecaying or nonsmooth pressures remain outside the proved class. Lean now also connects this pressure-slice class to the finite-time witness and BKM-premise layers. The constructor `ExplicitFiniteTimeRegularityWitness.of_add_scalar_smul_schwartz_of_abs_le_of_spatialDivergence_` uses the pressure-slice energy theorem to fill `bounded_energy_on` without assuming pressure-pairing integrability or zero pressure work as external hypotheses. The theorem `twoModeSchwartzDivergenceFreeInitialV` specializes this to the bounded two-mode divergence-free Schwartz ansatz, and `twoModeSchwartzDivergenceFreeIni` adds the internally proved finite-mode vorticity envelope. Thus the pressure-slice lane reaches the repaired BKM input surface with smooth pressure, bounded amplitudes, profile-level divergence freedom, and the pointwise momentum equation as the remaining explicit hypotheses. It does not prove the continuation clause; it removes another artificial pressure-shape restriction from the finite-mode premise. The pointwise momentum-equation hypothesis in that last sentence has also been sharpened into a concrete residual identity. The theorem `momentumEquation_twoModeSchwartzVelocity_schwartzPressureS` rewrites the pressure-slice momentum equation into its finite-mode expansion: scalar amplitude derivatives, four self/mixed convection terms, the pressure-slice gradient, and the two linear Laplacian terms. The bridge theorem `twoModeSchwartzDivergenceFreeInitialVelocity_exhibits_BKMContinuationPrem` then reaches the BKM-premise surface from this expanded closure directly. This is not a continuation theorem and it does not solve the closure equation for a nontrivial profile; it makes the next repair target small and inspectable. The pressure-slice closure route also now contains the exact anti-profile zero-flow baseline. Lean proves `explicitClosure_oneOneAntiProfileSchwartzVelocity_schwartzPressureSli` for any Schwartz profile f, the pair $f, -f$, unit amplitudes, and zero Schwartz pressure slice satisfy the expanded pressure-slice residual closure. The corresponding bridge `twoModeSchwartzDivergenceFreeInitialVeloc` then reaches BKM-premise data for zero initial velocity when f is divergence-free and $\nu \geq 0$. This

removes a small artificial mismatch between the pressure-slice explicit-closure route and the older zero-pressure sanity baseline, but it is still pure cancellation: it neither supplies a nonzero initial datum nor solves the non-cancelling closure equation. Lean has now generalized that pressure-slice cancellation boundary from unit amplitudes to arbitrary smooth equal amplitudes. The theorem `explicitClosure_equalAmplitudeAntiProfileSchwartzVelocity_schwartzPressureSlice_zeroPressure` expands the residual for $a(t)f+a(t)(-f)$, and `twoModeSchwartzDivergenceFreeInitialVelocity_exhibits_BKMC` routes any uniformly bounded smooth equal-amplitude anti-profile through the BKM-premise bridge. Thus a coefficient-time-dependence repair of the anti-profile baseline is already accounted for; the remaining finite-mode target must be genuinely non-cancelling. Lean now also proves that this zero-flow anti-profile boundary is exact for nonzero profiles: `antiProfileSchwartzVelocity_eq_zero_iff_equalAmplitude` says that $a(t)f + b(t)(-f)$ vanishes identically if and only if the amplitudes are pointwise equal, provided f is not the zero profile. The initial-slice version `antiProfileSchwartzInitialVelocity_eq_zero_iff_equa` gives the corresponding $a_0 = b_0$ criterion, and `exists_antiProfileSchwartzVelocity_ne_zero_of_exists_ampli` records the contrapositive. Therefore an anti-profile route cannot present an unequal-amplitude zero-flow repair; once amplitudes differ, the candidate is a genuine nonzero-flow problem again. The affine pressure-gauge variant has now been audited as well. The theorem `momentumEquation_equalAmplitudeAntiProfil` shows that replacing the zero pressure slice by any time-only affine-plus-Schwartz gauge still leaves the equal-amplitude anti-profile velocity equal to zero. The BKM bridge theorem `twoModeSchwartzDivergenceFreeIni` then packages the same branch with zero velocity, the stated time-only gauge pressure, the zero vorticity envelope, and zero integral budget. Thus a time-only pressure-gauge repair is classified as pressure gauge freedom on the zero flow, not as a new finite-mode construction. The canonical-gauge refinement `..._affineTimeOnlyPressure_canonicalGaugeWitness` strengthens this to an equality with `zeroFiniteTimeRegularityWitness` after applying the same `addTimeOnlyPressure` gauge, so the branch is not even a distinct witness modulo pressure gauge. Lean also rules out the next nearby pressure repair: a nonzero spatial-affine pressure gradient cannot be inserted on the same zero-flow branch. The gradient calculation `spatialPressureGradient_affineAddScalarSchwartzPressure_spatialAffine` shows that the pure spatial-affine member of the affine-plus-Schwartz pressure class has gradient $c(t)$. The witness-level theorem `ExplicitFiniteTimeRegularityWitness.zeroVelocity_spatialAffinePressure_im` then proves that any finite-time witness with zero velocity and such a pressure must have $c(t) = 0$ throughout its certified slab, and `not_momentumEquation_equalAmplitudeAntiProfileSchwartzVelocity_spati` gives the global momentum-equation contrapositive for the equal-amplitude anti-profile. Thus the spatial-affine pressure coefficient is not gauge freedom on the zero branch; it is forbidden unless it vanishes. Lean now also records two broader BKM-layer classifications. First, `BKMData_zeroVelocity_implies_initialVelocity_zero` proves that any zero-velocity BKM-data package must have zero initial velocity data. Its contrapositive `not_exists_BKMData_zeroVelocity_of_initialVe` rules out pairing the zero-flow branch with nonzero initial data. Second, `BKMData_zeroVelocity_implies_spatialP` proves that for an arbitrary pressure field p, any zero-velocity BKM-data package forces $\nabla p(t, x) = 0$ everywhere on its certified slab. Its contrapositive `not_exists_BKMData_zeroVelocity_of_exists_spatialPress` turns a single nonzero pressure gradient into a no-go theorem for zero-velocity BKM data with that pressure. The combined classification `BKMData_zeroVelocity_implies_initialVelocity_zero_and_spatialPre` bundles these two necessary conditions into one theorem, and the combined contrapositive `not_exists_BKMData_zer` blocks any zero-flow BKM package as soon as either the initial datum is nonzero or the pressure has one certified nonzero spatial-gradient value. The classification is now exact for smooth pressure fields: `BKMData_zeroVelocity_iff_initialVelocity_zero_and_spatialPressureGradient_zero` proves that such a BKM package exists if and only if those two conditions hold. The reverse direction reuses the lower-level `zeroVelocity_finiteTimeWitness_iff_spatialPressureGradient_zero` from `NavierStokesWitnessConstruction` and supplies the zero vorticity envelope, so the BKM layer is conservative over the finite-time zero-flow witness surface. The surviving gauge is now named

as a theorem: `BKMData_zeroVelocity_timeOnlyPressure_iff_initialVelocity_zero` states that for every smooth time-only pressure, zero-velocity BKM data exists exactly for zero initial velocity data. Its constructive half is exposed as `BKMData_zeroVelocity_timeOnlyPressure`, and the matching nonzero-data blocker is `not_exists_BKMData_zeroVelocity_timeOnlyPressure_of_initialVelocity_`
The spatial-affine pressure subclass is exact as well: `BKMData_zeroVelocity_spatialAffinePressure_iff_initia` uses the affine-gradient computation to replace the abstract pressure-gradient condition by the concrete statement that $c(t) = 0$ throughout the certified slab. Its constructive side is exposed as `BKMData_zeroVelocity_spatialAffinePressure_of_affineCoeff_zeroOn`, and the combined no-go theorem `not_exists_BKMData_zeroVelocity_spatialAffinePressure_of_initialVelocity_ne_zero_or` blocks the spatial-affine zero-flow package if either the initial datum is nonzero or the affine pressure coefficient is nonzero at one certified time. The older coefficient-only no-go `not_exists_BKMData_zeroVelocity_spa` remains as the assumption-light direct obstruction. The full affine-plus-localized pressure class is now exact as well. Lean first records the concrete gradient computation `spatialPressureGradient_affineAddScalarSc` reducing the zero-flow pressure force to

$$c(t) + \rho(t)\nabla q(x).$$

Then `BKMData_zeroVelocity_affineAddScalarSchwartzPressure_iff_initialVelocity_zero_and_pressure` proves that zero-flow BKM data with pressure $\langle c(t), x\rangle + \pi(t) + \rho(t)q(x)$ exists exactly when $u_0 = 0$ and this balance vanishes at every point of the certified slab. Its constructive half is named `BKMData_zeroVelocity_affineAddScalarSchwartzPressure_of_pressureGradientBalanceOn`, and the direct balance-failure no-go is `not_exists_BKMData_zeroVelocity_affineAddScalarSchwartzPressure_of_`
Most importantly for the manuscript repair route, `not_exists_BKMData_zeroVelocity_affineAddScalarSchwart` shows that if $\rho(t) \neq 0$ and the spatial gradient of q takes two different values at that time, no choice of affine coefficient can hide the localized pressure force inside a zero-flow BKM package. Lean now also records the positive consequence behind that no-go: `BKMData_zeroVelocity_affineAddScalarSchwartzPressure_` Any surviving zero-flow BKM datum in this pressure class, at any slab time with $\rho(t) \neq 0$, forces $\nabla q(x) = \nabla q(y)$ for every pair of spatial points. The follow-up lemma `schwartzMap_gradient_constant_eq_zero` removes that residual freedom: a Schwartz profile on $\mathbb{R}^3$ cannot have a nonzero globally constant gradient, since boundedness contradicts the affine growth forced by a nonzero derivative. Consequently `BKMData_zeroVelocity_affineAddScalarSchwartzPressure_implies_schwartzPressureGradient_zero_of_` and `BKMData_zeroVelocity_affineAddScalarSchwartzPressure_implies_affineCoeff_zero_of_nonzeroAmp` show that every nonzero-amplitude zero-flow BKM datum in this class has $\nabla q = 0$ and $c(t) = 0$ on that time slice. The newest Lean theorem packages this as the exact collapsed classification `BKMData_zeroVelocity_affineAddScalarSchwartzPressure_iff_initialVelocity_zero_and_affineCoeff_z` zero-flow BKM data in this pressure class exists exactly when $u_0 = 0$, the affine spatial coefficient vanishes throughout the certified slab, and the presence of any nonzero localized amplitude on that slab forces the fixed Schwartz pressure gradient to vanish everywhere. Thus the affine repair has no remaining constant-gradient loophole: a nonzero localized pressure force is not merely hard to tune, but excluded by the exact classification. The same collapse is now also available one layer lower, at the raw momentum equation for the equal-amplitude anti-profile branch: `momentumEquation_equalAmplitudeAntiProfileSchwartzVelocity_affineAddScalarSchwartzPressure_iff_` Because `twoModeSchwartzVelocity a a f (-f)` is definitionally a zero velocity field after the anti-profile cancellation theorem, the full pointwise equation with pressure $\langle c(t), x\rangle + \pi(t) + \rho(t)q(x)$ holds exactly when $c(t) = 0$ for every time and any nonzero localized amplitude forces the fixed Schwartz pressure gradient to vanish everywhere. Hence the pressure repair is already collapsed before adding BKM witness packaging. The BKM witness layer has now been checked explicitly for that same branch: `BKMData_equalAmplitudeAntiProfileSchwartzVelocity_affineAddScalarSchwartzPressure_iff_af`

states that BKM data with velocity `twoModeSchwartzVelocity a a f (-f)` and the full affine-plus-localized pressure exists exactly under the slabwise collapsed conditions. This removes the last distinction between the anti-profile BKM premise and the zero-flow pressure classification. The reusable normalization underneath this result is now explicit: `BKMData_equalAmplitudeAntiProfileSchwartzVelocity_iff_z` and `BKMData_equalAmplitudeAntiProfileInitialVelocity_iff_zeroVelocity` identify the cancelled velocity and cancelled initial-datum witness surfaces with zero-initial, zero-velocity BKM data for an arbitrary pressure field. The affine-plus-localized pressure classifications now factor through these general iff statements rather than through a pressure-specific rewrite. This normalization has also been promoted to an arbitrary-pressure exact boundary: `BKMData_equalAmplitudeAntiProfileSchwartzVelo` and `BKMData_equalAmplitudeAntiProfileInitialVelocity_iff_spatialPressureGradient_zero` say that, for any smooth pressure field, the anti-profile BKM-data package exists exactly when the pressure has zero spatial gradient on the certified slab. The companion no-go theorems `not_exists_BKMData_equalAmplitudeAntiProfileSchwartzVelocity_of_exists_spatialPressureGradient_` and `not_exists_BKMData_equalAmplitudeAntiProfileInitialVelocity_of_exists_spatialPressureGradie` remove the affine-plus-localized ansatz from the obstruction: any proposed pressure family with a nonzero slabwise spatial gradient is already impossible for the cancelled branch. That obstruction now reaches the repaired clause/data separation layer: `ExplicitFiniteEnergyBKMContinuationClause_zero_and_not` and `ExplicitFiniteEnergyBKMContinuationClause_equalAmplitudeAntiProfileInitialVelocity_and_not_` state that the repaired finite-energy BKM clause remains inhabited at the zero or syntactic anti-profile initial datum, while matching BKM data is impossible for any pressure with nonzero slabwise spatial gradient. Thus this repair gap is no longer tied to the affine-plus-localized pressure family. Lean also records the exact repaired-clause/BKM-data boundary for arbitrary smooth pressures: `ExplicitFiniteEnergyBKMContinuationClause_zero_and_BKMData_equalAmplitudeAntiProfileSchwartzVel` and `ExplicitFiniteEnergyBKMContinuationClause_equalAmplitudeAntiProfileInitialVelocity_and_BKMD` These say that the conjunction of the repaired finite-energy BKM clause and a matching anti-profile BKM-data package exists exactly when the arbitrary pressure field is spatially constant on the certified slab. The previous no-go statements are therefore not merely one-way obstructions: they are the negative face of the exact boundary for this clause-plus-data repair route. Lean now also pins the raw constructor-input layer for arbitrary pressure: `momentumEquation_equalAmplitudeAntiProfileSchwartzVelocit` states that the full pointwise momentum equation for the cancelled equal-amplitude anti-profile velocity holds exactly when the arbitrary pressure has zero spatial gradient everywhere. Consequently `ExplicitFiniteEnergyBKMContinuationClause_equalAmplitudeAntiProfileInitialVelocity_and_not_mome` keeps the repaired finite-energy BKM clause inhabited over the syntactic anti-profile initial datum, but rules out the momentum equation for any arbitrary pressure with a nonzero spatial gradient. This blocks a pressure repair before the BKM-data witness package is even formed, and it does so without returning to the affine-plus-localized ansatz. The exact version is `ExplicitFiniteEnergyBKMContinuationClause_` the repaired finite-energy BKM clause over the syntactic anti-profile initial datum together with the raw pointwise momentum equation exists exactly when the arbitrary pressure has zero spatial gradient everywhere. Thus the constructor-input route has a complete pressure-independent boundary, not only a negative counterexample family. Finally, the combined route-facing package is exact as well: `ExplicitFiniteEnergyBKMContinuationClause_equalAmplitudeAntiProfileInitialVelocity_and_BKMData_` states that adding matching BKM data to the repaired clause and raw constructor equation still leaves exactly the global zero-gradient condition. The BKM-data surface only asks for slabwise zero gradient by itself, but it cannot weaken the global pressure restriction imposed by the pointwise momentum equation. For the concrete affine-plus-localized Schwartz pressure family, Lean now also exposes the same exact paired boundary directly: `ExplicitFiniteEnergyBKMContinuationClause_zero_and_BKMData_equal` and `ExplicitFiniteEnergyBKMContinuationClause_equalAmplitudeAntiProfileInitialVelocity_and_BKMD` Thus the repaired finite-energy BKM clause plus a matching anti-profile BKM-data package exists ex-

actly under the collapsed slabwise pressure conditions: the affine coefficient vanishes on the certified slab, and any active localized amplitude forces the fixed Schwartz pressure gradient to vanish. This direct theorem prevents a future argument from treating the bare repaired clause as if it certified the affine-plus-localized pressure ansatz. The exact classification is now also exposed over the syntactic anti-profile initial datum: `BKMData_equalAmplitudeAntiProfileInitialVelocity_affineAddScalarSchwartzP`
Its left-hand side uses `ExplicitFiniteTimeRegularityWitness` ν (`twoModeSchwartzInitialVelocity` $(a\,0)$ $(a\,0)$
but the right-hand side is still exactly the same slabwise zero-flow pressure classification. The companion direct no-go theorem `not_exists_BKMData_equalAmplitudeAntiProfileInitialVelocity_affineAddScala`
rules out that syntactic witness package under the same bad-pressure hypotheses. Lean also records the direct no-go form `not_exists_BKMData_equalAmplitudeAntiProfileSchwartzVelocity_affineAddScalarS`
if the affine coefficient is nonzero somewhere on the slab, or if a nonzero localized amplitude is paired with a nonzero spatial gradient of the fixed Schwartz pressure, then the anti-profile BKM-data package does not exist. Lean now separates this obstruction from the bare repaired continuation clause:
`ExplicitFiniteEnergyBKMContinuationClause_zero_and_not_exists_BKMData_equalAmplitudeAntiProfil`
proves that the repaired finite-energy BKM continuation clause for the zero datum can still hold while the structured anti-profile BKM-data package with the proposed pressure is impossible. Thus a repair that cites only the bare continuation clause has forgotten exactly the velocity and pressure fields needed to certify the manuscript ansatz. The same obstruction is now stated over the syntactic anti-profile initial datum itself: `ExplicitFiniteEnergyBKMContinuationClause_equalAmplitudeAntiProfileInitia`
inhabits the repaired finite-energy BKM clause for `twoModeSchwartzInitialVelocity` $(a\,0)$ $(a\,0)$ f $(-f)$,
but rules out the matching structured BKM-data package over that same initial datum when the affine-plus-localized pressure is bad. This closes the syntactic repair in which the zero-normalized initial datum is presented as a separate finite-mode BKM witness surface. Lean also blocks the corresponding constructor-input repair: `ExplicitFiniteEnergyBKMContinuationClause_equalAmplitudeAntiProfil`
inhabits the repaired finite-energy BKM clause for the cancelled anti-profile initial datum, but proves that the full pointwise momentum equation needed to use that anti-profile velocity with the bad pressure cannot hold. Therefore the generic finite-mode BKM clause constructor cannot rescue the bad pressure family by bypassing the structured BKM-data statement. The BKM clause module now records the underlying normalization explicitly: `ExplicitBKMContinuationClause_equalAmplitudeAntiProfileI`
and `ExplicitFiniteEnergyBKMContinuationClause_equalAmplitudeAntiProfileInitialVelocity_iff_zero`
identify the unrepaired and repaired BKM clauses for an equal-amplitude anti-profile initial slice with the corresponding zero-datum clauses. This keeps the anti-profile continuation clause from being treated as a separate finite-mode object carrying pressure information. At the clause level, `finiteEnergyConcreteNavierStokesGlobalRegularityClause_iff` now makes the same repaired surface explicit: for a fixed finite-energy datum, the repaired structured fully concrete clause is just the repaired explicit regularity clause packaged through the concrete target shell. Lean now also packages the repaired/unrepaired structured-clause equivalence on that same fixed datum directly as `finiteEnergyConcreteNavierStokesGlobalRegularityClause_iff_of_finiteInitialKineticEnergy`,
so the repaired concrete shell is no longer only compared to the unrepaired one through the explicit surface. The equation layer now also exposes the forgetful directions themselves: `ExplicitFiniteEnergyAdmissibleNa`
packages the repaired explicit-to-unrepaired explicit step on a fixed finite-energy datum, while `finiteEnergyConcreteNavierStokesGlobalRegularityClause_implies_concreteNavierStokesGlobalRegula`
and `concreteNavierStokesGlobalRegularityClause_implies_finiteEnergyConcreteNavierStokesGlobalRe`
do the same directly on the structured concrete shell. At the theorem surface, Lean now also packages the structured-to-explicit export directly as `ConcreteNavierStokesMillenniumTarget_implies_ExplicitConcre`
and `FiniteEnergyConcreteNavierStokesMillenniumTarget_implies_ExplicitFiniteEnergyAdmissibleNavi`
so downstream continuation files can reuse these bridges instead of reopening the corresponding `_iff_explicit` equivalences locally. It now also packages the mixed bridge `ConcreteNavierStokesMillenniumTarg`

so downstream repaired continuation exports no longer need to reconstruct the two-step path through the repaired structured theorem surface either. The continuation layer now also uses that repaired input condition analytically: `ExplicitFiniteTimeRegularityWitness.finiteInitialKineticEnergy` and `not_nonempty_ExplicitFiniteTimeRegularityWitness_of_not_finiteInitialKineticEnergy` show that any actual slab witness on a nonnegative horizon already forces finite initial energy. Lean now packages that fact directly into repaired continuation surfaces: `ExplicitFiniteEnergyUniformVorticityCont` `ExplicitFiniteEnergyUniformVorticityContinuationTarget`, `ExplicitFiniteEnergyBKMContinuationClaus` and `ExplicitFiniteEnergyBKMContinuationTarget`. The equivalence theorems `ExplicitFiniteEnergyUniformV` and `ExplicitFiniteEnergyBKMContinuationClause_iff_of_nonneg_horizon` show that on nonnegative slabs these repaired clauses coincide with the older ones, precisely because any real slab witness already carries finite initial energy. Consequently the repaired theorem target now lands directly in both repaired continuation targets via `ExplicitFiniteEnergyAdmissibleNavierStokesMillenniumTarget_imp` and `ExplicitFiniteEnergyAdmissibleNavierStokesMillenniumTarget_implies_finiteEnergyBKMContinuat` while the older unrepaired whole-space theorem surfaces now also reach these same repaired continuation targets directly through `ExplicitConcreteNavierStokesMillenniumTarget_implies_finiteEnergyUnifor` `ConcreteNavierStokesMillenniumTarget_implies_finiteEnergyUniformVorticityContinuationTarget`, `ExplicitConcreteNavierStokesMillenniumTarget_implies_finiteEnergyBKMContinuationTarget`, and `ConcreteNavierStokesMillenniumTarget_implies_finiteEnergyBKMContinuationTarget`, and the same fixed-horizon directness is now exposed at clause level via `ExplicitFiniteEnergyAdmissibleNavierSt` `ExplicitConcreteNavierStokesMillenniumTarget_implies_finiteEnergyUniformVorticityContinuationC` `ExplicitFiniteEnergyAdmissibleNavierStokesMillenniumTarget_implies_finiteEnergyBKMContinuation` and `ExplicitConcreteNavierStokesMillenniumTarget_implies_finiteEnergyBKMContinuationClause`, while the older unrepaired continuation layer now also exposes the analogous target-level admissibility lifts `ExplicitUniformVorticityContinuationTarget_implies_finiteEnergyUniformVorticityContinuation` and `ExplicitBKMContinuationTarget_implies_finiteEnergyBKMContinuationTarget`, as well as the fixed-horizon corollaries `ExplicitUniformVorticityContinuationClause_implies_finiteEnergy` and `ExplicitBKMContinuationClause_implies_finiteEnergy`, so the same unrepaired→repaired step is now exposed uniformly on both the target and clause surfaces. The unrepaired continuation layer also exposes the analogous fixed-horizon corollaries `ExplicitConcreteNavierStokesMillenniumTarget_impl` `ConcreteNavierStokesMillenniumTarget_implies_uniformVorticityContinuationClause`, `ExplicitConcret` and `ConcreteNavierStokesMillenniumTarget_implies_BKMContinuationClause`. The uniform side now also exposes the lower-level clause repackaging bridges `ExplicitConcreteNavierStokesRegularityClause` and `ExplicitFiniteEnergyAdmissibleNavierStokesRegularityClause_implies_ExplicitFiniteEnergyUnif` as well as their fixed-datum all-horizons families `ExplicitConcreteNavierStokesRegularityClause_implies_Exp` and `ExplicitFiniteEnergyAdmissibleNavierStokesRegularityClause_implies_ExplicitFiniteEnergyUnif` so both fixed-horizon uniform clauses and their fixed-datum target families can be obtained directly from the imported whole-space explicit clause surfaces. The theorem surface now re-exports those fixed-datum families as `ExplicitConcreteNavierStokesMillenniumTarget_implies_uniformVorticityContinu` and `ExplicitFiniteEnergyAdmissibleNavierStokesMillenniumTarget_implies_finiteEnergyUniformVorti` so downstream code can stay on the theorem-target API while still recovering the full $\forall T$ continuation-clause family without reopening the whole-space clause binder locally. The same pattern now also covers the structured and repaired theorem surfaces via `FiniteEnergyConcreteNavierStokesMillenniumTarget_impl` `ExplicitConcreteNavierStokesMillenniumTarget_implies_finiteEnergyUniformVorticityContinuationC` `ConcreteNavierStokesMillenniumTarget_implies_finiteEnergyUniformVorticityContinuationClause_al` and `ConcreteNavierStokesMillenniumTarget_implies_uniformVorticityContinuationClause_allHorizons` The BKM side now also exposes the lower-level clause repackaging bridges `ExplicitConcreteNavierStokesRegular` and `ExplicitFiniteEnergyAdmissibleNavierStokesRegularityClause_implies_ExplicitFiniteEnergyBKMC` as well as their fixed-datum all-horizons families `ExplicitConcreteNavierStokesRegularityClause_implies_Exp`

and ExplicitFiniteEnergyAdmissibleNavierStokesRegularityClause_implies_ExplicitFiniteEnergyBKMC
so both fixed-horizon BKM clauses and their fixed-datum target families can be obtained directly from the imported whole-space explicit clause surfaces. The theorem surface now re-exports those fixed-datum families as ExplicitConcreteNavierStokesMillenniumTarget_implies_BKMContinuationClause_allHo
and ExplicitFiniteEnergyAdmissibleNavierStokesMillenniumTarget_implies_finiteEnergyBKMContinuat
so the same direct $\forall T$ access is available on the explicit BKM theorem-target API as well. The same pattern now also covers the structured and repaired theorem surfaces via FiniteEnergyConcreteNavierStokesMill
ExplicitConcreteNavierStokesMillenniumTarget_implies_finiteEnergyBKMContinuationClause_allHoriz
ConcreteNavierStokesMillenniumTarget_implies_finiteEnergyBKMContinuationClause_allHorizons,
and ConcreteNavierStokesMillenniumTarget_implies_BKMContinuationClause_allHorizons,
and the zero-data BKM clause baselines now factor through those reusable bridges instead of reopening the witness and envelope binders locally. The repaired continuation layer itself now contains the same BKM-to-uniform bridge as the unrepaired one: ExplicitFiniteEnergyBKMContinuationClause_implies_fi
and ExplicitFiniteEnergyBKMContinuationTarget_implies_finiteEnergyUniformVorticityContinuationT
transport repaired BKM data directly to repaired uniform-vorticity data, and the BKM target surface now also re-exports the corresponding fixed-datum uniform clause families as ExplicitBKMContinuationTarget_imp
ExplicitBKMContinuationTarget_implies_finiteEnergyUniformVorticityContinuationClause_allHorizon
and ExplicitFiniteEnergyBKMContinuationTarget_implies_finiteEnergyUniformVorticityContinuationC
so downstream code can recover the entire unrepaired or repaired uniform clause family directly from a BKM target assumption without first passing through the target-valued bridge theorem, while the same file now also records the composed theorem-surface exports ExplicitConcreteNavierStokesMillenniumTarge
ExplicitConcreteNavierStokesMillenniumTarget_implies_finiteEnergyUniformVorticityContinuationTa
their repaired/concrete analogues, and the matching fixed-datum clause families such as ExplicitConcreteNavierSt
and ConcreteNavierStokesMillenniumTarget_implies_finiteEnergyUniformVorticityContinuationClause
so the BKM route remains explicit even at the whole-theorem and whole-family surface, while the older bridge theorems to the unrepaired clauses survive as corollaries on nonnegative horizons. These older nonnegative-horizon forgetful steps are now also packaged as direct theorem-target families
ExplicitFiniteEnergyUniformVorticityContinuationTarget_implies_uniformVorticityContinuationClau
ExplicitFiniteEnergyAdmissibleNavierStokesMillenniumTarget_implies_uniformVorticityContinuation
FiniteEnergyConcreteNavierStokesMillenniumTarget_implies_uniformVorticityContinuationClause_all
ExplicitFiniteEnergyBKMContinuationTarget_implies_BKMContinuationClause_allNonnegHorizons,
ExplicitFiniteEnergyAdmissibleNavierStokesMillenniumTarget_implies_BKMContinuationClause_allNon
FiniteEnergyConcreteNavierStokesMillenniumTarget_implies_BKMContinuationClause_allNonnegHorizon
and ExplicitFiniteEnergyBKMContinuationTarget_implies_uniformVorticityContinuationClause_allNon
so downstream code can use a single repaired target and then recover the entire "for every nonnegative slab" family without reopening the fixed-horizon forgetful lemma by hand. These repaired targets are also already operational on the concrete zero family: ExplicitFiniteEnergyUniformVorticityContinuationTarg
and ExplicitFiniteEnergyBKMContinuationTarget_implies_zeroGlobalOutput apply them directly to the canonical zero slab witness together with the now explicit input fact finiteInitialKineticEnergy_zer
The API is now also clean one layer lower: ExplicitFiniteEnergyUniformVorticityContinuationClause_implie
and ExplicitFiniteEnergyBKMContinuationClause_implies_zeroGlobalOutput show that the repaired clause surface itself already has the same concrete zero-data operational consequence as the unrepaired one. The same is now true for the time-gauge sanity check: ExplicitUniformVorticityContinuationTa
and ExplicitBKMContinuationTarget_implies_zeroGlobalOutput_timeOnlyPressure now package the unrepaired target-level version directly, while ExplicitFiniteEnergyUniformVorticityContinuationClau
and ExplicitFiniteEnergyBKMContinuationClause_implies_zeroGlobalOutput_timeOnlyPressure
show that the repaired clause layer also survives adding any smooth time-dependent pressure gauge on a larger zero slab and restricting back down. Lean now also packages the constant-pressure special

cases directly on all four continuation surfaces via `ExplicitUniformVorticityContinuationClause_implies_zero`
`ExplicitUniformVorticityContinuationTarget_implies_zeroGlobalOutput_constantPressure`,
`ExplicitBKMContinuationClause_implies_zeroGlobalOutput_constantPressure`, and `ExplicitBKMContinua`
plus their repaired finite-energy analogues. The same zero-data operational API now exists one layer
higher on the actual whole-space theorem surfaces: `ExplicitConcreteNavierStokesMillenniumTarget_implies_`
`ConcreteNavierStokesMillenniumTarget_implies_zeroGlobalOutput`, and their repaired finite-
energy analogues. That whole-space layer is now also normalized through the same gauge interface:
`ExplicitConcreteNavierStokesMillenniumTarget_implies_zeroGlobalOutput_addPressureOfZeroSpatialG`
`ConcreteNavierStokesMillenniumTarget_implies_zeroGlobalOutput_addPressureOfZeroSpatialGradient`,
and the repaired finite-energy analogues package the generic zero-spatial-gradient transport once,
and the older time-only and constant-pressure theorem-surface exports are now just wrappers around
that shared zero-data theorem. The repaired target surface now packages the same fact directly via
`ExplicitFiniteEnergyUniformVorticityContinuationTarget_implies_zeroGlobalOutput_timeOnlyPressur`
and `ExplicitFiniteEnergyBKMContinuationTarget_implies_zeroGlobalOutput_timeOnlyPressure`.
The continuation layer then packages the whole point directly as `linearShearVelocityField_exhibits_uniformCa`
there exist explicit u, p, B satisfying the smoothness, momentum, incompressibility, initial-condition,
and uniform-vorticity parts of the finite-time witness surface, and the only missing slot is bounded
kinetic energy. The BKM file then sharpens the same point on the stricter time-integrated surface:
`linearShearVelocityField_has_constantBKMEnvelope` proves that the same field carries the lit-
eral constant envelope $\Omega(t) = |a|$ with integral bound $T|a|$, and `linearShearVelocityField_exhibits_BKMCandida`
packages it as a full BKM-side candidate whose only missing slot is again bounded kinetic energy.
Lean now also packages the precise mismatch directly as `linearShearInitialVelocity_exhibits_BKMCandidate_`
the explicit BKM-side candidate exists, but the finite-time witness type is still empty. The continua-
tion file also defines the associated explicit density `linearShearKineticEnergyDensity` and proves
`not_integrable_linearShearKineticEnergyDensity` by an exact scaling argument on $\mathbb{R}^3$: if the
density were integrable, Haar scaling by $x \mapsto 2x$ would force its integral to be both $4I$ and $I/8$, hence
zero, contradicting positivity. This is then pushed from the special shear ansatz to any field matching
nonzero linear-shear initial data at $t = 0$ via `not_integrable_kineticEnergyDensity_zero_of_matchesInitialV`
`not_boundedKineticEnergyUpTo_of_matchesInitialVelocity_linearShearInitialVelocity`, and
`not_boundedKineticEnergy_of_matchesInitialVelocity_linearShearInitialVelocity`. Con-
sequently `not_nonempty_ExplicitFiniteTimeRegularityWitness_linearShearInitialVelocity`,
`not_ExplicitConcreteNavierStokesGlobalOutput_linearShearInitialVelocity`, and `not_ExplicitConcret`
show that the current explicit bounded-energy theorem surfaces also reject this genuinely non-
constant initial-data family. Lean now also makes that family exact on the same route-facing surfaces.
`ExplicitConcreteNavierStokesGlobalOutput_linearShearInitialVelocity_iff`, `ExplicitConcreteNavierS`
and `concreteNavierStokesGlobalRegularityClause_linearShearInitialVelocity_iff` show
that the explicit global-output surface, the unrepaired explicit regularity clause, and the fully con-
crete structured clause hold for `linearShearInitialVelocity a` if and only if $a = 0$. So the first
genuinely nonconstant obstruction family now collapses exactly to the zero datum on the concrete the-
orem surfaces too, not merely at the nonzero obstruction level. Lean now extends the same exact col-
lapse to the affine full-drift family $x \mapsto (ax_1, b, c)$. Theorems `ExplicitConcreteNavierStokesGlobalOutput_linea`
`ExplicitConcreteNavierStokesRegularityClause_linearShearFullDriftInitialVelocity_iff`,
and `concreteNavierStokesGlobalRegularityClause_linearShearFullDriftInitialVelocity_iff`
show that these three route-facing theorem surfaces hold for `linearShearFullDriftInitialVelocity`
`a b c` if and only if $a = 0$, $b = 0$, and $c = 0$. So even after adding both constant drift directions, the
affine linear-shear extension still collapses exactly to the zero datum on the theorem surfaces them-
selves. Lean now also isolates the one-drift affine linear-shear subfamilies $x \mapsto (ax_1, 0, b)$ and $x \mapsto$
$(ax_1, b, 0)$. Theorems `ExplicitConcreteNavierStokesGlobalOutput_linearShearVerticalDriftInitialVeloc`

ExplicitConcreteNavierStokesRegularityClause_linearShearVerticalDriftInitialVelocity_iff, concreteNavierStokesGlobalRegularityClause_linearShearVerticalDriftInitialVelocity_iff, and their horizontal-drift analogues show that these route-facing theorem surfaces hold for each one-drift branch exactly when both coefficients vanish. Lean now also proves the lower-layer exact statements finiteInitialKineticEnergy_linearShearFullDriftInitialVelocity_iff and, on nonnegative slabs, nonempty_ExplicitFiniteTimeRegularityWitness_linearShearFullDriftInitialVelocity_i together with the corresponding base, vertical-drift, and horizontal-drift specializations. So the whole affine linear-shear hierarchy now collapses exactly to the zero datum already at the whole-space finite-energy surface and at the finite-time witness type, not only at the outer route-facing clauses. Lean now also isolates the base heat-shear family $x \mapsto (a\sin(kx_1), 0, 0)$. Theorems ExplicitConcreteNavierStokesGlobalOutput_heatShearInitialVelocity_iff, ExplicitConcreteNavierSto and concreteNavierStokesGlobalRegularityClause_heatShearInitialVelocity_iff show that the same three route-facing theorem surfaces hold for heatShearInitialVelocity a k exactly when $ak = 0$. Lean now also proves the lower-layer exact statements finiteInitialKineticEnergy_heatShearInitialV and, on nonnegative slabs, nonempty_ExplicitFiniteTimeRegularityWitness_heatShearInitialVelocity_iff So the bare oscillatory heat-shear family already collapses exactly to the zero datum at the route-facing surfaces, at the whole-space finite-energy surface, and at the finite-time witness type itself. Lean now also isolates the vertical-drift heat-shear subfamily $x \mapsto (a\sin(kx_1), 0, c)$. Theorems ExplicitConcreteNavierStokesGlobalOutput_heatShearVerticalDriftInitialVelocity_iff, ExplicitConcreteNavierStokesRegularityClause_heatShearVerticalDriftInitialVelocity_iff, and concreteNavierStokesGlobalRegularityClause_heatShearVerticalDriftInitialVelocity_iff show that the same three route-facing theorem surfaces hold for heatShearVerticalDriftInitialVelocity a k c exactly when $c = 0$ and $ak = 0$. Lean now also proves the lower-layer exact statements finiteInitialKineticEnergy_heatShearVerticalDriftInitialVelocity_iff and, on nonnegative slabs, nonempty_ExplicitFiniteTimeRegularityWitness_heatShearVerticalDriftInitialVelocity_if So this simpler oscillatory family already collapses exactly to the zero datum at the route-facing surfaces, at the whole-space finite-energy surface, and at the finite-time witness type itself. Lean now also isolates the streamwise-drift heat-shear subfamily $x \mapsto (d + a\sin(kx_1), 0, 0)$. Theorems ExplicitConcreteNavierStokesGlobalOutput_heatShearStreamwiseDriftInitialVelocity_iff, ExplicitConcreteNavierStokesRegularityClause_heatShearStreamwiseDriftInitialVelocity_iff, and concreteNavierStokesGlobalRegularityClause_heatShearStreamwiseDriftInitialVelocity_iff show that the same three route-facing theorem surfaces hold for heatShearStreamwiseDriftInitialVelocity a k d exactly when $d = 0$ and $ak = 0$. Lean now also proves the lower-layer exact statements finiteInitialKineticEnergy_heatShearStreamwiseDriftInitialVelocity_iff and, on nonnegative slabs, nonempty_ExplicitFiniteTimeRegularityWitness_heatShearStreamwiseDriftInitialVeloci So this horizontally drifted oscillatory family already collapses exactly to the zero datum at the route-facing surfaces, at the whole-space finite-energy surface, and at the finite-time witness type itself. Lean now does the same for the damped heat-shear family with full constant drift $x \mapsto (d + a\sin(kx_1), 0, c)$. Theorems ExplicitConcreteNavierStokesGlobalOutput_heatShearFullDriftInitialVeloci ExplicitConcreteNavierStokesRegularityClause_heatShearFullDriftInitialVelocity_iff, and concreteNavierStokesGlobalRegularityClause_heatShearFullDriftInitialVelocity_iff show that the same three route-facing theorem surfaces hold for heatShearFullDriftInitialVelocity a k d c exactly when $d = 0$, $c = 0$, and $ak = 0$. So even after adding both constant drifts to the oscillatory heat-shear profile, the whole-space route-facing surfaces still collapse exactly to the zero datum. Lean now also proves the lower-layer exact statements finiteInitialKineticEnergy_heatShearFullDriftInitia and, on nonnegative slabs, nonempty_ExplicitFiniteTimeRegularityWitness_heatShearFullDriftInitialVel So the same boundary is exact already at the whole-space finite-energy surface and at the witness type itself for the non-transport full-drift family, not just at the outer route-facing clauses.

Lean now also isolates the transported heat-shear subfamily $x \mapsto (a\sin(kx_1), b, 0)$. Theorems `ExplicitConcreteNavierStokesGlobalOutput_heatShearTransportInitialVelocity_iff`, `ExplicitConcrete` and `concreteNavierStokesGlobalRegularityClause_heatShearTransportInitialVelocity_iff` show that the same three route-facing theorem surfaces hold for `heatShearTransportInitialVelocity a k b` exactly when $b = 0$ and $ak = 0$. Lean now also proves the lower-layer exact statements `finiteInitialKineticEnergy_heatShearTransportInitialVelocity_iff` and, on nonnegative slabs, `nonempty_ExplicitFiniteTimeRegularityWitness_heatShearTransportInitialVelocity_iff`. So this transported oscillatory family already collapses exactly to the zero datum at the route-facing surfaces, at the whole-space finite-energy surface, and at the finite-time witness type itself. Lean now does the same for the transported full-drift heat-shear family with datum $x \mapsto (d + a\sin(kx_1), b, c)$. Theorems `ExplicitConcreteNavierStokesGlobalOutput_heatShearTransportFullDriftInitialVelocity_if` `ExplicitConcreteNavierStokesRegularityClause_heatShearTransportFullDriftInitialVelocity_iff`, and `concreteNavierStokesGlobalRegularityClause_heatShearTransportFullDriftInitialVelocity_iff` show that the same three route-facing theorem surfaces hold for `heatShearTransportFullDriftInitialVelocity a k b d c` exactly when $b = 0$, $d = 0$, $c = 0$, and $ak = 0$. So adding transport in the second coordinate does not reopen the route: the only surviving case is again the zero datum. Lean now also proves the lower-layer exact statements `finiteInitialKineticEnergy_heatShearTransportFullDriftInitialVelocit` and, on nonnegative slabs, `nonempty_ExplicitFiniteTimeRegularityWitness_heatShearTransportFullDriftI` So the same boundary is exact already at the whole-space finite-energy surface and at the witness type itself, not just at the outer route-facing clauses. Lean now also proves that the repaired explicit finite-energy regularity clause itself is completely nonrestrictive on these explicit families. Theorems `ExplicitFiniteEnergyAdmissibleNavierStokesRegularityClause_constantInitialVelocity_all`, `...linearShearFullDriftInitialVelocity_all`, `...heatShearFullDriftInitialVelocity_all`, and `...heatShearTransportFullDriftInitialVelocity_all`, together with the base and one-drift specializations derived from them, show that the repaired clause always holds on the constant, affine linear-shear, full-drift heat-shear, and transported full-drift heat-shear hierarchies. Combined with the exact finite-energy collapse theorems above, this means the repaired clause contributes no further obstruction on these ansatz classes: it is realized at the zero datum and vacuous everywhere else. Lean now pushes the same point one layer higher, to the repaired structured fully concrete theorem surface itself. Theorems `finiteEnergyConcreteNavierStokesGlobalRegularityClause_constantInitialVeloc` `...linearShearFullDriftInitialVelocity_all`, `...heatShearFullDriftInitialVelocity_all`, and `...heatShearTransportFullDriftInitialVelocity_all` show that once one of these explicit ansatz data is packaged as an actual `FiniteEnergyAdmissibleNavierStokesProblemData`, the repaired structured clause already holds automatically. So on these families even the repaired structured theorem surface contributes no additional analytic burden beyond the finite-energy admissibility gate. At the same time, Lean now exposes a precise target-shape caveat: the current continuation clauses themselves remain true on such impossible data, but only vacuously, because they quantify over a finite-time witness type that is already empty there. This is now stated explicitly by `ExplicitUniformVorticityContinuationClause_of_not_nonempty_finiteTimeWitness` and `ExplicitBKMContinuationClause_of_not_nonempty_finiteTimeWitness`, which isolate the vacuity mechanism itself, and then concretely by `ExplicitUniformVorticityContinuationClause_constantInit` and `ExplicitBKMContinuationClause_constantInitialVelocity`; Lean now also shows the same mismatch on the nonconstant shear family via `ExplicitUniformVorticityContinuationClause_linearShearIni` and `ExplicitBKMContinuationClause_linearShearInitialVelocity`; and it packages both mismatches directly as `ExplicitUniformVorticityContinuationClause_constantInitialVelocity_without_regu` and `ExplicitBKMContinuationClause_constantInitialVelocity_without_regularity`, as well as `ExplicitUniformVorticityContinuationClause_linearShearInitialVelocity_without_regularity` and `ExplicitBKMContinuationClause_linearShearInitialVelocity_without_regularity`. The

repaired continuation clauses inherit a second vacuity mode from the finite-energy input filter itself: `ExplicitFiniteEnergyUniformVorticityContinuationClause_of_not_finiteInitialKineticEnergy` and `ExplicitFiniteEnergyBKMContinuationClause_of_not_finiteInitialKineticEnergy` package the generic logical fact, while the constant and shear families are now also recorded directly on those repaired surfaces by `ExplicitFiniteEnergyUniformVorticityContinuationClause_constantInitialVelo` `ExplicitFiniteEnergyBKMContinuationClause_constantInitialVelocity_without_regularity`, `ExplicitFiniteEnergyUniformVorticityContinuationClause_linearShearInitialVelocity_without_regul` and `ExplicitFiniteEnergyBKMContinuationClause_linearShearInitialVelocity_without_regularity`. So the present continuation surfaces are not by themselves existence claims; the zero family supplies one honest concrete nonvacuity route, and linear shear now supplies an honest nonzero PDE-side candidate with explicit vorticity control, but genuinely nonzero inhabited finite-time witnesses still fail here precisely at the bounded-energy slot. This again isolates a defect in the present local whole-space continuation surface; it does not by itself rule out reasonable torus or Schwartz-restricted adaptations of the Goertzel route. Lean now pushes the same point onto a genuinely time-dependent transported heat-shear family. The new file `NavierStokesHeatShearTransportFullDriftWitnessObstruction.lean` proves that for nonnegative viscosity and any smooth time-only pressure gauge $p(t,x) = \pi(t)$, the exact pair $(u,p) = (\texttt{heatShearTransportFullDriftVelocityField}, \pi(t))$ satisfies the smoothness, slab momentum, incompressibility, initial-condition, and explicit slab-vorticity fields required by `ExplicitFiniteTimeRegularityWitness`; the vorticity bound is again literal, $B = |ak|$. But if the transport speed $b \neq 0$, then `not_exists_heatShearTransportFullDriftVelocityField_finiteTimeWitness_of` and `heatShearTransportFullDriftVelocityField_timeOnlyPressure_exhibits_nonEnergyFiniteTimeField` show that no exact witness can carry that velocity on any nonnegative slab. So even on this more route-shaped whole-space ansatz, the obstruction is still the same concrete one: the exact witness fails at `bounded_energy_on`, not at a pressure-gauge choice or a missing PDE identity. Cleanup note, 2026-05-05: this full-drift witness-layer file pair is now archived under `/home/zarclaw/automation/aihub/archive/mil-cleanup-20260505/`. It is retained as a historical bounded-energy obstruction, but it is no longer an active route module: later residual-curl target compositions and the multi-mode BKM-data over-determination obstruction carry the route-relevant pressure and target-surface failures more directly. Historically, the same archived file also packaged the repaired-to-unrepaired uniform forgetful step on nonnegative slabs directly as `ExplicitFiniteEnergyUniformVorticityContinuationClause_implies_uniformVorticityContinuationClau` and its target-to-fixed-horizon corollary `ExplicitFiniteEnergyUniformVorticityContinuationTarget_implies_` so the repaired whole-space theorem-surface export on a fixed nonnegative horizon can now factor through one reusable continuation-target theorem instead of reopening the repaired/unrepaired equivalence locally. The repaired uniform clause now also inherits the same restriction-based horizon monotonicity API directly: `ExplicitFiniteEnergyUniformVorticityContinuationClause_apply_of_uniformVortic` and `ExplicitFiniteEnergyUniformVorticityContinuationClause_mono_horizon` show that once finite initial energy is fixed, the repaired continuation surface restricts along shorter slabs exactly as the unrepaired one does.

The next file, `NavierStokesBKMContinuationTarget.lean`, sharpens that surface toward the actual BKM shape. Instead of a uniform-in-time bound, it asks for an explicit scalar envelope $\Omega : \mathbb{R} \to \mathbb{R}$ such that $\|\omega(t,x)\| \leq \Omega(t)$ on the slab, $\Omega(t) \geq 0$ on $[0,T]$, and Ω is integrable on $[0,T]$ with an explicit integral bound. This is still only a target, not a proof, but it is a more theorem-like continuation surface than the earlier uniform bound. Lean again proves the sanity theorem that the full explicit global theorem target subsumes this BKM-style continuation target, and hence so does the structured fully concrete theorem target. In this file we now also have a nontrivial bridge theorem: a uniform slab vorticity bound gives an explicit constant-in-time envelope that is integrable on $[0,T]$, and therefore the BKM-style continuation target implies the uniform-vorticity continuation target. So

this is no longer only parallel one-way subsumption from the full global theorem target; there is now a real continuation-surface implication that uses the continuation hypotheses. Combining that implication with the new zero-data uniform instantiation gives the corresponding operational BKM corollary: under the BKM target assumption and positive viscosity, one gets an explicit global output for zero initial data. At the same layer, Lean now also contains the direct clause-level operational formulation `ExplicitBKMContinuationClause_implies_zeroGlobalOutput`, which applies one concrete BKM clause instance directly to the canonical zero slab witness and explicit envelope data. The envelope input is now itself proved directly in `zeroFiniteTimeRegularityWitness_has_BKMEnvelope`, with the explicit choice $\Omega(t) = 0$ and $B = 0$ on $[0, T]$, via reusable zero-envelope components `vorticityEnvelopeOn_zero` and `integrableVorticityEnvelopeOn_zero`. It now also contains the constant-envelope lemma `integrableVorticityEnvelopeOn_const`, which on nonnegative slabs $[0, T]$ turns the constant envelope value B into the explicit linear bound $T \cdot B$, and the corresponding bridge `uniformVorticityBoundUpTo_implies_constantBKMEnvelope` showing that a uniform slab vorticity bound already produces that exact finite-time constant-envelope package. The same file now also adds reusable time-restriction transport lemmas `vorticityEnvelopeOn_restrict` and `integrableVorticityEnvelopeOn_restrict`, and uses them to refactor the uniform-to-BKM restriction bridge `uniformVorticityBoundUpTo_implies_BKMEnvelope_restrict`. At the same layer it now also includes a clause-level bridge theorem `ExplicitBKMContinuationClause_implies_uniformVortic` and the witness-level restricted-envelope transport theorem `ExplicitFiniteTimeRegularityWitness.BKMEnvelope`. Lean now also packages the repaired-to-unrepaired BKM forgetful bridge on nonnegative slabs: `ExplicitFiniteEnergyBKMContinuationClause_implies_BKMContinuationClause_of_nonneg_horizon` and its target-to-fixed-horizon corollary `ExplicitFiniteEnergyBKMContinuationTarget_implies_BKMContinuat`. It then composes that forgetful step with the unrepaired BKM-to-uniform bridge to obtain the repaired-to-unrepaired uniform variant on the same slabs: `ExplicitFiniteEnergyBKMContinuationClause_implie` and its target-to-fixed-horizon corollary `ExplicitFiniteEnergyBKMContinuationTarget_implies_uniformVorti`. The repaired BKM clause now also has the direct restriction-based horizon API `ExplicitFiniteEnergyBKMContinua` and `ExplicitFiniteEnergyBKMContinuationClause_mono_horizon`, so the repaired BKM surface itself now transports to larger horizons by the same witness-restriction argument as the unrepaired clause. The bridge stack now also reaches the repaired uniform surface directly: `ExplicitBKMContinuationClause_implies_finiteEnergyUniformVorticityContinuationClause` records the same-horizon unrepaired-BKM to repaired-uniform step, while `ExplicitBKMContinuationClause_impli` and `ExplicitFiniteEnergyBKMContinuationClause_implies_finiteEnergyUniformVorticityContinuationC` package the corresponding shorter-slab to larger-slab transport there. At the theorem-surface level this now yields the direct bridge `ExplicitBKMContinuationTarget_implies_finiteEnergyUniformVorticityCon` in addition to the older unrepaired-uniform and repaired-uniform target exports. The repaired theorem-surface fixed-horizon BKM export now factors through that target-level forgetful theorem, rather than reopening the repaired/unrepaired BKM equivalence locally. The newest pass sharpens that witness-level transport by using envelope nonnegativity on the restricted slab, via `vorticityEnvelopeOn.integrable_restrict`, so the witness-level bound is the restricted integral itself rather than its absolute value. The same pass also adds the BKM clause horizon-monotonicity theorem `ExplicitBKMContinuationClause_mono_horizon`. The latest bottom-up addition to that file is a small additive algebra for concrete envelopes: `vorticityEnvelopeOn_add` combines two slab envelopes into an envelope for the summed field $u + v$, using the concrete curl linearity lemma `spatialVorticity_add`, and `integrableVorticityEnvelopeOn_add` adds the corresponding time-integral bounds. So the BKM layer now supports not just restriction and constantization of envelope data, but also explicit composition of envelope data under field addition. The newest pass makes that addition theorem witness-friendly: `vorticityEnvelopeOn_add_of_smooth` discharges the pointwise differentiability side condition from full space-time smoothness, and the same file now also con-

tains the direct sum-bridge corollaries `uniformVorticityBoundUpTo_add_implies_BKMEnvelope` and `uniformVorticityBoundUpTo_add_implies_constantBKMEnvelope`, which turn two smooth uniform slab bounds into explicit BKM-envelope data for the summed field. The next pass adds the rescaling/sign-flip side of the same bottom-up algebra: `vorticityEnvelopeOn_const_smul` transports a slab envelope along constant velocity rescaling $u \mapsto a \cdot u$ with the expected factor $|a|\,\Omega(t)$, while `integrableVorticityEnvelopeOn_const_mul` and `integrableVorticityEnvelopeOn_abs_const_smul` transport the time-integrability bound itself under nonnegative and absolute-value rescaling. The sign-flip corollary `vorticityEnvelopeOn_neg` then comes for free. So the concrete BKM layer now supports not only restriction, constantization, and addition, but also explicit scaling symmetry of envelope data. The newest pass then turns that algebra into subtraction theorems: `vorticityEnvelopeOn_sub_of_smooth` gives a concrete envelope for $u - v$ from smooth envelopes for u and v, and the bridge corollaries `uniformVorticityBoundUpTo_sub_implies_BKMEnvelope` and `uniformVorticityBoundUpTo_sub_implies_constantBKMEnvelope` package the same perturbative move at the uniform-to-BKM interface. So the bottom-up continuation algebra now handles both addition and subtraction of smooth slab data. The latest pass then bundles the envelope and integrability sides back together into reusable package theorems `BKMEnvelopeData_add_of_smooth` and `BKMEnvelopeData_sub_of_smooth`. These no longer just say that the pointwise bound or the time-integral bound can be transported separately; they package exact finite-slab BKM data for $u + v$ and $u - v$ in one theorem surface, which is a better mouth for later witness-level perturbative arguments. The newest pass then lifts those bundled theorems to the witness layer itself via `ExplicitFiniteTimeRegularityWitness.BKMEnvelope_add` and `ExplicitFiniteTimeRegularityWitness.BKME` if two finite slab witnesses each carry explicit BKM data on the same horizon, then the sum or difference of their velocity fields carries explicit BKM data too. That is still not a continuation proof, but it is a cleaner perturbative surface for future witness-level arguments than re-unpacking the smoothness and envelope data by hand each time. The newest pass adds the rescaling/sign-flip side at the same witness level: `ExplicitFiniteTimeRegularityWitness.BKMEnvelope_const_smul` transports explicit BKM data along constant velocity rescaling, and `ExplicitFiniteTimeRegularityWitness.BKMEnvelope_neg` specializes that to sign reversal. So the witness-level perturbative surface now supports addition, subtraction, scaling, and negation of the carried BKM packages, without claiming that the transformed velocity fields are themselves full Navier–Stokes witnesses. The newest pass then combines perturbation with horizon restriction directly: `ExplicitFiniteTimeRegularityWitness.BKMEnvelope_add_restrict` and `ExplicitFiniteTimeRegularityWitness.BKMEnvelope_sub_restrict` show that once the sum or difference field carries explicit BKM data on a slab $0 \le t \le T$, the same concrete package can be pushed down to any shorter slab $0 \le t \le T' \le T$ without rebuilding the envelope algebra by hand. The newest pass closes the same smaller-slab transport under scaling symmetry as well: `ExplicitFiniteTimeRegularityWitness.BKMEnvelope_const_smul_restrict` and `ExplicitFiniteTimeRegularityWitness.BKMEnvelope_neg_restrict` show that constant rescaling and sign flip commute with this witness-level restriction surface too. The newest pass packages the whole perturbative pattern into one direct API theorem: `BKMEnvelopeData_linearCombination_of_smooth` gives exact finite-slab BKM data for $a \bullet u + b \bullet v$, with the envelope $|a|\,\Omega + |b|\,\Omega'$ and the corresponding bound $|a|\,B + |b|\,B'$, and `ExplicitFiniteTimeRegularityWitness.BKMEnvelope_linearCombination_restrict` pushes that same linear-combination package down to any shorter slab $0 \le t \le T' \le T$. So later perturbative arguments can now consume one concrete linear-combination theorem instead of manually chaining scale, add, and restrict lemmas. The newest pass mirrors this on the uniform-vorticity side too: `uniformVorticityBoundUpTo_linearCombination` packages the same $|a|\,B + |b|\,B'$ bound for the concrete uniform criterion, and the bridge corollaries `uniformVorticityBoundUpTo_linearCombination_implies_BKMEnvelope` and `uniformVorticityBoundUpTo_li` push that uniform perturbative surface directly back into the BKM layer. The newest pass lifts this to

the witness layer on both sides: `ExplicitFiniteTimeRegularityWitness.uniformVorticityBound_linearCombi`
and `ExplicitFiniteTimeRegularityWitness.uniformVorticityBound_linearCombination_restrict`
give direct witness-level uniform control for linear combinations on the full slab and on shorter
slabs, while `uniformVorticityBoundUpTo_linearCombination_implies_BKMEnvelope_restrict`
now gives the corresponding field-level restricted BKM bridge directly, so the witness theo-
rem can sit on a concrete PDE lemma instead of restitching the conversion by hand. Then
`ExplicitFiniteTimeRegularityWitness.uniformVorticityBound_linearCombination_implies_BKMEnvelop`
turns that same smaller-slab witness-level uniform control directly into explicit BKM data. The
newest pass then adds the constant-envelope version of that bridge on nonnegative shorter slabs:
`uniformVorticityBoundUpTo_implies_constantBKMEnvelope_restrict` packages the restricted
constant envelope $\Omega(t) = B$, and `uniformVorticityBoundUpTo_linearCombination_implies_constantBKMEnvel`
does the same for the linear-combination surface itself, while `ExplicitFiniteTimeRegularityWitness.uniformVor`
lifts the same constant-envelope route to witness-level linear combinations. The generic BKM side now
has the matching restricted package too: `BKMEnvelopeData_linearCombination_restrict_of_smooth`
gives the shorter-slab linear-combination envelope data directly at the concrete PDE layer, and
`ExplicitFiniteTimeRegularityWitness.BKMEnvelope_linearCombination_restrict` is now just
the witness-level lift of that same bottom-up package. The continuation layer now consumes these re-
striction bridges honestly on actual witnesses: `ExplicitUniformVorticityContinuationClause_apply_of_unifo`
applies a smaller-slab uniform clause to a larger-slab witness after restriction, `ExplicitBKMContinuationClause_app`
does the same for explicit BKM data, and `ExplicitBKMContinuationClause_apply_of_uniformVorticityBound_`
feeds the restricted uniform-to-BKM bridge directly into the BKM clause. This keeps the continuation
side honest: it uses smaller-slab restrictions of genuine witnesses, rather than treating linear combina-
tions of unrelated witnesses as new Navier–Stokes solutions. The latest pass also makes this continua-
tion surface explicitly pressure-gauge invariant: `ExplicitFiniteTimeRegularityWitness.BKMEnvelope_addTimeO`
shows that the carried BKM package is unchanged after adding any smooth time-only pressure gauge
to the witness, and `ExplicitBKMContinuationClause_apply_of_BKMEnvelope_addTimeOnlyPressure`
shows that the BKM clause can therefore be applied after such a gauge change without any hidden
pressure normalization assumption. The same invariance now also survives smaller-slab restriction:
`ExplicitFiniteTimeRegularityWitness.BKMEnvelope_addTimeOnlyPressure_restrict` pushes
the unchanged BKM package down to any shorter slab after the gauge change, and `ExplicitBKMContinuationClaus`
turns that directly into a smaller-horizon operational clause theorem. Lean now also exposes the same
operational mouth starting only from a uniform vorticity bound: `ExplicitBKMContinuationClause_apply_of_uni`
applies the BKM clause after a same-horizon smooth time-only pressure-gauge change using the un-
changed uniform bound, while `ExplicitBKMContinuationClause_apply_of_uniformVorticityBound_addTimeOn`
does the same after restricting a larger-slab gauged witness to a shorter horizon. Lean now also instan-
tiates this on the concrete zero witness family: `zeroFiniteTimeRegularityWitness_addTimeOnlyPressure_restr`
shows that the gauged-and-restricted zero witness itself already carries the explicit shorter-slab zero
BKM envelope, and `ExplicitBKMContinuationClause_implies_zeroGlobalOutput_timeOnlyPressure`
shows that a smaller-slab BKM clause can therefore be applied directly to that concrete gauged zero
witness. Lean now also records the matching bridge-side obstruction on a genuinely time-dependent
whole-space ansatz. In `NavierStokesEnergyBKMBridge.lean`, `not_energyBKMBridge_kineticIntegrabilityOnS`
shows that any field whose initial slice matches transported full-drift heat-shear data with nonzero
transport speed already fails the kinetic-energy integrability input of the energy/BKM bridge at time
0. At the exact candidate level, `heatShearTransportFullDriftVelocityField_has_BKMEnvelope_but_not_ener`
packages the route-shaped consequence: on every nonnegative slab with nonnegative viscosity, the
transported full-drift heat-shear field carries an explicit constant BKM envelope $\Omega(t) = |ak|$,
but it still cannot enter the energy/BKM bridge because the whole-space kinetic-energy integra-
bility hypothesis is already false. So even after the new time-only-pressure and exact-witness

audits, the BKM route fails at the same honest whole-space energy gate rather than at a missing envelope construction. Lean now also factors this through one reusable bridge theorem: `not_energyBKMBridge_kineticIntegrabilityOnSlab_of_matchesInitialVelocity_of_not_finiteInitialK` says that on any nonnegative slab, once the prescribed datum already has infinite initial kinetic energy, every matched field is excluded from the energy/BKM bridge immediately at the time-zero integrability input. The constant, heat-shear, and transported full-drift obstructions are now just named corollaries of that generic whole-space energy fact.

2.24 Compatibility Frontier

In `FeffermanCompatibilityFrontier.lean`, Lean peels that top-down bridge one step further. It defines `AlmostFeffermanBridge`, which contains:

1. a candidate velocity/pressure pair;
2. the corresponding regularity certificate;
3. the corresponding dynamics certificate;
4. the corresponding energy certificate;

but deliberately omits vorticity-to-velocity compatibility.

Lean then proves that once a compatibility proof is supplied, the full `TopDownFeffermanBridge` and hence the local Fefferman-style clause follow immediately. It also adds package-level reduction theorems showing that `ColeHopfIdentityData`, `FiniteModeColeHopfData`, `ColeHopfKernelSemigroupData`, `FiniteModeColeHopfKernelData`, `ManuscriptTruncationPackage`, and `SharedApproximationPackage` all reduce to that same compatibility frontier.

So the live constructive NS bottleneck is now even sharper in Lean:

> once candidate fields and their certificates are fixed, the remaining theorem is exactly the compatibility of the current vorticity tendril with the chosen velocity field.

2.25 Weak Self-Model Realization

In `FeffermanWeakSelfModel.lean`, Lean now proves the top-down stack is not vacuous. The file defines a weak but nontrivial predicate kit where:

1. smoothness, momentum, incompressibility, and initial-condition predicates are left trivial;
2. the only nontrivial output-side predicate is a uniform pointwise bound on the velocity field;
3. compatibility means the chosen velocity field is literally the current vorticity tendril.

With that choice, Lean constructs a canonical self-model:

1. velocity is the tendril vorticity itself;
2. pressure is identically zero;
3. the tendril envelope supplies the boundedness certificate;
4. compatibility is tautological.

It then proves:

1. every `UniformVorticityTendril` realizes the weak local Fefferman-style clause;
2. the same realization can be seen directly through the compatibility frontier;
3. all current NS packages therefore already realize that weak clause.

So the constructive picture is now stronger:

> the top-down NS bridge is not merely a decomposition of obligations; for a weak but genuine target, the current grassroots tendril already closes the whole stack in Lean.

2.26 Seeded and Bounded Seeded Strengthenings

In `FeffermanSeededSelfModel.lean`, Lean strengthens the weak self-model in two concrete ways:

1. the pressure predicate is no longer trivial: it must be identically zero;
2. the initial-condition predicate is no longer trivial: velocity at the distinguished base time 1 must match the seed slice extracted from the tendril itself.

The file proves that every current NS package still realizes that stronger seeded clause by the same self candidate and the same compatibility proof.

In `FeffermanBoundedSeededSelfModel.lean`, Lean strengthens the target once more by making the velocity-side regularity predicate nontrivial too: it must satisfy the same uniform pointwise bound already carried by the tendril envelope. The same self candidate still closes the clause.

In `FeffermanPressureSeededSelfModel.lean`, Lean strengthens the target again by making one more dynamics field nontrivial: the momentum slot now also requires the pressure field to vanish. Since the canonical self candidate still has identically zero pressure, the same self candidate and the same compatibility proof still close the clause.

So the current top-down picture is sharper:

> the current NS tendril now closes not just a boundedness-only toy target, but a proof-carrying family of self-model targets with zero pressure, explicit initial slice, a nontrivial velocity-bound predicate, and now also a nontrivial momentum-slot pressure condition.

2.27 First Non-Self Compatibility Frontier

In `FeffermanScaledPressureSeededFrontier.lean`, Lean introduces the first genuinely non-self candidate: velocity is a scalar multiple of the current vorticity tendril, while pressure stays identically zero. Lean proves:

1. the same strengthened pressure-seeded target still has full regularity, dynamics, and energy certificates for this rescaled candidate;
2. therefore the same-route version now reaches an explicit `AlmostFeffermanBridge` with a nontrivial compatibility mouth;
3. a nearby descendant route with rescaled compatibility closes immediately;
4. the same-route mouth already closes in special regimes, including the trivial scaling $a = 1$ and the degenerate zero-vorticity case.

So the constructive picture is sharper again:

Lean now contains not only self-model closures, but also the first explicit non-self candidate whose certificates are complete and whose remaining obstacle is exactly a compatibility theorem.

In `FeffermanSeedBlendPressureFrontier.lean`, Lean pushes this one step further. The new non-self candidate has velocity

$$u(t,x) = a\,\omega(t,x) + b\,\omega(1,x),$$

mixing the live tendril with its seeded slice while keeping pressure zero. Lean proves:

1. the same strengthened pressure-seeded target still has complete regularity, dynamics, and energy certificates for this affine seed-blend candidate;
2. therefore the same-route version again reaches an explicit `AlmostFeffermanBridge` whose live mouth is compatibility;
3. a nearby descendant route with affine seed-blend compatibility closes immediately;
4. the same-route mouth already closes in special cases such as $(a,b) = (1,0)$ and the zero-vorticity regime.

So the top-down NS tendril now reaches a second, strictly richer non-self frontier in Lean.

In `FeffermanSaturatedPressureFrontier.lean`, Lean adds the first nonlinear non-self candidate:

$$u(t,x) = a \cdot \frac{\omega(t,x)}{1 + |\omega(t,x)|}.$$

The file proves:

1. the saturated profile still satisfies the strengthened pressure-seeded target's regularity, dynamics, and energy slots;
2. the same-route version again reaches an explicit `AlmostFeffermanBridge` with compatibility as the only remaining mouth;
3. a nearby descendant route with saturated compatibility closes immediately;
4. zero vorticity already collapses the same-route mouth.

So the top-down ladder now contains not just linear and affine non-self frontiers, but also a first nonlinear saturated frontier in Lean.

In `FeffermanFrozenGatePressureFrontier.lean`, Lean adds a more manuscript-shaped nonlinear non-self candidate:

$$u(t,x) = a \cdot \frac{|\omega(1,x)|}{1 + |\omega(1,x)|} \cdot \omega(t,x).$$

The live tendril is now modulated by a nonlinear gate computed from the frozen seed slice. The file proves:

1. this frozen-seed-gated candidate still satisfies the strengthened pressure-seeded target's regularity, dynamics, and energy slots;
2. the same-route version again reaches an explicit `AlmostFeffermanBridge` with compatibility as the only remaining mouth;

3. a nearby descendant route with frozen-gate compatibility closes immediately;

4. zero vorticity again collapses the same-route mouth.

So the top-down ladder now contains not just linear, affine, and pointwise nonlinear frontiers, but also a first frozen-seed-gated frontier in Lean.

In `FeffermanSeedLiveProfileFrontier.lean`, Lean then packages these frontiers into one reusable theorem family. The new structure `SeedLiveProfile` covers any pointwise candidate of the form

$$u(t,x) = \Phi(\omega(1,x), \omega(t,x))$$

under the single bound

$$|\Phi(\text{seed}, \text{live})| \le c\big(|\text{seed}| + |\text{live}|\big).$$

The file proves:

1. every such profile automatically carries the strengthened pressure-seeded target through regularity, dynamics, and energy;

2. the same-route version always reaches an explicit `AlmostFeffermanBridge` whose only remaining mouth is compatibility;

3. the descendant route with the exact profile-compatibility predicate closes immediately;

4. the scalar-rescaled, affine seed-blend, nonlinear saturated, and frozen-seed-gated candidates are recovered as concrete instances.

So the NS top-down lane now has not only several explicit non-self frontier examples, but also a reusable seed/live profile shell that subsumes them. Lean now also records the dual stationary closure regime already on that pointwise shell: if the profile returns the seeded slice pointwise and the concrete windowed vorticity tendril is stationary between the seeded slice and every live slice, then the same pressure-seeded clause closes directly there too.

In `FeffermanSeedLiveOperatorFrontier.lean`, Lean pushes this one step further to a genuinely non-pointwise shell. The new structure `SeedLiveOperator` allows the candidate at x to depend on the full seeded and live slices as functions, provided it satisfies a uniform envelope bound in terms of seeded/live slice bounds. The file proves:

1. every such operator automatically carries the strengthened pressure-seeded target through regularity, dynamics, and energy;

2. the same-route version again reaches an explicit `AlmostFeffermanBridge` whose only remaining mouth is compatibility;

3. the descendant route with exact operator compatibility closes immediately;

4. the pointwise `SeedLiveProfile` shell embeds into this operator shell, and a concrete anchored non-pointwise seed/live blend is realized as an instance.

So the top-down NS lane now reaches not only pointwise non-self frontier families, but also a first proof-carrying non-pointwise operator shell.

In `FeffermanSeedLiveSampleKernelFrontier.lean`, Lean specializes that operator shell to a finite sampled-kernel family. The new structure `SeedLiveSampleKernel` builds candidates of the form

$$u(t,x) = \sum_{i \in \kappa} \Big(a_i\, \omega(1, \alpha_i(x)) + b_i\, \omega(t, \beta_i(x)) \Big),$$

where the seed and live slices are sampled through finite anchor maps and the only global control is the total absolute kernel mass. The file proves:

1. every such finite sampled kernel automatically induces a `SeedLiveOperator`, so it inherits the same regularity, dynamics, and energy certificates for the strengthened pressure-seeded target;
2. the same-route version again reaches an explicit `AlmostFeffermanBridge` whose only remaining mouth is compatibility;
3. the descendant route with exact sampled-kernel compatibility closes immediately;
4. the new frontier is therefore a proof-carrying finite mixing shell sitting strictly between the generic pointwise profile family and the fully non-pointwise operator shell.

So the top-down NS lane now reaches not only general operator shells, but also a first finite sampled-kernel family that still lands at the same compatibility mouth.

In `FeffermanSeedLiveShiftKernelFrontier.lean`, Lean then imposes a first genuinely spatial structure on that sampled-kernel shell. The new structure `SeedLiveShiftKernel` specializes to translation-style kernels on an additive state space:

$$u(t,x) = \sum_{i\in\kappa}\Big(a_i\,\omega(1, x+s_i) + b_i\,\omega(t, x+\ell_i)\Big).$$

The file proves:

1. every such shift kernel canonically reduces to a `SeedLiveSampleKernel`, so it inherits the same regularity, dynamics, and energy certificates for the strengthened pressure-seeded target;
2. the same-route version again reaches an explicit `AlmostFeffermanBridge` whose only remaining mouth is compatibility;
3. the descendant route with exact shift-kernel compatibility closes immediately;
4. a concrete diagonal shift kernel is realized as an instance.

So the top-down NS lane now reaches not only arbitrary finite sampled mixtures, but also a first translation-kernel shell that begins to look like a genuine finite convolution front.

2.28 Concrete Windowed Descendant Route

The newest specialization step pushes the same top-down frontier back down to the actual concrete windowed heat package. In `WindowedCutoffApproximation.lean`, Lean proves that the genuinely windowed cutoff

$$u \mapsto u\cdot\big(\mathrm{smoothTransition}(cu+1) - \mathrm{smoothTransition}(cu)\big)$$

still passes through the current approximation shell. So leaving the stricter `EnergyProfile` shell is not yet a failure of truncation convergence or of the induced Cole–Hopf convergence.

In `WindowedColeHopfHeatPackage.lean`, Lean proves the quantitative window bound

$$|\chi_c(u)| \le c^{-1},$$

which is enough to instantiate the present heat-decayed Cole–Hopf package. So the current concrete route already carries a genuine windowed shared approximation package and a genuine uniform vorticity tendril.

In `WindowedColeHopfHeatFoundationFrontier.lean` and `WindowedColeHopfHeatSeededModel.lean`, Lean then fixes the actual candidate fields on that concrete route:

1. velocity is the package vorticity field;
2. pressure is identically zero;
3. the candidate closes a stronger seeded local target with zero pressure, a distinguished initial slice at time 1, and a uniform pointwise bound.

So the current compatibility frontier is no longer only an abstract theorem schema. It now sits on one explicit concrete windowed route.

2.29 Concrete Windowed Same-Route Rigidity

The next new files show that the concrete route already has theorem-level same-route rigidity.

In `WindowedColeHopfHeatScaledSeededFrontier.lean`, Lean packages the scaled descendant $u(t,x) = a\,\omega(t,x)$ on the concrete windowed route. The new file `WindowedColeHopfHeatScaledPressureSeededFro` then lifts that same concrete rescaled descendant to the stronger pressure-seeded target already used by the later non-self frontiers, producing explicit concrete `AlmostFeffermanBridge` and `TopDownFeffermanBridge` objects and the direct clause theorem `WeightedObservable.windowedColeHopfHeat_re` It now also records the two honest same-route closure regimes on that stronger concrete target: the rescaled descendant closes the original compatibility mouth when $a = 1$, and more generally it closes that mouth for any a when the underlying windowed vorticity tendril vanishes identically. Lean packages those special cases as direct clause theorems `WeightedObservable.windowedColeHopfHeat_realizes_scaled` and `WeightedObservable.windowedColeHopfHeat_realizes_scaled_pressure_seeded_clause_of_zero_vort` In `WindowedColeHopfHeatSelfCompatibilityObstruction.lean`, Lean proves that if this scaled descendant is forced back through the original self-compatibility mouth on a genuinely nonzero vorticity state, then the scaling must satisfy $a = 1$. Equivalently, if $a \neq 1$, then no bridge of that same-route type exists on a nonzero-vorticity state.

In `WindowedColeHopfHeatSeedBlendFrontier.lean`, Lean packages the affine seed-blend descendant

$$u(t,x) = a\,\omega(t,x) + b\,\omega(1,x).$$

The new file `WindowedColeHopfHeatFrozenGateFrontier.lean` adds the first seeded pointwise nonlinear modulation on the same concrete route:

$$u(t,x) = a\,\frac{|\omega(1,x)|}{1+|\omega(1,x)|}\,\omega(t,x).$$

So the windowed route no longer reaches only scalar and affine descendants before entering the finite sampled-kernel ladder.

The follow-up file `WindowedColeHopfHeatFrozenGateObstruction.lean` already makes that frozen-gate mouth rigid. Lean proves that if original self-compatibility holds at a nonzero live witness, then

$$a\,\frac{|\omega(1,y)|}{1+|\omega(1,y)|} = 1.$$

Since the frozen gate is always strictly less than 1, any such witness forces $a > 1$. So on any genuinely nonzero live state there can be no same-route bridge of this literal frozen-gate form when $a \leq 1$. Lean also proves that any two nonzero live witnesses on such a bridge must see the same frozen gate, so this first seeded nonlinear pointwise route is already rigid on the seed side too.

The new files `WindowedColeHopfHeatSeedLiveProfileFrontier.lean` and `WindowedColeHopfHeatSeedLiveP` then package the shared pointwise mouth directly on the same concrete windowed route:

$$u(t,x) = P(\omega(1,x), \omega(t,x)).$$

The first file proves that the scaled, affine seed-blend, saturated, and frozen-gate descendants are exact specializations of this one generic shell. It also now exposes the honest same-route theorem surface for that concrete shell itself: if the profile already equals the live input pointwise, then the original compatibility mouth closes directly; and if the underlying windowed vorticity tendril vanishes identically and $P(0,0) = 0$, then the same pressure-seeded clause closes again on that zero state. The same concrete file also exports the retained uniform-vorticity bound from the descendant-route `TopDownFeffermanBridge`. The second proves the shared witness law: at any nonzero live witness,

$$P(\omega(1,y),\omega(t,y)) = \omega(t,y).$$

So the pointwise search space is now structurally organized. These descendants are no longer independent repair guesses; they all have to close through the same concrete pointwise compatibility equation on the actual windowed package.

The next new files, `WindowedColeHopfHeatSeedLiveOperatorFrontier.lean` and `WindowedColeHopfHeatSee` lift this one step further to the full non-pointwise operator shell:

$$u(t,x) = O(\omega(1,\cdot),\omega(t,\cdot),x).$$

The first file proves that the pointwise profile shell, the finite sampled-kernel shell, and the finite shift-kernel shell are all exact specializations of this same concrete operator frontier on the windowed route. It also now exposes the honest same-route theorem surface for the operator shell itself: if the operator already returns the live slice pointwise, then the original compatibility mouth closes directly; and if the underlying windowed vorticity tendril vanishes identically while the operator sends the zero seeded/live pair to zero, then the same pressure-seeded clause closes again on that zero state. The same concrete file also exports the retained uniform-vorticity bound from the descendant-route `TopDownFeffermanBridge`. These concrete operator theorems are backed by the corresponding generic route lemmas in `FeffermanSeedLiveOperatorFrontier.lean`. Lean now also records the dual stationary closure regime on that shell: if the operator returns the seeded slice pointwise and the concrete windowed vorticity tendril is stationary between the seeded slice and every live slice, then the same pressure-seeded clause closes again. So the non-pointwise operator frontier now has three honest same-route closure mouths at theorem surface: live-return, seed-return under stationarity, and zero-vorticity collapse. The second proves the shared same-route law:

$$O(\omega(1,\cdot),\omega(t,\cdot),y) = \omega(t,y).$$

So the non-pointwise search space is now structurally compressed as well. The sampled-kernel and shift-kernel descendants no longer sit as separate repair stories; they all have to close through the same concrete operator equation on the actual windowed package.

The next new file, `WindowedColeHopfHeatSeedOnlyOperatorObstruction.lean`, turns that shared operator mouth into the first genuinely dynamical obstruction. If the chosen operator ignores the live slice entirely, then same-route closure forces

$$\omega(t_1,y) = \omega(t_2,y)$$

for every witness y and every pair of times t_1, t_2. So any witness where the concrete windowed route actually changes in time already rules out that whole seed-only non-pointwise family. This is the first explicit Lean result saying that a surviving repair must depend on the live state in a real way, not just on the frozen seeded slice. The same file immediately specializes this to finite sampled kernels with zero live coefficients, so the first finite non-pointwise shell already inherits this

live-dependence requirement. Since finite shift kernels are only sampled kernels with translation anchors, the same obstruction now bites the first genuinely spatial seed-only shell too.

The next file, `WindowedColeHopfHeatLiveOnlySampleKernelObstruction.lean`, proves the dual live-dependent finite-kernel obstruction. If the sampled kernel ignores the seeded slice entirely, then same-route closure reduces to

$$\sum_i a_i\,\omega(t,\alpha_i(y)) = \omega(t,y).$$

So whenever all sampled live witnesses vanish but the target live value does not, the corresponding live-only sampled-kernel bridge cannot exist. Since shift kernels are just sampled kernels with translation anchors, the same obstruction now bites the first genuinely spatial live-only shell as well.

In `WindowedColeHopfHeatSampleKernelFrontier.lean`, Lean then proves that this concrete affine descendant is exactly the diagonal one-sample instance of the generic sampled-kernel shell. So the concrete windowed route now connects downward into the abstract ladder not only through the generic seed-blend frontier, but also through the finite sampled-kernel layer. This matters because the older affine same-route obstruction now survives inside a first finite non-pointwise family, not only inside the pointwise seed-blend shell. The same concrete file now also exposes the direct same-route theorem surface inherited from the operator shell: arbitrary sampled kernels admit a direct clause theorem once self-compatibility is supplied, the live-return regime closes that route immediately, the zero-vorticity regime closes that same-route mouth automatically, and the same sampled-kernel route now also records the seed-return stationary regime inherited from the operator shell. The descendant-route bridge still exports the original windowed uniform-vorticity envelope. In addition, the diagonal specialization now re-exports that finite-kernel route directly on the older seed-blend target: Lean packages a direct clause theorem for the diagonal sampled kernel, a seed-blend-self-compatible same-route closure theorem, a direct live-endpoint closure theorem at ‘(a,b)=(1,0)‘, a direct stationary seed-return closure theorem at the ‘(a,b)=(0,1)‘ seed endpoint, and the corresponding zero-vorticity closure theorem, all with the seed-blend initial slice as the visible boundary data.

In `WindowedColeHopfHeatShiftKernelFrontier.lean`, Lean pushes this one rung further upward into genuinely spatial descendants. On additive-monoid state spaces, the anchored blend

$$u(t,x) = a\,\omega(t,x) + b\,\omega(1,x+s) + d\,\omega(t,x+s)$$

is proved to be exactly a finite translation-kernel instance. So the concrete windowed route now reaches the abstract shift-kernel layer too, not merely the one-point sampled-kernel shell. The same concrete shift-kernel file now also packages the direct same-route clause theorem under self-compatibility, the inherited live-return regime, the automatic zero-vorticity closure on that shell, the inherited seed-return stationary regime, and the retained uniform-vorticity bound on the descendant-route bridge. The anchored translation specialization now also re-exports those same-route theorems on the anchored three-term descendant itself: Lean gives direct anchored-shift clause wrappers from anchored self-compatibility, from any live-return regime that makes the specialized operator collapse back to the live slice, from any seed-return stationary regime that makes the specialized operator collapse back to the seeded slice, and from the zero-vorticity regime.

The next file, `WindowedColeHopfHeatShiftKernelObstruction.lean`, makes that spatial mouth rigid on the same-route side. Lean proves three witness-forced coefficient laws for the anchored descendant:

1. if the shifted initial and shifted live slices vanish while the unshifted live slice is nonzero, then $a = 1$;

2. if the unshifted live and shifted live slices vanish while the shifted initial slice is nonzero, then $b = 0$;

3. if the unshifted live and shifted initial slices vanish while the shifted live slice is nonzero, then $d = 0$.

The matching bridge-level impossibility corollaries are formalized too. So the first concrete spatial descendant is not merely packaged in Lean; its original self-compatibility mouth is already sharply constrained there.

The next widening step, `WindowedColeHopfHeatMaskedShiftKernelObstruction.lean`, lifts that same rigidity from one anchored spatial descendant to a finite family. In that family, every seed term uses the same shift s, while each live term either stays at x or moves to $x + s$ according to a Boolean mask. Lean proves that the whole family collapses back to the three-coefficient anchored spatial route with coefficients given by the total unshifted-live mass, total seed mass, and total shifted-live mass. So the same witness patterns now force, at finite-family level,

$$a_{\text{base}} = 1, \qquad b_{\text{seed}} = 0, \qquad d_{\text{shift}} = 0,$$

with the corresponding bridge-level impossibility corollaries when any of these three identities is violated. So the first concrete spatial same-route rigidity is no longer a one-instance fact; it already controls a nontrivial finite spatial shell.

The next widening step, `WindowedColeHopfHeatGeneralShiftKernelObstruction.lean`, removes the common-shift restriction and works directly at the arbitrary finite translation-kernel shell. Lean proves three generic collapse laws there:

1. if every seeded anchor except 0 vanishes, every live anchor except 0 vanishes, and the unshifted initial witness is nonzero, then the total zero-anchor mass

$$a_{\text{seed},0} + a_{\text{live},0}$$

   is forced to equal 1;

2. if every seeded anchor vanishes, every nonzero live anchor vanishes, and the unshifted live witness is nonzero, then the total zero-shift live mass is forced to equal 1;

3. if the live slice at the base point vanishes, every live anchor vanishes, every seeded anchor except a chosen shift u vanishes, and the u-shifted seeded witness is nonzero, then the total seeded mass at that anchor is forced to equal 0;

4. if the live slice at the base point vanishes, every seeded anchor vanishes, every live anchor except a chosen shift u vanishes, and the u-shifted live witness is nonzero, then the total live mass at that anchor is forced to equal 0.

So the linear same-route collapse is no longer confined to the anchored or masked common-shift subfamilies. It already lives at the full finite translation-kernel shell, stated in terms of anchorwise coefficient sums, with the matching bridge-level impossibility corollaries when any of those forced anchor masses is violated. In particular, the old identity-anchor sampled kernel law

$$a_{\text{seed}} + a_{\text{live}} = 1$$

is now visible as a full finite shift-kernel phenomenon at anchor 0, not as a separate low-level accident.

The follow-up file `WindowedColeHopfHeatGeneralShiftKernelMaskedSpecialization.lean` shows that the older masked common-shift shell is already recovered from this newer general linear theorem. In the nondegenerate case $s \neq 0$, Lean identifies the masked seed mass with the general seeded mass at anchor s, the masked unshifted-live mass with the general zero-shift live mass, and the masked shifted-live mass with the general live mass at anchor s. So the old masked coefficient laws and their impossibility corollaries are now derived from the full finite shift-kernel shell rather than reproved in parallel. The proof also keeps the exact caveat visible: when $s = 0$, the masked shifted-live terms collapse onto the zero anchor, so the old unshifted and shifted live masses are no longer separated by the general shell.

The companion file `WindowedColeHopfHeatGeneralShiftKernelIdentitySampleSpecialization.lean` does the same job for the older identity-anchor sampled-kernel sum law. It packages any identity-anchor sampled kernel as the zero-shift specialization of the general linear shell, transports the top-down bridge across that exact initial-slice and candidate equality, and then derives the old law

$$a_{\text{seed}} + a_{\text{live}} = 1$$

from the full finite shift-kernel theorem itself. So even the identity-anchor sampled-kernel sum rule is no longer a parallel low-rung argument; it is now an explicit specialization of the top linear shell.

The new file `WindowedColeHopfHeatSampleKernelObstruction.lean` makes that survival explicit at bridge level on the diagonal one-sample kernel. The same witness patterns that forced $a + b = 1$, $a = 1$, or $b = 0$ for the pointwise affine descendant now force the same identities for a same-route bridge whose candidate is presented through the sampled-kernel shell. So the first finite non-pointwise family does not evade the affine same-route rigidity; it inherits it.

The next file `WindowedColeHopfHeatIdentitySampleKernelObstruction.lean` strengthens that conclusion from the one-point diagonal case to every finite sampled kernel whose seed and live anchors are both the identity. Lean proves that such a family collapses back to the affine seed-blend route with coefficients given by the total live and seed masses. So finite repetition at the same point still does not create a new same-route escape corridor; it merely repackages the old affine obstruction in a larger finite shell. The latest pass also exports the two missing contrapositive blockers for this identity-anchor shell: under the zero-initial/nonzero-live witness, any same-route bridge forces total live mass $= 1$, and under the zero-live/nonzero-initial witness it forces total seed mass $= 0$. Therefore a finite identity-anchor sampled kernel with the wrong live or seed total is ruled out directly at the top-down bridge level, not just after manually unpacking the affine reduction.

In `WindowedColeHopfHeatSeedBlendObstruction.lean`, Lean proves bridge- level rigidity laws for the same-route mouth:

1. a nonzero initial slice forces $a + b = 1$;
2. a witness with zero initial slice but nonzero live vorticity forces $a = 1$;
3. a witness with zero live vorticity but nonzero initial slice forces $b = 0$;
4. violating any of these identities rules out the corresponding same-route bridge on the concrete windowed route.

So the NS crux has now moved one step forward: the issue is no longer merely that abstract non-self frontiers exist. The issue is that on the concrete windowed route, original same-route closure is already provably rigid.

The newest nonlinear step adds `WindowedColeHopfHeatSaturatedFrontier.lean`, which packages the concrete saturated descendant

$$u(t,x) = a\,\frac{\omega(t,x)}{1+|\omega(t,x)|}.$$

The corresponding obstruction file `WindowedColeHopfHeatSaturatedObstruction.lean` proves the first nonlinear same-route rigidity theorem on the concrete windowed route: if the saturated descendant is forced back through original self-compatibility at a nonzero vorticity witness, then

$$a = 1 + |\omega(t,x)|.$$

So in particular, on any nonzero-vorticity state there can be no same-route bridge of this saturated form when $a \le 1$. The new bridge-level upgrade is stronger: any same-route saturated bridge now forces all nonzero witnesses to have the same absolute vorticity magnitude. So two distinct nonzero magnitudes already rule out this nonlinear same-route closure entirely.

The next file `WindowedColeHopfHeatSaturatedSampleKernelObstruction.lean` proves that this nonlinear descendant already hits a boundary even at the diagonal one-sample sampled-kernel rung. If a sampled-kernel bridge of that form realizes the saturated candidate with $a \neq 0$, then all nonzero initial witnesses must have the same absolute vorticity magnitude. Therefore any state with two nonzero initial witnesses of distinct magnitudes rules out that same-route sampled-kernel realization altogether.

The new file `WindowedColeHopfHeatSaturatedIdentitySampleKernelObstruction.lean` pushes this one step beyond the diagonal case to every finite sampled kernel whose seed and live anchors are both the identity. Lean proves that if such a bridge realizes the saturated descendant with $a \neq 0$, then the same constant-magnitude rigidity still holds on the initial slice. So finite repetition at the same point does not open a nonlinear same-route escape corridor either.

The next file `WindowedColeHopfHeatSaturatedMaskedShiftKernelObstruction.lean` connects that nonlinear boundary to the widened finite spatial shell. Lean proves that if the saturated descendant is realized through the finite masked shift-kernel family and there is a witness with zero shifted initial slice, zero shifted live slice, and nonzero unshifted live slice, then the total unshifted-live mass satisfies

$$a_{\mathrm{base}}\big(1+|\omega(t,x)|\big) = a.$$

So if there are two such zero-shifted witnesses with distinct nonzero absolute vorticity magnitudes, the masked-shift shell cannot realize the saturated descendant at all. In other words, the nonlinear route already exits the widened finite spatial linear shell unless the surviving witnesses are constant-magnitude in exactly the same rigid way.

The next file `WindowedColeHopfHeatSaturatedShiftKernelObstruction.lean` pushes that boundary from the widened common-shift family to arbitrary finite translation kernels. Here the witness pattern is stated directly at the shell level: every seeded anchor must vanish, every nonzero live anchor must vanish, and the unshifted live witness must remain nonzero. Lean proves that under those hypotheses the total zero-shift live mass satisfies

$$a_0\big(1+|\omega(t,x)|\big) = a.$$

The same file now also proves the nonlinear zero-anchor total-mass analogue of the newer linear top-shell theorem: if every nonzero seeded anchor and every nonzero live anchor vanish while the unshifted initial witness remains nonzero, then

$$\big(a_{\mathrm{seed},0} + a_{\mathrm{live},0}\big)\big(1+|\omega(1,x)|\big) = a.$$

So the old identity-anchor sampled-kernel saturated law is no longer just a lower-rung fact in waiting; the full nonlinear shift-kernel shell now already sees the exact zero-anchor total-mass quantity that an identity specialization would need. So if two such witnesses survive with distinct nonzero absolute magnitudes, the entire finite translation-kernel shell cannot realize the saturated descendant. This is the first NS point where the nonlinear boundary is no longer tied to a single concrete shell encoding; it already lives at the general finite shift-kernel level. The same Lean file now also specializes that obstruction back to the first anchored translation shell, so the original concrete spatial descendant does not reopen this nonlinear corridor either.

The companion file `WindowedColeHopfHeatSaturatedShiftKernelConstantMagnitudeFrontier.lean` records the first honest constructive survivor inside that same full nonlinear shell. If the concrete windowed vorticity tendril has constant absolute magnitude

$$|\omega(t,x)| = r$$

for every t and x, then Lean proves the saturated candidate is exactly the zero-shift live-only finite shift kernel with coefficient

$$\frac{a}{1+r}.$$

So in that degenerate constant-magnitude regime there is an explicit top-down bridge from the full finite shift-kernel shell back to the nonlinear saturated descendant itself. This is useful because it shows the nonlinear story is no longer purely negative: the current shell really does have a narrow surviving corridor, and the surrounding obstruction theorems are now cutting around that corridor rather than merely around empty space. The same file now also exports that survivor directly at theorem surface: under the same constant-magnitude hypothesis, Lean packages the resulting saturated clause as a direct corollary rather than leaving the route only in existential bridge form.

The new file `WindowedColeHopfHeatSaturatedShiftKernelInvariantFrontier.lean` writes down the first honest widening of that positive corridor. Lean now proves that if the windowed vorticity tendril is stationary between the seeded slice and the live slice, invariant under every seed and live shift used by a finite kernel, and still has constant absolute magnitude r, then any finite shift kernel with total coefficient mass

$$a_{\text{seed}} + a_{\text{live}} = \frac{a}{1+r}$$

realizes the saturated candidate. So the nonlinear full shift-kernel shell is not merely nonempty at one hand-picked zero-shift point: it contains a whole invariant finite-kernel corridor. That is still highly rigid, but it is a real constructive shell theorem rather than just a surviving singleton. Lean now also exposes the same corridor directly on the nonlinear saturated target: once the finite kernel has the right total mass and the concrete windowed tendril is shift-invariant with constant magnitude, the saturated clause follows immediately via that finite shift-kernel route.

The companion file `WindowedColeHopfHeatSaturatedShiftKernelInvariantObstruction.lean` now closes the same shell at the candidate-equality level. Under the same stationarity and the same invariance under every seed and live shift used by the kernel, if the finite shift-kernel candidate coincides with the nonlinear saturated descendant and the total coefficient mass is nonzero, then every two nonzero witnesses must have the same absolute magnitude. So the full invariant finite-kernel corridor is no longer only constructive; it is already exact in the same constant-magnitude sense. This candidate-level statement now also has its own zero-mass audit: without assuming nonzero total mass, exact candidate equality forces either $a_{\text{seed}} + a_{\text{live}} = 0$ or the same equal-absolute-magnitude conclusion. Equivalently, if two nonzero witnesses have unequal magnitudes and the finite shift-kernel candidate still equals the nonlinear saturated descendant, then the total coefficient mass must be

zero. The zero-mass escape has now been audited one layer further. At every nonzero witness, exact candidate equality forces

$$(a_{\text{seed}} + a_{\text{live}})(1 + |\omega(t, x)|) = a.$$

Consequently, if the total coefficient mass is zero and $a \neq 0$, the invariant finite shift-kernel candidate cannot equal the saturated descendant at all. The remaining zero-mass corridor is therefore only the trivial zero-saturation case, not a nonlinear continuation repair. The latest strengthening removes a second possible workaround: exact equality at a nonzero witness also forces the sign of the finite kernel's total coefficient mass to match the sign of the saturated coefficient. In Lean, if $a > 0$, then candidate equality forces $0 < a_{\text{seed}} + a_{\text{live}}$; if $a < 0$, it forces $a_{\text{seed}} + a_{\text{live}} < 0$. Consequently a nonpositive total mass with positive saturated coefficient, or a nonnegative total mass with negative saturated coefficient, rules out candidate equality outright. The coefficient formula also gives a size audit: since $1 + |\omega(t, x)| \geq 1$, exact equality at a nonzero witness forces

$$|a_{\text{seed}} + a_{\text{live}}| \leq |a|.$$

Thus an invariant finite kernel whose total coefficient mass is larger than the saturated coefficient in absolute value cannot realize the nonlinear saturated candidate, even if its sign is correct.

That file now also contains the corresponding proof-carrying bridge-level corollary. If a top-down bridge through the same finite invariant shift-kernel shell has candidate equal to the nonlinear saturated descendant, then the same equal-absolute-magnitude conclusion follows; equivalently, unequal nonzero witness magnitudes rule out the existence of such a bridge. So the full invariant shell now has both sides in Lean: a constructive corridor and a matching $\neg\exists B$ obstruction. The bridge theorem has now also been strengthened into a zero-mass audit: without assuming nonzero total kernel mass, Lean proves that bridge equality forces either $a_{\text{seed}} + a_{\text{live}} = 0$ or the same equal-absolute-magnitude conclusion. Thus the nonzero-mass hypothesis is not hidden in the proof path; it is isolated as the only degenerate escape hatch for this same-route bridge. Lean also packages the corresponding classifier: if such a bridge does exist in the presence of two nonzero witnesses with unequal magnitudes, then its total shift-kernel coefficient mass must be zero. The same sign audit is now present at the bridge level: a bridge whose candidate is exactly the nonlinear saturated descendant forces positive total mass when $a > 0$ and negative total mass when $a < 0$. Therefore the wrong-sign aggregate-kernel cases are not merely candidate-level failures; they block the existence of a top-down bridge through this invariant finite shift-kernel shell. The bridge layer now has the same absolute-mass audit as well: any top-down bridge realizing the saturated descendant through this invariant finite shift-kernel shell forces $|a_{\text{seed}} + a_{\text{live}}| \leq |a|$. Therefore overlarge aggregate kernels are excluded at bridge level, not merely at the raw candidate-equality level. The absolute-mass audit has also been sharpened at both levels: if the total aggregate mass is nonzero and the comparison is made at a nonzero vorticity witness, exact equality forces the strict inequality

$$|a_{\text{seed}} + a_{\text{live}}| < |a|.$$

Consequently the old overlarge-mass obstruction now closes the equality boundary as well: under nonzero total mass, even $|a| \leq |a_{\text{seed}} + a_{\text{live}}|$ rules out exact candidate equality and rules out the corresponding top-down bridge. This does not contradict the constant-magnitude survivor; it says that the survivor must use the coefficient-matched smaller aggregate mass $a/(1 + |\omega|)$, not a same-magnitude aggregate coefficient. The shared regression file now pins both the candidate-level and bridge-level generic obstructions directly, including the zero-mass-or-equal-magnitude versions and the unequal-witness zero-mass classifiers, not only through the later anchored specialization, so future edits cannot silently retain the concrete anchored corollary while losing the full finite shift-kernel theorem surface. It also pins the new coefficient formula and the zero-mass/nonzero-saturation

nonexistence theorem at both candidate and bridge level, and now also pins the wrong-sign, overlarge-mass, and nonzero-mass equality-boundary nonexistence theorems. This makes the degenerate escape hatch explicit rather than tacit.

Current blocker status. The invariant finite shift-kernel audit is now a strong same-route rigidity theorem, not just a heuristic objection. It blocks any exact saturated candidate or bridge whose total mass is zero with $a \neq 0$, has the wrong sign, has nonzero total mass with $|a| \leq$ the aggregate absolute mass, or has nonzero total mass in the presence of two nonzero witnesses with unequal absolute vorticity magnitude. It is not yet a global $> 90\%$ blocker for the NS proof attempt, because the constant-magnitude, coefficient-matched corridor remains formally constructive, and a future analytic route could bypass this exact finite invariant shift-kernel bridge. To upgrade it, one must show that the manuscript's SG-Cole-Hopf continuation step necessarily factors through this exact invariant finite-kernel/saturated bridge on a nonconstant vorticity profile, or prove that every admissible repair still reduces to the same coefficient identity.

The next file `WindowedColeHopfHeatSaturatedAnchoredShiftInvariantFrontier.lean` pushes that positive corridor back down to the first concrete spatial anchored descendant

$$b\,\omega(t,x) + c\,\omega(1,x+s) + d\,\omega(t,x+s).$$

Under the same stationarity, the same invariance under $x \mapsto x+s$, the same constant-magnitude regime $|\omega(t,x)| = r$, and the concrete mass law

$$b + c + d = \frac{a}{1+r},$$

Lean proves that this anchored spatial candidate is exactly the nonlinear saturated descendant itself, and therefore yields a concrete top-down bridge through the anchored translation shell. So the new positive corridor is no longer merely abstract full-shell data: it already reaches a genuinely spatial windowed descendant. The same file now also exports the direct clause-level corollary on the saturated target, so the anchored spatial corridor is usable without unpacking an existential bridge witness first.

The companion file `WindowedColeHopfHeatSaturatedAnchoredShiftInvariantObstruction.lean` then proves that this concrete anchored corridor is already exact at the candidate-equality level once the total coefficient sum $b + c + d$ is nonzero. The same anchored file now also inherits the strict mass audit from the full finite shift-kernel layer: exact anchored candidate equality at a nonzero witness forces

$$|b + c + d| < |a|.$$

Thus the concrete anchored survivor cannot use a same-magnitude aggregate coefficient; if $|a| \leq |b+c+d|$ and $b + c + d \neq 0$, the anchored candidate cannot equal the saturated descendant at that nonzero witness. It now also proves the corresponding bridge-level obstruction for the proof-carrying anchored translation-kernel route. Under the same stationarity and the same invariance under $x \mapsto x+s$, if a top-down bridge through the anchored shift-kernel compatibility predicate has candidate equal to the nonlinear saturated descendant, then every two nonzero witnesses must have the same absolute magnitude. So unequal nonzero magnitudes do not just block the older sampled or masked shells, and do not merely block syntactic candidate equality; they already rule out a proof-carrying anchored spatial same-route bridge for this first concrete survivor itself. The bridge layer now carries the same equality-boundary mass blocker as well: under $b + c + d \neq 0$, $|a| \leq |b + c + d|$ rules out any anchored top-down bridge whose candidate is the nonlinear saturated descendant at a nonzero witness.

The follow-up file `WindowedColeHopfHeatSaturatedShiftKernelInvariantAnchoredSpecialization.lean` then closes the hierarchy gap: it shows that this anchored exactness theorem is not a parallel argument, but an explicit specialization of the newer full invariant finite shift-kernel exactness theorem through the anchored translation-kernel transport. So at the nonlinear exactness level the ladder is now cleaner too: full invariant shell first, anchored spatial theorem second.

The follow-up file `WindowedColeHopfHeatSaturatedShiftKernelMaskedSpecialization.lean` then shows that the earlier masked common-shift shell really is a special case of this newer general theorem. In the nondegenerate case $s \neq 0$, Lean identifies the old masked unshifted-live coefficient with the new shell-level zero-shift live mass, and then derives the old masked coefficient law and its impossibility corollary directly from the general finite shift-kernel obstruction. The proof also exposes the exact caveat: when $s = 0$, the "shifted" live terms collapse back onto the zero anchor, so the old masked base mass and the new shell-level zero-shift mass are no longer the same quantity.

The positive side now exists there too. The new file `WindowedColeHopfHeatSaturatedMaskedShiftKernelFron` packages the constructive corridor on that masked route directly at theorem surface: if the total coefficient mass of the masked common-shift kernel is

$$\frac{a}{1+r},$$

the concrete windowed vorticity tendril is invariant under the single common shift $x \mapsto x + s$, and $|\omega(t, x)| = r$, then the masked route already realizes the nonlinear saturated clause. With the additional stationarity hypothesis between the seeded and live slices, Lean upgrades the same route to a proof-carrying top-down bridge whose candidate is exactly the nonlinear saturated descendant. So the masked specialization is no longer only an obstruction corollary of the full shell: its constant-magnitude survivor is now exposed directly too.

The next file `WindowedColeHopfHeatSaturatedShiftKernelIdentitySampleSpecialization.lean` does the nonlinear identity-anchor job as well. It packages any identity-anchor sampled kernel as the zero-shift specialization of the full nonlinear shift-kernel shell, transports the top-down bridge across the exact initial-slice equality, and then derives the old identity-anchor saturated law

$$(a_{\text{seed}} + a_{\text{live}})(1 + |\omega(1, x)|) = a$$

and the corresponding constant-magnitude impossibility corollary directly from the top nonlinear shell. So even the older identity-anchor sampled-kernel saturated rigidity is no longer a parallel low-rung argument; it is now an explicit specialization of the full finite nonlinear shift-kernel theorem.

That route now also has the matching constructive theorem surface. The file `WindowedColeHopfHeatSaturatedI` proves that if both anchor families are the identity, the total seed/live mass is

$$a_{\text{seed}} + a_{\text{live}} = \frac{a}{1+r},$$

and the concrete windowed vorticity tendril has constant absolute magnitude $|\omega(t, x)| = r$, then the finite sampled-kernel route already realizes the nonlinear saturated initial slice itself. With the additional stationarity hypothesis between the seeded slice and the live slice, Lean upgrades that specialization to a proof-carrying sampled-kernel bridge whose candidate is exactly the nonlinear saturated descendant. So the identity-anchor shell now has both sides too: the old impossibility law remains, but the constant- magnitude corridor is now exposed directly at theorem surface rather than only through the ambient shift-kernel file.

The diagonal one-sample shell now has that same positive nonlinear surface too. The new file `WindowedColeHopfHeatSaturatedDiagonalSampleKernelFrontier.lean` simply instantiates the

identity-anchor corridor on `diagonalSampleKernel`: if

$$p + q = \frac{a}{1 + r}$$

and the concrete windowed vorticity tendril has constant absolute magnitude $|\omega(t, x)| = r$, then the diagonal sampled route already realizes the nonlinear saturated clause; with stationarity, Lean upgrades it to a proof-carrying diagonal sampled-kernel bridge whose candidate is exactly the nonlinear saturated descendant. So the old affine one-point shell now has a direct constructive saturated wrapper in addition to its exactness laws.

3 Finite-Mode Galerkin Toy ODE Cell

The pivot away from the shear and cancelled anti-profile cells is now recorded in the finite-dimensional projected ODE layer. In `NavierStokesFiniteModeGalerkinToy.lean`, the existing equilibrium audit already proves that the zero coefficient curve solves the projected ODE exactly when the forcing vector vanishes, and that a frozen coefficient curve solves it exactly when the projected RHS vanishes at that coefficient vector. The newest Lean pass adds two constructive non-frozen global solution families. For pure forcing coefficients, `finiteModeProjectedNSRHS_pureForcing` identifies the RHS with the forcing vector and `finiteModeAffineForcingCurve_solves` proves that $a_0 + tf$ solves $a' = f$ in every finite mode dimension. For decoupled diagonal-linear coefficients, `finiteModeProjectedNSRHS_diagonalLinear` identifies the RHS componentwise as $\lambda_i a_i$, and `finiteModeDiagonalLinearExpCurve_solves` proves that $a_i(t) = a_{0,i} \exp(\lambda_i t)$ solves $a_i' = \lambda_i a_i$. The next pass adds the inhomogeneous diagonal-affine family $a_i' = f_i + \lambda_i a_i$: Lean proves `finiteModeProjectedNSRHS_diagonalAffine` and the equilibrium-centered solution theorem `finiteModeDiagonalAffineEquilibriumCurve_solves`. If the supplied center satisfies $f_i + \lambda_i a_i^{\mathrm{eq}} = 0$, then

$$a_i(t) = a_i^{\mathrm{eq}} + (a_{0,i} - a_i^{\mathrm{eq}}) \exp(\lambda_i t)$$

solves the projected ODE globally, with no division or nonzero-rate side condition. The same file now also exposes a finite-mode energy identity: `finiteModeEnergy` reuses the existing finite square-sum `gamma`, and `finiteModeEnergy_hasDerivAt_of_projectedNS` proves that any coefficient path solving the projected ODE has energy derivative

$$\frac{d}{dt} \sum_i a_i(t)^2 = 2 \sum_i a_i(t)\, \mathrm{RHS}_i(a(t)).$$

For nonpositive diagonal-linear rates, `finiteModeDiagonalLinearEnergyRHS_nonpos` proves the corresponding dissipativity inequality, and `finiteModeDiagonalLinearExpCurve_energy_hasDerivAt` gives the exact energy derivative along the explicit exponential solution. These are ODE-level sanity anchors rather than PDE pressure repairs: they make the finite-mode Galerkin base constructive on simple families while leaving the hard reconstruction and Navier–Stokes closure obligations untouched.

4 Bounded-Energy Finite-Mode Schwartz Cell

The finite-mode linear and matrix-linear whole-space candidates were useful as obstruction tests, but their spatial profiles grow at infinity and Lean now formally rejects them from the finite-energy witness

class. The positive replacement is to keep the finite-dimensional mode idea while changing the spatial basis: `NavierStokesFiniteModeBoundedEnergy.lean` introduces `twoModeSchwartzVelocity`

$$u(t,x) = a(t)f(x) + b(t)g(x),$$

where $f, g \in \mathcal{S}(\mathbb{R}^3, \mathbb{R}^3)$, together with the initial slice `twoModeSchwartzInitialVelocity` and the pressure class `affineAddScalarSchwartzPressure`. Lean proves that this two-mode Schwartz ansatz is finite-energy in the precise local sense whenever the scalar amplitudes are uniformly bounded: `finiteInitialKineticEnergy_twoModeSchwartzInitialVelocity`, `MatchesInitialVelocity_twoModeSchwart` `boundedKineticEnergy_twoModeSchwartzVelocity_of_abs_le`, and `boundedKineticEnergyUpTo_twoModeSchw` The packaged theorem `twoModeSchwartzVelocity_finiteInitial_matches_boundedEnergyUpTo_of_abs_le` records the exact constructive status: bounded-energy finite-mode exploration is now possible without leaving the whole-space finite-energy domain.

The latest obstruction pass makes the old matrix-linear boundary exact at the initial-energy interface. In `NavierStokesFiniteModeEnergyObstruction.lean`, Lean proves `finiteInitialKineticEnergy_finite` the time-zero slice $x \mapsto g(0)Ax$ has finite initial kinetic energy if and only if either every entry of A is zero or $g(0) = 0$. Thus the whole-space matrix-linear ansatz cannot be repaired into the Clay-style finite-energy input class while it remains an active nonzero linear-growth profile; a repair must genuinely localize or decay the spatial modes.

The same boundary now holds at the finite-time witness layer on nonnegative slabs. Lean proves `nonempty_ExplicitFiniteTimeRegularityWitness_finiteModeLinearMatrixTimeVelocity_initial_iff`: when $0 \le T$, the witness type for initial datum $x \mapsto g(0)Ax$ is inhabited if and only if $A = 0$ entrywise or $g(0) = 0$. The forward direction uses the general theorem that every witness on a nonnegative slab has finite initial energy; the reverse direction is the canonical zero witness after the degenerate initial slice is identified with zero.

The slab-bounded-energy boundary is now exact as well. Lean proves `boundedKineticEnergyUpTo_finiteModeL` on a fixed slab $0 \le t \le T$, the velocity $u(t,x) = g(t)Ax$ satisfies the finite-time bounded-energy predicate if and only if either $A = 0$ entrywise or $g(t) = 0$ throughout that slab. Thus the obstruction is not an artifact of the time-zero input filter; any nonzero matrix-linear mode that is active even once on the slab is excluded directly by the finite-time energy slot.

The finite-time smooth PDE package now has the same exact boundary. Lean proves `finiteModeLinearMatrix_t` for symmetric trace-free A, smooth g, g', and a derivative relation, the affine-quadratic pressure package gives smooth (u,p), pointwise momentum, and incompressibility on the slab $0 \le t \le T$, while `boundedKineticEnergyUpTo` for u is equivalent to $A = 0$ entrywise or $g(t) = 0$ on the whole slab. Thus the slab theorem is not just a predicate-level obstruction; it classifies the energy slot inside the concrete finite-time PDE package.

The global bounded-energy boundary is exact too. Lean proves `boundedKineticEnergy_finiteModeLinearMatr` the whole-time predicate `boundedKineticEnergy` holds for $u(t,x) = g(t)Ax$ if and only if either $A = 0$ entrywise or $g(t) = 0$ for every time $t \in \mathbb{R}$. This rules out repairs that preserve a nonzero linear-growth matrix mode but try to move its activity outside the chosen slab; the global finite-energy slot sees every active time slice.

The exact boundary now reaches the whole smooth PDE package rather than only the velocity predicate. Lean proves `finiteModeLinearMatrix_timeDependent_global_smooth_momentum_incompressible_b` for symmetric trace-free A, smooth g, g', and the derivative relation between them, the affine-quadratic pressure construction yields smooth (u,p), pointwise momentum, and incompressibility at all times, while `boundedKineticEnergy` for the constructed u is equivalent to $A = 0$ entrywise or $g \equiv 0$. This is still not analytic existence progress; it is an exact incompatibility certificate for the polynomial matrix-linear finite-mode route.

The same file now also discharges the basic smoothness side conditions for this cell. The theorem `smoothSpaceTimeVelocity_twoModeSchwartzVelocity` proves that smooth scalar amplitudes a, b and Schwartz spatial profiles f, g give a smooth velocity field on $\mathbb{R} \times \mathbb{R}^3$, while `smoothSpaceTimePressure_affineAddScalarSchwartzPressure` packages the corresponding affine-plus-localized Schwartz pressure smoothness. The stronger package `twoModeSchwartzVelocity_finiteInitial_sm` therefore gives finite initial energy, smoothness, initial matching, and slab-bounded energy from smooth bounded amplitudes.

The next Lean pass also closes the incompressibility side condition for this finite-mode cell. The theorem `spatialDivergence_twoModeSchwartzVelocity` identifies the spatial divergence of $a(t)f(x)+b(t)g(x)$ as the amplitude-weighted sum of the initial divergences of f and g. Consequently `spatialDivergence_twoModeSchwartzVelocity_zero` proves that divergence-free Schwartz profiles produce an incompressible two-mode space-time velocity. The package `twoModeSchwartzVelocity_finiteInit` now records finite initial energy, smoothness, initial matching, incompressibility, and slab-bounded energy together.

This is not yet a Navier–Stokes existence theorem for arbitrary Schwartz data. The constructor `ExplicitFiniteTimeRegularityWitness.of_twoModeSchwartzVelocity_affineAddScalarSchwartzPressure` only produces a finite-time witness after the smoothness, momentum, incompressibility, and initial-condition hypotheses are supplied. Its value is that the energy slot is no longer the obstruction for this finite-mode family; the remaining burden is the real PDE closure of the chosen modes and pressure. The companion constructor `ExplicitFiniteTimeRegularityWitness.of_twoModeSchwartzVelocity_affineAdd` removes the smoothness hypotheses from that witness interface by deriving them from coefficient smoothness, leaving the pointwise momentum equation, incompressibility, initial match, viscosity sign, derivative witnesses, and amplitude bounds as the explicit remaining assumptions. The stronger constructor `ExplicitFiniteTimeRegularityWitness.of_twoModeSchwartzVelocity_affineAddScalarSchwartzPr` also derives incompressibility from the two profile-level divergence-free assumptions. At that point the only genuinely PDE-specific remaining assumption in this finite-mode witness lane is the pointwise momentum equation, together with bookkeeping data for the initial match, viscosity, scalar derivatives, and amplitude bounds.

The next time-dependent cleanup removes two of those bookkeeping obligations from the typed two-mode surface. Lean now proves `hasDerivAt_deriv_of_contDiff`, extracting $\mathrm{HasDerivAt}\, a\, (\mathrm{deriv}\, a(t))\, t$ from a smooth scalar amplitude, and `timeVelocityDerivative_twoModeSchwartzVelocity_deriv`, identifying the concrete time derivative of $a(t)f(x) + b(t)g(x)$ as $(\mathrm{deriv}\, a(t))f(x) + (\mathrm{deriv}\, b(t))g(x)$. The constructor `ExplicitFiniteTimeRegularityWitness.of_twoModeSchwartzVelocity_affineAddScalarSch` then uses those derivative witnesses and the canonical initial slice internally. Thus, on the smooth divergence-free bounded two-mode Schwartz surface, the external finite-time witness inputs have been narrowed to the pointwise momentum equation, viscosity sign, and amplitude bounds, plus the smoothness and profile divergence-free assumptions that define the cell.

The latest Lean pass closes that momentum slot for a concrete finite-energy member of the same Schwartz family. For the zero-amplitude specialization $a = b = 0$, Lean proves `twoModeSchwartzVelocity_zero_zero`, so the velocity is literally the zero velocity field independently of the Schwartz profiles f, g. It also proves `affineAddScalarSchwartzPressure_zero_timeOnly`, identifying the pressure class with a time-only gauge when the affine spatial coefficient and localized Schwartz amplitude vanish. Using the existing zero-velocity Navier–Stokes theorem, this yields `momentumEquation_zeroTwoModeSchwartzVelocity_affineTimeOnlyPressure`, an actual pointwise proof of

$$\partial_t u + (u \cdot \nabla)u + \nabla p = \nu\, \Delta u$$

inside the bounded-energy two-mode Schwartz construction. This is still the degenerate zero-

amplitude case rather than a nonzero finite-dimensional Schwartz heat closure, but it removes the previous external `heq` placeholder for at least one genuine Schwartz member of the cell.

The follow-up Lean pass moves the momentum work to genuinely nonzero two-mode amplitudes. Lean now proves `spatialLaplacian_twoModeSchwartzVelocity`, expanding Δu modewise, and `spatialConvection_twoModeSchwartzVelocity`, expanding $(u \cdot \nabla)u$ into the two self-interaction terms and the two mixed directional-derivative terms. Combining those expansions with the time derivative identity yields `momentumEquation_twoModeSchwartzVelocity_of_explicitClosure`: for arbitrary smooth amplitudes a, b, the pointwise Navier–Stokes momentum equation follows once the affine-plus-Schwartz pressure gradient closes the fully expanded finite-mode residual. The specialization `momentumEquation_oneOneTwoModeSchwartzVelocity_of_explicitClosure` records the constant nonzero-amplitude case $a(t) = b(t) = 1$, and `oneOneTwoModeSchwartzVelocity_nonzero_of_exists_pro` shows that this velocity is genuinely nonzero whenever the two profiles do not cancel everywhere. This still leaves the harder constructive task of producing a nonzero Schwartz profile pair and affine-plus-Schwartz pressure satisfying the explicit residual closure, but the PDE momentum proof is no longer the zero-field tautology and no longer hides the nonlinear term.

The next Lean pass specializes this nonzero-amplitude momentum result to the inviscid zero-pressure boundary case. The theorem `momentumEquation_oneOneTwoModeSchwartzVelocity_inviscid_zeroPres` states that, for $a(t) = b(t) = 1$ and $\nu = 0$, the pointwise equation

$$\partial_t u + (u \cdot \nabla)u + \nabla p = 0 \, \Delta u$$

with $p = 0$ follows directly from the vanishing of the explicit self-plus-mixed convection expansion. Lean also packages this as `ExplicitFiniteTimeRegularityWitness.of_oneOneTwoModeSchwartzVelocity_invi` which discharges smoothness, initial matching, incompressibility, bounded energy, amplitude bounds, and viscosity sign for the constant-one two-mode Schwartz cell. Finally, `oneOneTwoModeSchwartzVelocity_nonzero` records the nonzero side condition via $\exists x,\ f(x)+g(x) \neq 0$. This is deliberately weaker than a positive-viscosity Navier–Stokes construction: it isolates a usable exact nonzero finite-energy momentum witness at the Euler boundary while keeping the profile-level convection closure explicit.

The current Lean pass then returns to positive viscosity and constructs a concrete residual-closure baseline with nonzero Schwartz profiles. It imports the Mathlib smooth bump-function API and defines `nsUnitBumpSchwartzVelocity`, a compactly supported smooth first-coordinate bump converted to a Schwartz velocity profile. Lean proves `nsUnitBumpSchwartzVelocity_nonzero` by evaluating the bump at the origin. Pairing this profile with its negative gives the anti-profile identity `oneOneAntiProfileSchwartzVelocity_zero`; hence the total two-mode velocity cancels even though both spatial profiles are nonzero. Using the zero-velocity momentum theorem, Lean proves `momentumEquation_oneOneAntiProfileSchwartzVelocity_affineTimeOnlyPressure_of_posViscosity` and then expands it back to the profile-level residual in `explicitClosure_oneOneAntiProfileSchwartzVelocity_z` The resulting existential theorem `exists_nonzeroSchwartzProfiles_pressure_explicitClosure_fullViscousN` states that, for every $\nu > 0$, there exist nonzero Schwartz profiles f, g and a pressure field p (in the proof, $p = 0$) satisfying the fully expanded viscous residual closure. This is an exact Lean construction, but it is still a cancellation construction: it does not yet provide a non-cancelling positive-viscosity finite-energy flow. The same cancellation baseline has now been lifted to the time-indexed Schwartz pressure-slice route via `explicitClosure_oneOneAntiProfileSchwartzVelocity_schwartzPressureSlice_zero` and `twoModeSchwartzDivergenceFreeInitialVelocity_exhibits_BKMContinuationPremise_oneOneAntiProf` That confirms the repaired pressure-slice bridge has no hidden incompatibility with the zero-flow sanity case, while leaving the same non-cancelling positive-viscosity profile problem as the real finite-mode target. The latest equal-amplitude strengthening proves the same statement for every smooth scalar amplitude $a(t)$ with a uniform bound. This rules out a time-dependent equal-coefficient

workaround to the cancellation baseline, but does not change the current boundary: a useful positive-viscosity construction still needs profiles and coefficients whose total velocity does not cancel. The newest exactness result closes that loophole as a theorem: for a nonzero profile f, anti-profile zero flow is equivalent to equal amplitudes, and unequal amplitudes force the two-mode velocity to be nonzero somewhere. This does not prove that the resulting unequal-amplitude residual closure is impossible, but it does prevent the zero-flow cancellation baseline from being quietly rebranded as an unequal-amplitude construction. The witness-level sharpening then makes the BKM-premise artifact explicit: `twoModeSchwartzDivergenceFreeInitialVelocity_exhibits_BKMContinuationPremise_equalAmplitudeAnti` returns a finite-time witness whose velocity is literally 0 and whose pressure is the zero time-indexed Schwartz pressure slice. Thus the equal-amplitude anti-profile BKM branch cannot be used as evidence for a nonzero finite-mode flow; any such route must leave this classified zero-witness case. The follow-up `..._zeroWitness_zeroEnvelope` theorem sharpens this again: the same witness carries the zero vorticity envelope and zero integral budget. Hence the BKM datum on this branch is not merely bounded by a generic finite-mode estimate; it is exactly the zero-envelope baseline. Finally, `..._canonicalZeroWitness` identifies the finite-mode witness itself with `zeroFiniteTimeRegularityWitness`. The equal-amplitude branch has therefore collapsed to the canonical zero finite-time witness, not a distinct two-mode construction. The affine time-only pressure-gauge variant closes the adjacent repair loophole: `..._affineTimeOnlyPressure_zeroWitness_zeroEnvel` allows any smooth time-only gauge pressure in the affine-plus-Schwartz pressure class, but still returns zero velocity, the stated gauge pressure, the zero vorticity envelope, and zero integral budget. Hence this repair only changes the pressure gauge on the zero flow; it does not produce a nonzero finite-mode BKM branch. The follow-up `..._affineTimeOnlyPressure_canonicalGaugeWitness` identifies the witness itself with the canonical zero finite-time witness after `addTimeOnlyPressure`. This removes the possible repair that treats a time-only pressure gauge as a separate BKM construction. The pure spatial-affine pressure variant is now blocked rather than normalized: `spatialPressureGradient_affineAddScalarSchwartzPressure_spatialAffine` computes the gradient as $c(t)$, and `ExplicitFiniteTimeRegularityWitness.zeroVelocity_spatialAffinePressure_implies` forces $c(t) = 0$ on every certified time in any zero-velocity finite-time witness. Therefore a nonzero spatial-affine pressure coefficient cannot be used as a pressure-gauge workaround for the equal-amplitude anti-profile collapse. The BKM-layer theorem `BKMData_zeroVelocity_implies_initialVelocity_zero` closes the initial-data side: zero-velocity BKM data can only be packaged for zero initial data. Its contrapositive `not_exists_BKMData_zeroVelocity_of_initialVelocity_ne_zero` blocks any nonzero initial-data workaround. The theorem `BKMData_zeroVelocity_implies_spatialPressureGradient_zero` is the pressure classification statement: zero-velocity BKM data can only carry a pressure whose spatial gradient vanishes on the certified slab. Its contrapositive `not_exists_BKMData_zeroVelocity_of_exists_spat` is the arbitrary-pressure obstruction at one certified slab point, and `BKMData_zeroVelocity_implies_initialVeloc` records the combined necessary condition. The corresponding disjunctive no-go theorem `not_exists_BKMData_zeroV` prevents a repair from treating the initial datum and pressure-gradient requirements independently: either failure already rules out the zero-flow BKM package. The exact iff theorem `BKMData_zeroVelocity_iff_initialVelocity_zero_and_spatialPressureGradient_zero` also records the positive side for smooth pressure fields: if the initial datum is zero and the spatial pressure gradient vanishes on the certified slab, the zero finite-time witness carries the zero vorticity envelope and integral budget. Thus the BKM package neither repairs nor weakens the finite-time zero-flow witness classification. The time-only pressure survivor is also packaged exactly through `BKMData_zeroVelocity_timeOnlyPressure_iff_initialVelocity_zero`, with the positive theorem `BKMData_zeroVelocity_timeOnlyPressure` and the nonzero-data no-go `not_exists_BKMData_zeroVelocity_timeOnlyPressure_of_initialVelocity_ne_zero` as reusable corollaries. The spatial-affine pressure subclass now has the same exact treatment: `BKMData_zeroVelocity_spatial`

classifies it by zero initial data and slabwise vanishing of c, while `BKMData_zeroVelocity_spatialAffinePressure_` and `not_exists_BKMData_zeroVelocity_spatialAffinePressure_of_initialVelocity_ne_zero_or_exists_` record the positive and combined negative sides. Supplying a vorticity envelope and an integral budget on the same finite-time witness does not convert a genuine affine pressure force into gauge freedom. The full affine-plus-localized class now has the matching exact theorem `BKMData_zeroVelocity_affineAddScalarSchwartzPressure_iff_initialVelocity_zero_and_pressureGradi` It replaces the abstract condition $\nabla p = 0$ by the concrete balance $c(t) + \rho(t)\nabla q(x) = 0$. The no-go `not_exists_BKMData_zeroVelocity_affineAddScalarSchwartzPressure_of_initialVelocity_ne_zero_or_e` then closes the obvious affine-cancellation repair: a nonzero localized amplitude cannot be canceled by one vector $c(t)$ when the Schwartz pressure profile has nonconstant spatial gradient. The companion theorem `BKMData_zeroVelocity_affineAddScalarSchwartzPressure_implies_schwartzPressureGradient_c` states the residual burden directly: if such BKM data exists and $\rho(t) \neq 0$ on the certified slab, the fixed Schwartz pressure profile has the same spatial gradient at all spatial points on that time slice. Lean then proves `schwartzMap_gradient_constant_eq_zero`, so this residual burden collapses to zero gradient for any Schwartz profile. The derived BKM theorem `BKMData_zeroVelocity_affineAddScalarSchwartzPressure_implies_affineCoeff_zero_of_nonzeroAmplitu` therefore also forces $c(t) = 0$. The surviving nonzero-amplitude localized pressure term is only time-gauge-equivalent in the zero-flow BKM package; it cannot carry a spatial force. The raw equal-amplitude anti-profile momentum equation now has the same collapsed characterization through `momentumEquation_equalAmplitudeAntiProfileSchwartzVelocity_affineAddScalarSchwartzPressure_iff_` Thus the affine-plus-localized pressure ansatz cannot rescue the cancellation branch even before the finite-time BKM witness layer is introduced: the only surviving spatial pressure force is zero. The actual finite-time witness layer now has the arbitrary-pressure version of that boundary as well: `exists_ExplicitFiniteTimeRegularityWitness_equalAmplitudeAntiProfileInitialVelocity_velocity_pr` classifies witnesses whose velocity is the cancelled two-mode field and whose pressure is an arbitrary smooth pressure by zero spatial pressure gradient on the certified slab. Its no-go corollary `not_exists_ExplicitFiniteTimeRegularityWitness_equalAmplitudeAntiProfileInitialVelocity_velocit` rules out such a witness from a single nonzero slabwise pressure-gradient point. This is a witness-structure statement, not merely another conjunction of the repaired clause and BKM-data exact theorems. The internal-pressure variant `ExplicitFiniteTimeRegularityWitness_equalAmplitudeAntiProfileIniti` then removes the remaining bookkeeping escape hatch: if an actual witness itself carries the cancelled anti-profile velocity, then the pressure stored inside that witness has zero spatial gradient on the certified slab. The same-witness BKM-envelope theorem `ExplicitFiniteTimeRegularityWitness_equalAmplitudeAnt` adds the complementary positive fact: the very same witness admits the identically zero vorticity envelope with zero integral budget. Thus the BKM layer has no hidden analytic content on the cancelled branch; the pressure equation is the active boundary. The BKM-data normal-form theorem `BKMData_equalAmplitudeAntiProfileInitialVelocity_iff_zeroVorticityEnvelope_zeroBudget` now packages this as an exact equivalence: arbitrary envelope and budget data on the syntactic cancelled initial datum exists iff the same BKM witness can be presented with the zero vorticity envelope and zero integral budget. The finite-mode bridge has therefore stopped extending this cancelled same-witness zero-envelope cell and moved to a genuinely non-cancelling branch: `oneOneTwoModeSchwartzVelocity_nonzero_inviscidZeroPressure_BKMPremise_of_convectionClosure` assumes two divergence-free Schwartz profiles whose pointwise sum is nonzero somewhere and whose explicit inviscid convection closure vanishes. Lean then packages a nonzero constant-one two-profile velocity, its inviscid zero-pressure finite-time witness, and an internally supplied BKM envelope for that same witness. This is a positive finite-mode construction outside the cancelled anti-profile normal-form family. Lean now also records why this positive branch cannot simply be re-used as a positive-viscosity zero-pressure construction. The residual extraction theo-

rem `explicitClosure_oneOneTwoModeSchwartzVelocity_zeroPressure_of_momentumEquation` shows that any constant-one two-mode zero-pressure momentum equation must identify the explicit convection residual with $\nu(\Delta f+\Delta g)$. Consequently `not_momentumEquation_oneOneTwoMode_zeroPressure_of_in` proves the no-go form: if $\nu > 0$, the inviscid convection residual vanishes, and $\Delta f + \Delta g$ is nonzero at even one space-time point, then the zero-pressure positive-viscosity momentum equation is impossible. Thus the new non-cancelling anchor is genuinely an Euler-boundary sanity check; the positive-viscosity route must either supply a compensating pressure gradient or prove a strong Laplacian cancellation, not merely appeal to the inviscid BKM package. The follow-up theorem names this compensating-pressure boundary exactly: `momentumEquation_oneOneTwoModeSchwartzVelocity_iff_pressureG` proves that, under the same inviscid convection closure, the full momentum equation with an arbitrary pressure field is equivalent to

$$\nabla p(t,x) = \nu(\Delta f(t,x) + \Delta g(t,x)).$$

The companion no-go theorem `not_momentumEquation_oneOneTwoMode_of_inviscidClosure_pressureGradient` rules out the branch from a single failed pointwise equality. For the affine-plus-localized pressure class, Lean also proves the positive package `oneOneTwoModeSchwartzVelocity_nonzero_posViscosity_BKMPremise_of_` if the pressure ansatz satisfies that exact gradient balance, then the same nonzero constant-one two-mode velocity yields a positive-viscosity finite-time witness and an internally supplied BKM envelope. The pressure repair is therefore not left as a vague task; it has been reduced to one explicit pointwise gradient balance, with both the positive packaging theorem and the single-point mismatch obstruction formalized. For the concrete affine-plus-localized pressure family, this boundary has now been sharpened to a pair-difference obstruction. The theorem `not_momentumEquation_oneOneTwoMode_affineAddScalarSchwartzPressure_of_lapSum_pair_mismatch` uses

$$p(t,x) = \langle c(t), x\rangle + \pi(t) + \rho(t)q(x)$$

inside the arbitrary-pressure exact balance and subtracts the equation at two spatial points. The affine coefficient and time-only gauge cancel, so any successful structured repair must satisfy

$$\nu\big((\Delta f + \Delta g)(t,x) - (\Delta f + \Delta g)(t,y)\big) = \rho(t)\big(\nabla q(x) - \nabla q(y)\big)$$

for every pair of spatial points. A single pair mismatch blocks the full momentum equation. The corollary `not_momentumEquation_oneOneTwoMode_affineAddScalarSchwartzPressure_of_zeroLocalizedAmplitud` handles the inactive localized-pressure slice: when $\rho(t) = 0$ and $\nu > 0$, the summed viscous Laplacian residual must be spatially constant at that time. The new no-existence theorem `not_exists_momentumEquation_oneOneTwoMode_affineAddScalarSchwartzPressure_of_no_scalar_lapSum_p` removes the chosen-amplitude loophole: if at some time no scalar r can match the viscous pair-differences to the fixed q-gradient pair-differences, then no functions c, π, ρ in this pressure family can satisfy the full momentum equation. The two-pair theorem `not_exists_momentumEquation_oneOneTwoMode_affin` turns this into a finite witness: if two spatial pairs at one time have the same fixed q-gradient pair-difference but different summed viscous Laplacian residual pair-differences, then the shared scalar amplitude $\rho(t)$ cannot fit both pairs. Its repeated-gradient corollary `not_exists_momentumEquation_oneOneTwoMode_a` is the most concrete obstruction: when two points have the same ∇q-value but different summed viscous Laplacian residuals, the localized scalar amplitude cannot repair that branch at all. This obstruction is now composed at the BKM-data layer rather than stopping at the raw global momentum-equation surface. The theorem `not_exists_BKMData_oneOneTwoMode_affineAddScalarSchwartzPressure_of_s` assumes the two-pair witness time lies inside the certified slab $0 \le t \le T$. From a proposed finite-time witness it reads off the four slabwise momentum equations stored in the witness record, subtracts

the two pairs, and obtains the same contradiction: equal fixed q-gradient pair-differences would force equal summed Laplacian residual pair-differences. Hence adding a vorticity envelope and integral budget does not create a hidden repair channel for the affine-plus-one-Schwartz pressure family. The composition theorem `oneOneTwoModeSchwartzVelocity_inviscidBKM_and_not_exists_posViscosity_BKMData_af` pairs the positive non-cancelling inviscid finite-time BKM-premise package with this positive-viscosity no-data theorem. This is the route-level over-determination result: the nonzero constant-one two-mode branch exists as an inviscid BKM premise, but under the finite two-pair mismatch it cannot be promoted to positive-viscosity BKM data by choosing any affine coefficient, time gauge, or scalar localized Schwartz pressure amplitude. The finite-time witness layer now also has the corresponding pressure-family independent curl obstruction. In `NavierStokesUniformVorticityContinuationTarget.lean`, theorem `not_exists_ExplicitFiniteTimeRegularityWitness_velocity_of_momentumPressureResidual_vor` states that if the exact pressure residual

$$R_\nu(u) = \nu\Delta u - \partial_t u - (u\cdot\nabla)u$$

has nonzero curl at any point of the certified slab $0 \le t \le T$, then no finite-time smooth witness over that slab can have velocity u. The proof uses the witness's own slabwise momentum equation to identify ∇p with $R_\nu(u)$ on the time slice, then applies $\nabla\times\nabla p = 0$ for the stored smooth pressure. The BKM corollary `not_exists_BKMData_velocity_of_momentumPressureResidual_vorticity_ne_zero_on` in `NavierStokesBKMContinuationTarget.lean` strips off the extra vorticity-envelope and integral-budget package. Thus any future route that tries to repair a non-curl-free residual by changing the pressure family is blocked at the finite-time witness surface, not merely at a chosen ansatz. This obstruction has now been factored into positive necessary-condition lemmas. The finite-time witness theorem `ExplicitFiniteTimeRegularityWitness.spatialVorticity_momentumPressureResidual_zero_on` states directly that every certified witness has curl-free exact pressure residual on its slab, and `ExplicitFiniteTimeRegularityWitness.spatialVorticity_momentumPressureResidual_of_velocity_eq_z` transports that statement across an identified velocity field. The BKM layer adds matching lemmas `BKMData_momentumPressureResidual_vorticity_zero_on` and `BKMData_momentumPressureResidual_vorticit` These are the reusable interface points: a later concrete residual computation need only produce one nonzero curl point on the certified slab, and the finite-time/BKM witness package is ruled out without opening any pressure family-specific normal form. The same invariant now reaches the global-output surface. The definition `ExplicitConcreteNavierStokesGlobalOutputWithVelocity` packages the old explicit global output with a prescribed velocity field, and `ExplicitConcreteNavierStokesGlobalOutput_iff_` checks that the old proposition is exactly its existential closure. The lemma `ExplicitConcreteNavierStokesGlobal` then states the global analogue of the witness invariant: any prescribed velocity that is part of a smooth global output has curl-free exact pressure residual at every spacetime point. The no-go theorem `not_ExplicitConcreteNavierStokesGlobalOutputWithVelocity_of_momentumPressureResidual_vortic` therefore lets a single nonzero residual-curl computation rule out a candidate velocity as a global output, before considering BKM or continuation packaging. The concrete transported-shear residual-curl computation is now consumed at these target surfaces in `NavierStokesTransportedShearResidualCurlObstructi` The theorem `not_exists_ExplicitFiniteTimeRegularityWitness_velocity_amplitudeShearTransportVeloc` states that if an arbitrary transported amplitude $A(t)\sin(k(x_1 - bt))e_0 + be_1$ fails the sampled heat ODE $A'(t)+\nu k^2 A(t) = 0$ at a point of the certified slab and $k \neq 0$, then no finite-time witness on that slab can have this velocity. Its BKM companion `not_exists_BKMData_velocity_amplitudeShearTransportVeloc` shows that an envelope and integral budget do not repair the failure, and `not_ExplicitConcreteNavierStokesGlob` rules out the same prescribed velocity at the global-output surface. This is the first direct composition from the transported heat-ODE residual-curl calculation to all three exact target slots. Lean has now also moved one layer outside this structured pressure family. In `VectorCalculusR3.lean`, `spatialVorticity_pressureGradientVelocityField` proves the exact integrability condition for

arbitrary smooth pressure fields: the velocity field $u(t,x) = \nabla_x p(t,x)$ is curl-free at every point. The theorem `not_exists_smoothPressure_pressureGradientVelocityField_eq_of_spatialVorticity_ne_zero` therefore blocks any attempt to recast a nonzero-vorticity velocity as a pure smooth pressure gradient. The regression `linear_shear_unit_not_smooth_pressure_gradient_regression` instantiates this obstruction on the unit linear shear. This is a bottom-up vector-calculus obstruction, not a BKM or Sobolev continuation theorem; it rules out pressure-as-velocity repairs for nonzero-curl fields but does not by itself solve the residual-balancing problem for arbitrary pressure-forced momentum equations. The same file now lifts this from a pure representation no-go to a necessary condition for the actual momentum equation. The definition `momentumPressureResidual` names

$$R_\nu(u) = \nu\Delta u - \partial_t u - (u\cdot\nabla)u,$$

and `pressureGradientVelocityField_eq_momentumPressureResidual_of_momentumEquation` shows that any smooth-pressure solution must satisfy $\nabla p = R_\nu(u)$. Combining this with curl-of-gradient gives `spatialVorticity_momentumPressureResidual_of_momentumEquation` and the no-go theorem `not_exists_smoothPressure_momentumEquation_of_momentumPressureResidual_vorticity_ne_zero`. The concrete witness `not_exists_smoothPressure_momentumEquation_undampedUnitHeatShearVelocityField` uses the undamped unit heat-shear profile at unit viscosity: its pressure residual is the negative profile, whose curl is nonzero at the origin. Thus that velocity cannot be repaired by any smooth pressure, not merely by any previously named affine or localized pressure class. This has now been generalized from the unit example to the full wrong-viscosity heat-shear family. The theorem `momentumPressureResidual_heatShearVelocityField` computes

$$R_\mu(u_\nu) = (\nu-\mu)k^2 u_\nu$$

for the heat-shear profile evolved at viscosity ν but tested against a momentum equation at viscosity μ. Consequently `spatialVorticity_momentumPressureResidual_heatShearVelocityField_origin` identifies the origin curl as $(\mu-\nu)ak^3e_2$, and `not_exists_smoothPressure_momentumEquation_heatShearVelocit` rules out every smooth pressure when $\mu \neq \nu$, $a \neq 0$, and $k \neq 0$. This is a real family-level obstruction: a pressure cannot compensate for using the wrong viscous heat decay on any nondegenerate heat-shear mode. Lean now pushes this one level further, beyond fixed exponential heat decay. The general ansatz `amplitudeShearVelocityField` has the form

$$u(t,x) = A(t)\sin(kx_1)e_0.$$

The file proves the slice-congruence lemmas needed to reuse the heat-shear spatial calculations at a sampled time, and `momentumPressureResidual_amplitudeShearVelocityField_slice` computes

$$R_\mu(u)(t,\cdot) = -(A'(t) + \mu k^2 A(t))\sin(kx_1)e_0$$

whenever A has derivative $A'(t)$ at that time. The origin-curl theorem `spatialVorticity_momentumPressureResid` therefore identifies the obstruction as $(A'(t)+\mu k^2 A(t))ke_2$. Consequently `not_exists_smoothPressure_momentumE` rules out every smooth pressure whenever the sampled scalar heat ODE $A'(t) + \mu k^2 A(t) = 0$ fails and $k \neq 0$. This blocks arbitrary time-amplitude reparametrizations of the shear mode: the missing heat ODE cannot be hidden in pressure. Lean now proves the same point for the transported arbitrary-amplitude shear

$$u(t,x) = A(t)\sin(k(x_1 - bt))e_0 + be_1.$$

`momentumPressureResidual_amplitudeShearTransportVelocityField_slice` shows that the transport contribution cancels the phase derivative, leaving the same scalar heat-ODE residual $-(A'(t) +$

$\mu k^2 A(t)) \sin(k(x_1 - bt))e_0$. The phase-point curl theorem `spatialVorticity_momentumPressureResidual_amplitu` tests at $x_1 = bt$ and obtains $(A'(t) + \mu k^2 A(t))ke_2$. Consequently `not_exists_smoothPressure_momentumEquation_` rules out every smooth pressure whenever the transported scalar heat ODE fails and $k \neq 0$. Thus adding constant transport cannot hide a bad amplitude in the advection term. The negative restatement `not_exists_ExplicitFiniteTimeRegularityWitness_equalAmplitudeAntiProfileInitialVelocity_ve` rules out any such witness whose hidden pressure contains even one nonzero spatial-gradient point on that slab. Lean now also packages the finite-time BKM layer for the same anti-profile branch: `BKMData_equalAmplitudeAntiProfileSchwartzVelocity_affineAddScalarSchwartzPressure_iff_affineCo` This theorem says that asking for BKM data with the cancelled two-mode velocity is equivalent to the slabwise zero-flow pressure classification, not a weaker or independently repairable premise.

5 Current Boundary

The following are *not* yet formalized in Lean in the present NS lane:

1. the manuscript's specific chart-cutoff datum on SDiff^m;
2. the finite-mode truncation / approximation theorem actually needed for that datum;
3. instantiations of the new top-down bridge layers with real velocity and pressure candidates, real regularity and dynamics certificates, and a real vorticity-to-velocity compatibility theorem beyond the current self, scalar-rescaled, affine seed-blend, nonlinear saturated, frozen-seed-gated, bounded seed/live operator toy candidates, and the current concrete windowed descendants;
4. strengthening the current seeded / bounded-seeded self models toward something recognizably closer to the Clay/Fefferman predicates than pointwise bounds, zero pressure, and a distinguished initial slice;
5. the local hypotheses $(H1)$, $(H2)$;
6. the manuscript's claimed identity-only propagation lemma;
7. a non-cancelling positive-viscosity two-mode Schwartz profile and pressure satisfying the expanded residual closure;
8. the BKM-closing bridge for the full infinite-dimensional argument.

So the honest current status is:

> Lean has formalized the generic interface shell that a proof attempt would need, but not the manuscript-specific analytic instantiations.

6 Verification

The newest invariant-shell exactness and generic bridge regression step was checked directly with:

```
lake build Mettapedia.FluidDynamics.NavierStokes.\
WindowedColeHopfHeatSaturatedShiftKernelInvariantObstruction
lake build Mettapedia.FluidDynamics.NavierStokes.\
WindowedColeHopfHeatSaturatedAnchoredShiftInvariantFrontier
lake build Mettapedia.FluidDynamics.NavierStokes.\
```

```
WindowedColeHopfHeatSaturatedAnchoredShiftInvariantObstruction
lake build Mettapedia.FluidDynamics.NavierStokes.\
WindowedColeHopfHeatSaturatedShiftKernelInvariantAnchoredSpecialization
lake build Mettapedia.FluidDynamics.NavierStokes.ContinuationRegression
```

The newest bounded-energy finite-mode Schwartz step was checked directly with:

```
lake build Mettapedia.FluidDynamics.NavierStokes.\
NavierStokesFiniteModeBoundedEnergy
lake build Mettapedia.FluidDynamics.NavierStokes.\
NavierStokesFiniteModeBoundedEnergyRegression
```

The pressure-integrable energy/continuation bridge step was checked with the project wrapper:

```
/home/zarclaw/automation/aihub/scripts/mettapedia-lean-task.sh build \
  Mettapedia.FluidDynamics.NavierStokes.\
NavierStokesEnergyContinuationBridgeRegression
```

Focused dependency reports were written to:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T173145-energy-continuation-pressure-integrable.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T173145-energy-continuation-pressure-integrable-regression.txt
```

Both reports record zero direct and zero recursive sorries for the checked bridge and regression declarations. The Schwartz-pressure-slice strengthening was then checked with:

```
/home/zarclaw/automation/aihub/scripts/mettapedia-lean-task.sh build \
  Mettapedia.FluidDynamics.NavierStokes.\
NavierStokesEnergyContinuationBridgeRegression
```

with focused dependency reports:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T174530-energy-pressure-schwartz-slices.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T174531-energy-continuation-schwartz-pressure-slices.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T174531-energy-continuation-schwartz-pressure-slices-regression.txt
```

These reports record zero direct and zero recursive sorries for the pressure slice lemmas, the bridge theorem, and the regression theorem. The pressure-slice witness and finite-mode BKM-premise connection was then checked with:

```
/home/zarclaw/automation/aihub/scripts/mettapedia-lean-task.sh build \
  Mettapedia.FluidDynamics.NavierStokes.\
NavierStokesEnergyContinuationBridgeRegression
/home/zarclaw/automation/aihub/scripts/mettapedia-lean-task.sh build \
  Mettapedia.FluidDynamics.NavierStokes.\
NavierStokesFiniteModeSchwartzBKMBridgeRegression
```

with focused dependency reports:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T175704-energy-continuation-schwartz-pressure-slice-witness.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T175709-energy-continuation-schwartz-pressure-slice-witness-regression.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T175959-finite-mode-schwartz-bkm-pressure-slice-premise-visible.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T175859-finite-mode-schwartz-bkm-pressure-slice-premise-regression.txt
```

These reports record zero direct and zero recursive sorries for the new witness, finite-mode BKM-premise, and regression declarations. The explicit pressure-slice residual-closure bridge was then checked with:

```
/home/zarclaw/automation/aihub/scripts/mettapedia-lean-task.sh build \
  Mettapedia.FluidDynamics.NavierStokes.\
NavierStokesFiniteModeBoundedEnergyRegression
/home/zarclaw/automation/aihub/scripts/mettapedia-lean-task.sh build \
  Mettapedia.FluidDynamics.NavierStokes.\
NavierStokesFiniteModeSchwartzBKMBridgeRegression
```

with focused dependency reports:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T181127-finite-mode-pressure-slice-explicit-closure.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T181132-finite-mode-pressure-slice-explicit-closure-regression.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T181135-finite-mode-pressure-slice-explicit-closure-bkm.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T181139-finite-mode-pressure-slice-explicit-closure-bkm-regression.txt
```

These reports record zero direct and zero recursive sorries for the expanded momentum-closure theorem, its finite-mode BKM bridge, and their regressions. The pressure-slice anti-profile cancellation baseline was then checked with:

```
/home/zarclaw/automation/aihub/scripts/mettapedia-lean-task.sh build \
  Mettapedia.FluidDynamics.NavierStokes.\
NavierStokesFiniteModeBoundedEnergyRegression
/home/zarclaw/automation/aihub/scripts/mettapedia-lean-task.sh build \
  Mettapedia.FluidDynamics.NavierStokes.\
NavierStokesFiniteModeSchwartzBKMBridgeRegression
```

with focused dependency reports:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T182427-finite-mode-pressure-slice-antiprofile-closure.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T182432-finite-mode-pressure-slice-antiprofile-closure-regression.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T182436-finite-mode-pressure-slice-antiprofile-bkm.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T182441-finite-mode-pressure-slice-antiprofile-bkm-regression.txt
```

These reports record 49 bounded-energy declarations, 40 bounded-energy regression declarations, 4 finite-mode BKM declarations, and 7 finite-mode BKM regression declarations respectively, all with zero direct and zero recursive sorries. The equal-amplitude anti-profile strengthening was then checked with:

```
/home/zarclaw/automation/aihub/scripts/mettapedia-lean-task.sh build \
  Mettapedia.FluidDynamics.NavierStokes.\
NavierStokesFiniteModeBoundedEnergyRegression
/home/zarclaw/automation/aihub/scripts/mettapedia-lean-task.sh build \
  Mettapedia.FluidDynamics.NavierStokes.\
NavierStokesFiniteModeSchwartzBKMBridgeRegression
```

with focused dependency reports:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T183449-finite-mode-pressure-slice-equal-antiprofile-closure.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T183449-finite-mode-pressure-slice-equal-antiprofile-closure-regression.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T183449-finite-mode-pressure-slice-equal-antiprofile-bkm.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T183449-finite-mode-pressure-slice-equal-antiprofile-bkm-regression.txt
```

These reports record 53 bounded-energy declarations, 41 bounded-energy regression declarations, 5 finite-mode BKM declarations, and 8 finite-mode BKM regression declarations respectively, all with zero direct and zero recursive sorries. The anti-profile zero-flow exactness boundary was then checked with:

```
/home/zarclaw/automation/aihub/scripts/mettapedia-lean-task.sh build \
  Mettapedia.FluidDynamics.NavierStokes.\
NavierStokesFiniteModeBoundedEnergyRegression
```

with focused dependency reports:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T184624-finite-mode-antiprofile-zero-exactness.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T184624-finite-mode-antiprofile-zero-exactness-regression.txt
```

These reports record 56 bounded-energy declarations and 44 bounded-energy regression declarations respectively, all with zero direct and zero recursive sorries. The witness-level zero-flow audit was then checked with:

```
/home/zarclaw/automation/aihub/scripts/mettapedia-lean-task.sh build \
  Mettapedia.FluidDynamics.NavierStokes.\
NavierStokesFiniteModeSchwartzBKMBridgeRegression
```

with focused dependency reports:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T190035-finite-mode-equal-antiprofile-zero-witness.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T190034-finite-mode-equal-antiprofile-zero-witness-regression.txt
```

These reports record 6 finite-mode BKM bridge declarations and 9 bridge regression declarations respectively, all with zero direct and zero recursive sorries. The zero-envelope refinement was checked with the same focused build and with focused dependency reports:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T191246-finite-mode-equal-antiprofile-zero-envelope.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T191246-finite-mode-equal-antiprofile-zero-envelope-regression.txt
```

These reports record 7 finite-mode BKM bridge declarations and 10 bridge regression declarations respectively, again all with zero direct and zero recursive sorries. The canonical-zero-witness refinement was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T192509-finite-mode-equal-antiprofile-canonical-zero-witness.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T192509-finite-mode-equal-antiprofile-canonical-zero-witness-regression.txt
```

These reports record 8 finite-mode BKM bridge declarations and 11 bridge regression declarations respectively, all with zero direct and zero recursive sorries. The affine time-only pressure-gauge audit was checked with the lower finite-mode and BKM bridge builds and with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T194340-finite-mode-antiprofile-affine-time-only.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T194349-finite-mode-antiprofile-affine-time-only-regression.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T194355-finite-mode-antiprofile-affine-time-only-bkm.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T194359-finite-mode-antiprofile-affine-time-only-bkm-regression.txt
```

These reports record 57 bounded-energy declarations, 45 bounded-energy regression declarations, 9 BKM bridge declarations, and 12 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The canonical-gauge witness refinement was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T195624-finite-mode-antiprofile-affine-time-only-canonical-gauge.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T195628-finite-mode-antiprofile-affine-time-only-canonical-gauge-regression.txt
```

These reports record 10 BKM bridge declarations and 13 BKM bridge regression declarations respectively, again all with zero direct and zero recursive sorries. The spatial-affine pressure obstruction was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T201255-finite-mode-antiprofile-spatial-affine-pressure-obstruction.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T201303-finite-mode-antiprofile-spatial-affine-pressure-obstruction-regression.tx
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T201308-finite-mode-antiprofile-spatial-affine-pressure-obstruction-bkm-regressio
```

These reports record 62 bounded-energy declarations, 50 bounded-energy regression declarations, and 13 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The BKM-data lift was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T202220-finite-mode-bkm-spatial-affine-pressure-obstruction.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T202220-finite-mode-bkm-spatial-affine-pressure-obstruction-regression.txt
```

These reports record 11 BKM bridge declarations and 14 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The broader BKM pressure-gradient obstruction was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T203139-finite-mode-bkm-pressure-gradient-obstruction.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T203143-finite-mode-bkm-pressure-gradient-obstruction-regression.txt
```

These reports record 12 BKM bridge declarations and 15 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The zero-velocity BKM pressure-gradient classification was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T203905-finite-mode-bkm-zero-velocity-pressure-gradient-classification.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T203909-finite-mode-bkm-zero-velocity-pressure-gradient-classification-regression
```

These reports record 13 BKM bridge declarations and 16 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The zero-velocity BKM initial-data obstruction was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T204702-finite-mode-bkm-zero-velocity-initial-data-obstruction.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T204708-finite-mode-bkm-zero-velocity-initial-data-obstruction-regression.txt
```

These reports record 15 BKM bridge declarations and 18 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The combined zero-velocity BKM classification and disjunctive no-go theorem were checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T205836-finite-mode-bkm-zero-velocity-combined-classification-layout-fixed.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T205840-finite-mode-bkm-zero-velocity-combined-classification-layout-fixed-regres
```

These reports record 17 BKM bridge declarations and 20 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The exact zero-velocity BKM classification was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T210805-finite-mode-bkm-zero-velocity-exact-classification.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T210805-finite-mode-bkm-zero-velocity-exact-classification-regression.txt
```

These reports record 18 BKM bridge declarations and 22 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The time-only zero-velocity BKM gauge specialization was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T211701-finite-mode-bkm-time-only-classification.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T211701-finite-mode-bkm-time-only-classification-regression.txt
```

These reports record 21 BKM bridge declarations and 24 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The exact spatial-affine zero-velocity BKM classification was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T212437-finite-mode-bkm-spatial-affine-exact-classification-layout-fixed.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T212437-finite-mode-bkm-spatial-affine-exact-classification-layout-fixed-regressi
```

These reports record 24 BKM bridge declarations and 27 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The exact full affine-plus-localized zero-velocity BKM classification was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T213803-finite-mode-bkm-full-affine-localized-classification.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T213803-finite-mode-bkm-full-affine-localized-classification-regression.txt
```

These reports record 30 BKM bridge declarations and 33 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The nonzero-amplitude constant-gradient consequence was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T214530-finite-mode-bkm-nonzero-amplitude-constant-gradient.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T214533-finite-mode-bkm-nonzero-amplitude-constant-gradient-regression.txt
```

These reports record 31 BKM bridge declarations and 34 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The stronger Schwartz constant-gradient collapse and its BKM consequences were checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T215836-finite-mode-bkm-schwartz-constant-gradient-zero.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T215839-finite-mode-bkm-schwartz-constant-gradient-zero-regression.txt
```

These reports record 34 BKM bridge declarations and 37 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The exact collapsed zero-flow classification was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T220735-finite-mode-bkm-exact-collapsed-zero-flow.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T220735-finite-mode-bkm-exact-collapsed-zero-flow-regression.txt
```

These reports record 35 BKM bridge declarations and 38 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The equal-amplitude anti-profile momentum-equation collapse for the full affine-plus-localized pressure class was checked with the same focused build and with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T221833-finite-mode-antiprofile-full-affine-localized-momentum-collapse.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T221833-finite-mode-antiprofile-full-affine-localized-momentum-collapse-regressio
```

These reports record 36 BKM bridge declarations and 39 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The matching BKM-data collapse for the same anti-profile pressure family was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T222809-finite-mode-antiprofile-full-affine-localized-bkm-collapse.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T222809-finite-mode-antiprofile-full-affine-localized-bkm-collapse-regression.txt
```

These reports record 37 BKM bridge declarations and 40 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The direct no-go corollary for the same BKM-data surface was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T224212-finite-mode-antiprofile-full-affine-localized-bkm-no-go.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T224212-finite-mode-antiprofile-full-affine-localized-bkm-no-go-regression.txt
```

These reports record 38 BKM bridge declarations and 41 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The follow-up clause/data separation theorem for the same bad pressure surface was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T224952-finite-mode-antiprofile-bare-bkm-clause-mismatch.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T224955-finite-mode-antiprofile-bare-bkm-clause-mismatch-regression.txt
```

These reports record 39 BKM bridge declarations and 42 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The constructor-input separation theorem for the same anti-profile pressure family was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T225853-finite-mode-antiprofile-bkm-clause-momentum-input-mismatch.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T225857-finite-mode-antiprofile-bkm-clause-momentum-input-mismatch-regression.txt
```

These reports record 40 BKM bridge declarations and 43 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The matching data-level anti-profile initial-datum no-go was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T231542-finite-mode-antiprofile-initial-clause-data-mismatch.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T231542-finite-mode-antiprofile-initial-clause-data-mismatch-regression.txt
```

These reports record 41 BKM bridge declarations and 44 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The exact initial-datum BKM-data collapse was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T232245-finite-mode-antiprofile-initial-bkm-data-exact-collapse.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T232245-finite-mode-antiprofile-initial-bkm-data-exact-collapse-regression.txt
```

These reports record 43 BKM bridge declarations and 46 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The reusable pressure-independent anti-profile BKM-data normalizations were checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T233022-finite-mode-antiprofile-bkm-data-general-normalization.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T233022-finite-mode-antiprofile-bkm-data-general-normalization-regression.txt
```

These reports record 45 BKM bridge declarations and 48 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The arbitrary-pressure anti-profile BKM-data exact boundary and no-go forms were checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T233950-finite-mode-antiprofile-bkm-data-arbitrary-pressure.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T233950-finite-mode-antiprofile-bkm-data-arbitrary-pressure-regression.txt
```

These reports record 49 BKM bridge declarations and 52 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The corresponding arbitrary-pressure finite-energy clause/data separation was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T234907-finite-mode-antiprofile-arbitrary-pressure-clause-data-separation.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T234907-finite-mode-antiprofile-arbitrary-pressure-clause-data-separation-regressi
```

These reports record 51 BKM bridge declarations and 54 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The exact arbitrary-pressure repaired-clause/BKM-data boundary was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T235603-finite-mode-antiprofile-arbitrary-pressure-clause-data-exact.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T235606-finite-mode-antiprofile-arbitrary-pressure-clause-data-exact-regression.tx
```

These reports record 53 BKM bridge declarations and 56 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The arbitrary-pressure raw momentum-equation boundary and constructor-input separation were checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T001726-finite-mode-antiprofile-arbitrary-pressure-momentum-boundary.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T001744-finite-mode-antiprofile-arbitrary-pressure-momentum-boundary-regression.tx
```

These reports record 57 BKM bridge declarations and 60 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The exact arbitrary-pressure finite-energy clause/momentum-equation boundary was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T003021-finite-mode-antiprofile-arbitrary-pressure-momentum-exact-boundary.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T003025-finite-mode-antiprofile-arbitrary-pressure-momentum-exact-boundary-regress
```

These reports record 58 BKM bridge declarations and 61 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The exact arbitrary-pressure finite-energy clause/BKM-data/momentum-equation boundary was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T003632-finite-mode-antiprofile-arbitrary-pressure-clause-data-momentum-exact.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T003636-finite-mode-antiprofile-arbitrary-pressure-clause-data-momentum-exact-regr
```

These reports record 59 BKM bridge declarations and 62 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The arbitrary-pressure finite-time witness boundary was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T004723-finite-mode-antiprofile-arbitrary-pressure-witness-exact.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T004723-finite-mode-antiprofile-arbitrary-pressure-witness-exact-regression.txt
```

These reports record 61 BKM bridge declarations and 64 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The internal-pressure finite-time witness obstruction was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T005328-finite-mode-antiprofile-internal-witness-pressure-zero.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T005328-finite-mode-antiprofile-internal-witness-pressure-zero-regression.txt
```

These reports record 62 BKM bridge declarations and 65 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The same-witness zero BKM-envelope theorem was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T010739-finite-mode-antiprofile-same-witness-zero-bkm-envelope.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T010738-finite-mode-antiprofile-same-witness-zero-bkm-envelope-regression.txt
```

These reports record 64 BKM bridge declarations and 67 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The zero-budget BKM-data normal form was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T011620-finite-mode-antiprofile-bkm-zero-budget-normal-form.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T011620-finite-mode-antiprofile-bkm-zero-budget-normal-form-regression.txt
```

These reports record 65 BKM bridge declarations and 68 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The non-cancelling constant-one inviscid zero-pressure BKM premise was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T012731-finite-mode-one-one-inviscid-nonzero-bkm-premise.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T012731-finite-mode-one-one-inviscid-nonzero-bkm-premise-regression.txt
```

These reports record 66 BKM bridge declarations and 69 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The positive-viscosity zero-pressure obstruction for the non-cancelling constant-one two-mode branch was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T013850-finite-mode-one-one-positive-viscosity-zero-pressure-obstruction-20260504
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T013854-finite-mode-one-one-positive-viscosity-zero-pressure-obstruction-regressi
```

These reports record 64 bounded-energy declarations and 52 bounded-energy regression declarations respectively, all with zero direct and zero recursive sorries. The exact pressure-gradient balance and positive-viscosity pressure-repaired BKM package for that same non-cancelling branch were checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T014937-finite-mode-one-one-pressure-gradient-lapsum-balance-20260504T234926Z.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T014937-finite-mode-one-one-pressure-gradient-lapsum-balance-regression-20260504T
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T014937-finite-mode-one-one-positive-viscosity-pressure-repaired-bkm-20260504T234
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T014937-finite-mode-one-one-positive-viscosity-pressure-repaired-bkm-regression-2
```

These reports record 66 bounded-energy declarations, 54 bounded-energy regression declarations, 67 BKM bridge declarations, and 70 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The concrete affine-plus-localized pair-difference obstruction for that pressure-repaired non-cancelling branch was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T020140-finite-mode-affine-pressure-pair-obstruction.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T020140-finite-mode-affine-pressure-pair-obstruction-regression.txt
```

These reports record 69 BKM bridge declarations and 72 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The scalar-amplitude-free and repeated-gradient affine-pressure obstructions for the same non-cancelling branch were checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T021801-finite-mode-affine-pressure-no-scalar-pair-fit.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T021801-finite-mode-affine-pressure-no-scalar-pair-fit-regression.txt
```

These reports record 71 BKM bridge declarations and 74 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The finite two-pair obstruction for the same scalar-amplitude repair route was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T022633-finite-mode-affine-pressure-two-pair-obstruction.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T022637-finite-mode-affine-pressure-two-pair-obstruction-regression.txt
```

These reports record 72 BKM bridge declarations and 75 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The BKM-data lift and route-level multi-mode composition were checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T045056-multi-mode-overdetermination-composition.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T045057-multi-mode-overdetermination-composition-regression.txt
```

These reports record 74 BKM bridge declarations and 76 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The pressure-family-independent finite-time residual-curl witness obstruction was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T050309-witness-residual-curl-uniform.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T050404-witness-residual-curl-uniform-regression.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T050356-witness-residual-curl-bkm.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T050408-witness-residual-curl-bkm-regression.txt
```

These reports record 532 uniform-continuation declarations, 66 uniform regression declarations, 253 BKM-continuation declarations, and 5 BKM regression declarations respectively, all with zero direct and zero recursive sorries. The positive residual-curl necessary-condition factorization was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T052004-witness-residual-curl-zero-uniform.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T052036-witness-residual-curl-zero-uniform-regression.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T052039-witness-residual-curl-zero-bkm.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T052043-witness-residual-curl-zero-bkm-regression.txt
```

These reports record 535 uniform-continuation declarations, 68 uniform regression declarations, 255 BKM-continuation declarations, and 7 BKM regression declarations respectively, all with zero direct and zero recursive sorries. The prescribed-velocity global-output residual-curl layer was checked with:

/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T052826-global-output-residual-curl-velocity.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T052859-global-output-residual-curl-velocity-regression.txt

These reports record 539 uniform-continuation declarations and 71 uniform regression declarations respectively, both with zero direct and zero recursive sorries. The transported-shear residual-curl target composition was checked with:

/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T053749-transported-shear-residual-curl-target-obstruction.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T053749-transported-shear-residual-curl-target-obstruction-regression.txt

These reports record 3 obstruction declarations and 3 regression declarations respectively, all with zero direct and zero recursive sorries. The internal-pressure no-go form was checked with:

/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T010113-finite-mode-antiprofile-internal-witness-pressure-no-go.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T010114-finite-mode-antiprofile-internal-witness-pressure-no-go-regression.txt

These reports record 63 BKM bridge declarations and 66 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The concrete affine-plus-localized repaired-clause/BKM-data exact boundary was checked with:

/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T000829-finite-mode-antiprofile-affine-localized-clause-data-exact.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T000835-finite-mode-antiprofile-affine-localized-clause-data-exact-regression.txt

These reports record 55 BKM bridge declarations and 58 BKM bridge regression declarations respectively, all with zero direct and zero recursive sorries. The exact BKM-clause zero-normalization for the equal-amplitude anti-profile initial datum was checked with:

/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T230610-finite-mode-antiprofile-bkm-clause-zero-normalization-v2.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260504T230614-finite-mode-antiprofile-bkm-clause-zero-normalization-regression-v2.txt

These reports record 4 BKM clause declarations and 6 BKM clause regression declarations respectively, all with zero direct and zero recursive sorries. The concrete wrong-viscosity heat-shear residual obstruction was checked with:

/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T032418-wrong-viscosity-heat-shear-residual.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T032424-wrong-viscosity-heat-shear-residual-regression.txt

These reports record 287 vector-calculus declarations and 18 vector-calculus regression declarations respectively, all with zero direct and zero recursive sorries. The arbitrary time-amplitude heat-ODE pressure obstruction was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T034413-amplitude-shear-heat-ode-pressure-obstruction.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T034419-amplitude-shear-heat-ode-pressure-obstruction-regression.txt
```

These reports record 305 vector-calculus declarations and 19 vector-calculus regression declarations respectively, all with zero direct and zero recursive sorries. The transported arbitrary-amplitude heat-ODE pressure obstruction was checked with:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T040329-amplitude-transport-shear-heat-ode-pressure-obstruction.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T040335-amplitude-transport-shear-heat-ode-pressure-obstruction-regression.txt
```

These reports record 317 vector-calculus declarations and 20 vector-calculus regression declarations respectively, all with zero direct and zero recursive sorries. The finite-mode Galerkin ODE pivot was checked with:

```
/home/zarclaw/automation/aihub/scripts/mettapedia-lean-task.sh build \
  Mettapedia.FluidDynamics.NavierStokes.\
NavierStokesFiniteModeGalerkinToyRegression
```

with focused dependency reports:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T041538-finite-mode-galerkin-pure-forcing-diagonal-linear.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T041541-finite-mode-galerkin-pure-forcing-diagonal-linear-regression.txt
```

These reports record 28 finite-mode Galerkin toy declarations and 10 regression declarations respectively, all with zero direct and zero recursive sorries. The equilibrium-centered diagonal-affine extension was then checked with the same focused regression build and the additional dependency reports:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T042629-finite-mode-galerkin-diagonal-affine-equilibrium.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T042632-finite-mode-galerkin-diagonal-affine-equilibrium-regression.txt
```

These reports record 34 finite-mode Galerkin toy declarations and 13 regression declarations respectively, all with zero direct and zero recursive sorries. The finite-mode coefficient-energy identity and diagonal-linear dissipativity extension was then checked with the same focused regression build and the additional dependency reports:

```
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T043703-finite-mode-galerkin-energy-identity.txt
/home/zarclaw/automation/aihub/state/ns/deps/\
DEPS-20260505T043706-finite-mode-galerkin-energy-identity-regression.txt
```

These reports record 42 finite-mode Galerkin toy declarations and 16 regression declarations respectively, all with zero direct and zero recursive sorries. The full active `NavierStokes` lane was then rechecked with the grouped build command

```
mods=$(find Mettapedia/FluidDynamics/NavierStokes -maxdepth 1 -name '*.lean' \
  | sort | sed 's|/|.|g; s|\.lean$||')
lake build $mods
```

7 Next Lean Target

The natural next formal step is no longer more prose. It is to keep pushing top-down from the target:

1. strengthen the proof-carrying target again by making at least one more regularity or dynamics predicate genuinely nontrivial while keeping the clause closable by current packages;
2. push the concrete windowed route beyond same-route rigidity and toward a real velocity-generation or pressure-generation theorem on that fixed candidate;
3. reconnect whichever concrete descendant survives that test to the mode-space chart / approximation shells.